

- MASTER_INTERVIEW.md
- README_ZH.md
- Kafka秒杀优惠券改造清单.md
- Kafka秒杀链路后续优化实施计划.md
- Kafka秒杀链路面试准备文档.md
- Kafka秒杀链路面试题.md
- S1-shop查询基线压测报告.md
- 二级缓存改造后验证报告.md
- 二级缓存改造方案.md
- 二级缓存改造清单.md
- 压测对比测试计划.md
- 本地缓存架构图.md
- 秒杀并发与Kafka面试话术-1_3_8分钟版.md
- 秒杀链路测试结果记录.md
- 自动化测试平台实现方案-面试版.md
- 自动化测试平台面试讲稿-校招版.md
- 自动化测试平台面试讲稿.md
- 自动化测试平台面试速记卡-校招.md
- 限流设计方案.md
- 项目专题面试题-校招版-参考回答.md
- 项目专题面试题-校招版.md
- 项目技术问答.md
- 项目技术问答详解.md
- 项目面试总复习文档.md
- 项目面试题库-详细版.md

MASTER_INTERVIEW.md

hm-dianping 项目面试总文档

更新时间：2026-05-05

说明：这份文档不是把旧资料机械拼接，而是基于当前项目代码、配置和已落地优化重新生成的单文件总面试文档。

后续如果需要继续补充，建议优先维护这一份。

1. 项目定位

hm-dianping 是一个基于 Spring Boot + Redis + MySQL + Kafka + Nginx 的本地生活服务类项目，业务上参考了大众点评和小红书的一些常见场景，主要包括：

- 用户手机号验证码登录
- 商户查询与商户类型列表
- 博客探店、点赞、关注、Feed 流
- 优惠券与秒杀下单
- 自动化测试与压测验证

这个项目最值得讲的不是基础 CRUD，而是围绕高并发、缓存、消息队列、限流和可恢复性做的工程化增强。

2. 一分钟项目介绍

我的 hm-dianping 是一个基于 Spring Boot + Redis + MySQL + Kafka + Nginx 的本地生活服务项目，核心功能包括用户登录、商户查询、博客探店、关注 Feed 和优惠券秒杀。

我在这个项目里重点做了几类工程化改造：

1. 商户查询做了 Caffeine + Redis 二级缓存，优化热点读请求

2. 秒杀链路做成了 Redis Lua + Redis Stream + Kafka + MySQL 的异步架构
3. 通过 Redisson、数据库唯一索引和 `stock > 0` 条件更新保证幂等和防超卖
4. 给 Kafka 消费链路补了手动 ACK、有限次重试、死信队列、失败任务表和 Redis 回滚
5. 给验证码、登录和秒杀入口补了一套基于 Spring AOP + Redis 的轻量级限流
6. 又补了 API / UI 自动化测试和压测材料，方便后续验证改造效果

如果面试官继续展开，我最适合重点讲的模块是：

- 商户缓存
 - 秒杀链路
 - Kafka 可靠性增强
-

3. 技术栈

3.1 后端框架

- Java 8
- Spring Boot 2.3.12
- Spring MVC
- MyBatis-Plus
- Spring Data Redis
- Spring Kafka
- Redisson
- Hutool

3.2 中间件与存储

- MySQL
- Redis

- Kafka
- Nginx

3.3 工程化能力

- Caffeine 本地缓存
 - Redis Lua
 - Redis Stream
 - 分布式锁
 - 接口限流
 - API / UI 自动化测试
 - Allure 测试报告
-

4. 我做过的核心优化

这一段很适合用来回答“你在项目里真正做了什么”。

4.1 缓存优化

- 引入 `Caffeine + Redis` 二级缓存
- 商户详情和商户类型列表优先命中本地缓存
- 处理缓存穿透、击穿和多节点缓存失效

4.2 秒杀链路升级

- 秒杀入口仍然保留 Redis Lua 原子校验
- 通过 Redis Stream 承接入口消息
- 增加 Relay，把 Redis Stream 转发到 Kafka
- Kafka Consumer 异步创建订单

4.3 Kafka 可靠性增强

- Producer 开启 `acks=all`

- Producer 保持幂等开关 `enable.idempotence=true`
- Consumer 改成手动 ACK
- 增加有限次重试
- 增加 DLT 死信处理
- 增加失败任务表与定时重试
- 增加 Redis 回滚逻辑

4.4 稳定性增强

- 增加验证码、登录和秒杀入口限流
- 增加 Kafka lag 监控定时任务
- 增加秒杀库存对账任务

4.5 测试与验证

- 增加 API 自动化测试
- 增加 UI 自动化测试
- 增加秒杀链路测试和压测材料

5. 项目模块总览

5.1 用户模块

- 手机号验证码登录
- Redis 存验证码
- Redis Hash 存 Token 会话
- 用户签到与连续签到统计

5.2 商户模块

- 商户详情查询
- 按类型分页查询商户
- 按名称关键字查询商户

- 商户类型列表
- 附近商户查询可结合 Redis GEO 讲

5.3 博客模块

- 发布博客
- 热门博客查询
- 博客点赞
- 点赞用户列表
- 关注用户动态流

5.4 关注模块

- 关注 / 取关
- 是否已关注
- 共同关注

5.5 优惠券与秒杀模块

- 普通优惠券新增和查询
- 秒杀券新增
- 秒杀下单
- 异步订单落库
- 死信、失败任务和补偿

6. 登录与用户模块怎么讲

6.1 验证码登录流程

实际代码在：

- `UserController`
- `UserServiceImp1`
- `RefreshTokenInterceptor`

- `LoginInterceptor`

流程可以这样讲：

1. 用户请求发送验证码
2. 后端校验手机号格式
3. 生成 6 位验证码
4. 将验证码写入 Redis, key 为 `login:code:{phone}`, TTL 为 2 分钟
5. 用户提交手机号和验证码登录
6. 服务端从 Redis 取验证码校验
7. 查询用户, 不存在就创建
8. 生成 Token
9. 将用户信息转成 Hash 存入 Redis, key 为 `login:token:{token}`
10. 后续请求通过拦截器刷新 Token 并完成登录校验

6.2 为什么登录态放 Redis

因为 Redis 更适合做分布式会话管理：

- 支持多实例共享
- 读写快
- 天然支持 TTL
- 比传统 HttpSession 更适合后续扩展

6.3 签到怎么讲

签到用 Redis Bitmap：

- key: `sign:{userId}:yyyyMM`
- 每天签到就是把当月第 N 天对应 bit 位置为 1
- 连续签到统计通过 `BITFIELD` 读出后再按位计算

这个点很适合讲 Redis 数据结构应用。

7. 商户缓存怎么讲

7.1 当前缓存架构

项目里商户详情和商户类型列表采用了三级查询路径：

- L1: Caffeine 本地缓存
- L2: Redis
- L3: MySQL

相关代码：

- `CacheClient`
- `LocalCacheConfig`
- `ShopServiceImpl`
- `ShopTypeServiceImpl`

7.2 为什么要加 Caffeine

因为热点商户数据如果每次都访问 Redis，虽然比 MySQL 快，但仍然有网络开销。

本地缓存命中时可以直接在 JVM 内返回，延迟更低，也能降低 Redis QPS。

7.3 处理了哪些缓存问题

缓存穿透

- 对查不到的数据写空值缓存
- 空值缓存 TTL 较短

缓存击穿

- 用互斥锁方案控制缓存重建
- 也保留了逻辑过期方案能力

缓存雪崩

- TTL 引入随机抖动
- 避免同一批 key 同时过期

7.4 多节点本地缓存一致性怎么处理

因为每个服务实例都有自己的 Caffeine，本地缓存不共享。所以在更新商户后，项目会：

1. 更新数据库
2. 删除 Redis 缓存
3. 删除当前节点本地缓存
4. 通过 Redis Pub/Sub 广播 `cache:invalidate`
5. 其他节点收到消息后删除本地缓存

相关代码：

- `CacheInvalidationPublisher`
- `CacheInvalidationListener`
- `CacheInvalidationConfig`

7.5 商户查询链路怎么讲

`queryById()` 里实际走的是：

1. 先查本地缓存
2. 本地未命中查 Redis
3. Redis 未命中查 MySQL
4. 再回填 Redis 和本地缓存

7.6 商户类型列表为什么也适合本地缓存

因为商户类型列表：

- 数据量小

- 更新少
- 读频繁

这种数据非常适合放本地缓存。

8. 商户 GEO 怎么讲

商户查询模块里有一个按坐标查附近商户的能力，代码在 `ShopServiceImpl.queryShopByType()`。

思路是：

1. 如果没有坐标参数，直接查数据库分页
2. 如果有坐标参数，优先走 Redis GEO
3. 在 Redis 中以 `shop:geo:{typeId}` 存商户地理位置
4. 查询 5km 内商户，并带距离排序
5. 取出结果里的店铺 id，再回表查完整商户信息

这里可以讲的亮点是：

- GEO 适合做附近商户
 - Redis 返回 id + distance
 - 最后还要回数据库或缓存查业务详情
-

9. 博客、点赞、关注、Feed 怎么讲

9.1 点赞为什么适合 Redis ZSet

博客点赞用的是 Redis ZSet，核心 key 是：

- `blog:liked:{blogId}`

原因：

- 可以去重

- 可以判断用户是否点过赞
- 可以按时间排序查最近点赞用户

实现上：

- 点赞时把 `userId` 写入 ZSet
- `score` 用当前时间戳
- 取消点赞时从 ZSet 删除

9.2 关注关系为什么适合 Set

关注关系天然适合集合：

- `follows:{userId}`

可以支持：

- 关注 / 取关
- 判断是否关注： `SISMEMBER`
- 查询共同关注： `SINTER`

9.3 Feed 流怎么讲

当用户发布博客后：

1. 先查所有粉丝
2. 把博客 id 推到粉丝收件箱
3. 收件箱用 ZSet 保存，`score` 用时间戳
4. 查询 Feed 流时使用滚动分页

这样可以讲：

- 推模式
 - ZSet 滚动分页
 - `minTime + offset` 去重翻页
-

10. 秒杀链路怎么完整讲

这是项目最值得展开的一块。

10.1 一句话概括

我的秒杀链路采用的是 Redis 前置原子校验 + Kafka 异步削峰 + MySQL 最终落库 的分层方案。

10.2 入口链路

接口在：

- `POST /voucher-order/seckill/{id}`

控制器：

- `VoucherOrderController`

核心服务：

- `VoucherOrderServiceImpl.seckillVoucher()`

入口流程：

1. 获取当前用户 ID
2. 生成全局订单 ID
3. 执行 `seckill.lua`

Lua 脚本做的事：

1. 校验库存是否充足
2. 校验用户是否已下单
3. 预扣 Redis 库存
4. 记录用户下单标记
5. `XADD` 写入 `stream.orders`

如果 Lua 返回：

- 1：库存不足
- 2：重复下单
- 0：受理成功

10.3 为什么要 Lua

因为秒杀资格校验必须原子执行。

如果把“查库存、查一人一单、扣库存、写标记”拆成多个 Redis 命令，并发下容易出现超卖和重复下单。

所以 Lua 的价值是：

- 原子性
- 快速前置拦截
- 降低无效请求进入数据库

10.4 为什么接口能很快返回

因为这个接口返回的不是“订单已经落库完成”，而是：

请求已经通过 Redis 原子校验并成功进入异步链路。

真正的订单创建发生在后面的 Kafka Consumer 里。

这也是秒杀接口能抗住高并发的关键之一。

11. Redis Stream 到 Kafka 的桥接怎么讲

这是项目里很像真实工程改造的一块。

相关代码：

- RedisStreamToKafkaRelay

11.1 这个 Relay 做了什么

1. 启动时创建 Redis Stream 消费组
2. 持续消费 `stream.orders`
3. 把 Redis Stream 消息转成 `VoucherOrderMessage`
4. 调用 Producer 发到 Kafka
5. 只有 Kafka 发送成功后才 ACK Redis Stream

11.2 为什么不直接在业务代码里发 Kafka

因为当前链路保留了 Redis Lua 和 Redis Stream 的前置逻辑。
Relay 的价值是：

- 解耦业务入口和 Kafka
- 降低 Redis Stream 到 Kafka 的消息丢失风险
- 只有 Kafka 发成功才确认 Stream

11.3 Pending List 恢复怎么讲

Relay 启动时会先恢复 pending-list 中未确认消息，说明我考虑了：

- 服务中断
- 中间转发失败
- 未 ACK 消息重放

这个点很适合面试讲“可恢复性”。

12. Kafka 在项目里怎么讲

12.1 为什么要引入 Kafka

不要简单说“把 Redis Stream 换成 Kafka”，更准确的讲法是：

我保留 `Redis` 做入口原子校验，同时引入 `Kafka` 来增强异步削峰、解耦、重试、死信和消费恢复能力。

12.2 Kafka 的职责边界

Kafka 负责：

- 异步削峰
- 解耦入口和落库
- 消费重试
- 死信转移
- 消费恢复

Kafka 不负责：

- 秒杀资格校验
- 原子扣 Redis 库存
- 最终业务幂等
- 直接保证数据库最终一致性

12.3 生产者配置怎么讲

当前配置里：

- `acks=all`
- `retries=3`
- `enable.idempotence=true`

这样可以说明：

- 等待更严格的 Broker 确认
- 失败会自动重试
- 减少生产端重复写入风险

12.4 Consumer 为什么改成手动 ACK

当前 `application.yaml` 里：

- `spring.kafka.listener.ack-mode=manual_immediate`

KafkaConfig 里也显式配置了手动 ACK。

理由是：

- 秒杀消费成功后才能提交 offset
- 如果还没落库就先自动提交，会导致消息丢失
- 手动 ACK 更适合高价值订单类消息

一句话回答：

业务成功后再 ACK，失败不 ACK，让 Kafka 重试。

12.5 为什么要有限次重试

因为不是所有异常都适合无限重试。

当前配置：

- `SeekToCurrentErrorHandler`
- `FixedBackOff(1000ms, 3次)`

这样做的好处：

- 临时故障可以自动恢复
- 不会因为无限重试阻塞消息

12.6 为什么要死信队列

当消息重试 3 次仍失败时，会进入：

- `seckill-order-dlt`

死信的价值是：

- 隔离顽固失败消息
- 不影响正常消息继续消费
- 为后续人工处理或补偿留入口

相关代码：

- `VoucherOrderDeadLetterConsumer`

12.7 Kafka 的消息体里为什么带多个字段

订单消息里会带：

- `orderId`
- `userId`
- `voucherId`
- `traceId`

原因：

- `orderId`：最终订单标识
 - `userId`：业务幂等和用户维度处理
 - `voucherId`：库存和优惠券维度处理
 - `traceId`：链路排查和日志追踪
-

13. 幂等、防超卖、一致性怎么讲

13.1 幂等靠哪几层

这是一个经典高频题。

我项目里的幂等不是只靠一层，而是多层协同：

1. Redis 层：Lua 判断一人一单
2. Consumer 层：落库前再次校验
3. 数据库层：唯一索引最终兜底

13.2 超卖怎么防

主要两层：

1. Redis Lua 前置拦截库存不足请求
2. MySQL 落库时再用 `stock > 0` 条件更新兜底

所以就算极端并发下 Redis 和 MySQL 存在时间差，数据库也不会无限扣成负数。

13.3 为什么有 Lua 还要唯一索引

因为 Redis 是前置快速校验，数据库是最终业务真相。

真正的幂等兜底必须落在数据库层，否则重复消费、重试边界和极端异常场景下仍可能出问题。

13.4 为什么有唯一索引还要 Redisson

唯一索引能兜底，但如果只靠它：

- 大量并发请求会直接打数据库
- 冲突异常压力会很大

Redisson 的作用是：

- 在应用层提前限制同一用户并发创建订单
- 减少数据库冲突开销

13.5 最终一致性怎么讲

一定要明确：

- Redis 预扣库存只是前置优化
- MySQL 才是最终订单和库存结果

如果 Redis 成功但 MySQL 失败，就会出现不一致，需要靠：

- Kafka 重试
 - 死信处理
 - 失败任务表
 - Redis 回滚
 - 定时补偿
-

14. 失败补偿怎么讲

这一块是你项目的加分项。

14.1 死信消费后做了什么

`VoucherOrderDeadLetterConsumer` 收到死信后会：

1. 反序列化消息
2. 调用失败任务服务保存失败任务
3. 同时触发 Redis 回滚

14.2 为什么死信也要回滚 Redis

因为消息进入死信，说明主消费链路已经彻底失败了。

如果 Redis 预扣还不回滚：

- 库存会被长期占住
- 用户会被错误标记成已下单

所以死信不是简单打日志，而是要把业务状态收拢回来。

14.3 失败任务表有什么价值

`VoucherOrderFailTaskServiceImpl` 里维护了一套失败任务状态机：

- `RETRY_PENDING`
- `RETRYING`
- `DONE`
- `ROLLBACK_DONE`
- `MANUAL_HANDLE_REQUIRED`

它的价值在于：

- 失败可追踪

- 失败可自动重试
- 失败可人工处理
- 失败状态可运营化

14.4 自动重试怎么做

定时任务：

- `VoucherOrderFailTaskRetryScheduler`

会定期捞出 `RETRY_PENDING` 的任务重试发送回 Kafka。

超过最大次数后转为：

- `MANUAL_HANDLE_REQUIRED`

14.5 Redis 回滚怎么讲

回滚服务：

- `SeckillRedisRollbackService`

底层通过：

- `seckill_rollback.lua`

回滚内容包括：

- Redis 预扣库存
- 一人一单标记

并且做成幂等形式，避免重复回滚造成新问题。

15. 可观测性和治理怎么讲

当前项目已经有两个很适合讲“可观测性”的定时任务。

15.1 Kafka lag 监控

定时任务：

- `KafkaLagMonitorScheduler`

作用：

- 定期检查 Kafka 消费堆积
- 超过阈值时打告警日志

15.2 秒杀库存对账

定时任务：

- `SeckillStockReconcileScheduler`

作用：

- 对比 Redis 秒杀库存和数据库库存
- 发现不一致时记录告警

15.3 为什么这些点重要

因为项目如果只做“功能能跑”，其实还不够像真实工程。
有监控、有对账、有失败任务，才更像在考虑生产可维护性。

16. 限流怎么讲

16.1 为什么要加限流

虽然项目原本有秒杀并发保护，但没有通用接口限流。
验证码、登录和秒杀入口都属于高风险接口，容易被刷，所以补了限流。

16.2 当前限流方案

当前采用的是：

- 自定义注解 `@RateLimit`
- Spring AOP 切面拦截
- Redis 固定窗口计数
- 验证码接口附加冷却时间控制

核心代码：

- `RateLimitAspect`
- `RateLimitServiceImpl`
- `RateLimitKeyResolver`
- `UserController`
- `VoucherOrderController`

16.3 已接入哪些接口

`/user/code`

- 手机号维度限流
- 60 秒最多 3 次
- 30 秒冷却时间

`/user/login`

- 手机号维度限流
- 60 秒最多 20 次

`/voucher-order/seckill/{id}`

- 用户维度限流
- 1 秒最多 3 次

16.4 为什么先用固定窗口

因为第一版优先考虑：

- 快速落地

- 低侵入
- 容易维护

固定窗口虽然有边界效应，但对于当前验证码、登录和秒杀入口已经够用了。

如果后面要继续升级：

- 登录和验证码可升级为滑动窗口
- 更平台化的入口可考虑令牌桶或网关层限流

16.5 面试一句话总结

我补了一套基于 `Spring AOP + Redis` 的轻量级限流体系，先覆盖验证码、登录和秒杀入口，采用固定窗口计数和验证码冷却时间控制。

17. 测试和压测怎么讲

17.1 自动化测试做了什么

当前项目里已经补了：

- API 自动化测试
- UI 自动化测试
- Allure 报告支持
- 秒杀链路回归场景

从 `tests/README.md` 可以总结出覆盖重点：

- `shop-type` 基础可用性
- `/user/me` 未登录拦截
- 自动化登录后鉴权校验
- 秒杀券发现与秒杀下单
- 重复下单拦截
- 登出后 Token 失效
- UI 秒杀入口跳转与业务反馈

17.2 自动化测试为什么值得讲

因为这能说明：

- 你不是只写功能
- 你会做回归验证
- 你改缓存、限流、秒杀链路后，有验证手段

17.3 压测怎么讲

项目里也保留了压测计划和测试结果思路，可以重点讲：

- QPS
- TP95 / TP99
- 平均响应时间
- 成功率
- Redis 压力
- MySQL 压力
- Kafka lag
- 超卖率
- 重复下单率

17.4 如果面试官问“你怎么证明自己真的压过”

你可以答：

- 我有压测计划文档
- 我有链路测试结果记录
- 我会讲清测试入口、并发方式、关注指标和结论

18. 面试官高频问题怎么答

18.1 为什么不用数据库直接扣库存

因为秒杀并发高，直接查库扣库存会：

- 把大量请求直接打到数据库
- 容易锁竞争严重
- 响应慢

所以我把资格校验和预扣前移到了 Redis Lua。

18.2 为什么 Redis 预扣后数据库还要再扣一次

因为 Redis 预扣只是前置优化，不是最终业务真相。
最终订单和库存还是要以 MySQL 为准。

18.3 为什么不直接用 Kafka，非要先 Redis Stream

因为入口最核心的是 Redis Lua 原子校验，既然入口消息已经在 Redis 侧形成了本地消息流，
那加一层 Relay 可以让 Kafka 和入口解耦，同时做到“Kafka 成功后再 ACK Stream”。

18.4 Kafka 消费失败怎么办

我区分两类：

- 可恢复异常：不 ACK，让 Kafka 重试
- 不可恢复异常：执行 Redis 回滚并 ACK，防止无限重试

超过最大重试次数后进入 DLT，再落失败任务表。

18.5 为什么有唯一索引还要 Redisson

唯一索引是数据库兜底，Redisson 是应用层提前拦截。
两者作用层次不同，结合起来更稳。

18.6 为什么要死信队列

因为不是所有消息都适合无限重试。

死信队列的价值是把顽固失败消息隔离出来，保证正常消息继续消费，并给后续补偿留入口。

18.7 你项目里最像真实公司的点是什么

我觉得最像的是：

- 秒杀链路不是同步扣库，而是异步削峰
- 消费链路考虑了手动 ACK、重试、死信、补偿和回滚
- 缓存不只考虑命中率，也考虑多节点一致性
- 还补了限流、自动化测试和监控定时任务

19. 项目亮点和不足怎么讲

19.1 可以写进简历的 3 个亮点

如果简历只保留 3 个点，我建议写：

1. 秒杀链路：Redis Lua + Redis Stream + Kafka + MySQL
2. 商户缓存：Caffeine + Redis 二级缓存 + Pub/Sub 失效同步
3. Kafka 可靠性增强：手动 ACK、DLT、失败任务表、Redis 回滚

19.2 当前不足要怎么诚实回答

可以这样讲：

1. 限流目前还是轻量版，规则主要写在代码里，还没外置
2. 限流还没补 Nginx / 网关层粗限流
3. 自动化测试覆盖了关键路径，但还不是全量集成测试
4. 监控更多还是日志和定时任务级别，还没接完整指标平台

5. 多实例线上部署时，还可以继续加强配置管理、告警和链路追踪

这种回答比空泛地说“还能继续优化”更像真实开发者。

20. 30 秒 / 1 分钟 / 3 分钟版本

20.1 30 秒版本

我的 hm-dianping 是一个 Spring Boot + Redis + MySQL + Kafka 的本地生活服务项目，核心亮点是商户查询做了 Caffeine + Redis 二级缓存，秒杀链路做成了 Redis Lua + Kafka 异步削峰，并补了手动 ACK、死信、失败补偿和接口限流。

20.2 1 分钟版本

这个项目主要包括用户登录、商户查询、博客探店、关注 Feed 和优惠券秒杀。我重点做了两块：一块是商户查询的二级缓存，通过 Caffeine + Redis 降低热点读压力；另一块是秒杀链路，把原来的同步处理升级成 Redis Lua 前置原子校验 + Redis Stream + Kafka + MySQL 的异步架构。除此之外，我还补了 Redisson 锁、数据库唯一索引、手动 ACK、死信、失败任务表、Redis 回滚、接口限流以及 API/UI 自动化测试。

20.3 3 分钟版本

我的 hm-dianping 是一个本地生活服务项目，主要业务有用户登录、商户查询、博客探店、关注 Feed 和秒杀下单。项目最值得讲的是缓存和秒杀。

缓存这块，我把商户详情和商户类型列表做成了 L1 Caffeine + L2 Redis + L3 MySQL 的二级缓存体系，并且处理了缓存穿透、击穿和多节点本地缓存失效问题。

秒杀这块，入口先通过 Redis Lua 做库存校验和一人一单校验，校验通过后把订单消息写入 Redis Stream，再由 Relay 转发到 Kafka，最后 Consumer 异步落库到 MySQL。为了保证可靠性，我又把 Kafka Consumer 改成了手动 ACK，并补了有限次重试、死信队列、失败任务表和 Redis 回滚。这样项目就不仅是能跑，还具备了高并发、可恢复和可补偿能力。

另外我还补了基于 Spring AOP + Redis 的轻量级限流，以及 API / UI 自动化测试，用来验证这些改造没有把核心链路搞坏。

21. 建议复习顺序

建议你按下面顺序复习：

1. 先背第 2 节一分钟介绍
2. 再背第 10、12、13、14 节秒杀、Kafka、幂等和补偿
3. 然后背第 7 节缓存
4. 再补第 16 节限流
5. 最后看第 18、19、20 节，准备快问快答

如果你后面还想继续精简，我可以再从这份文档里给你继续拆两份：

- 5分钟讲稿版
- 极速速记卡版

?????README_ZH.md

hm-dianping 项目说明

hm-dianping 是一个基于 Spring Boot + Redis + MySQL + Kafka + Nginx 的大众点评类项目。

项目围绕本地生活服务场景，实现了用户登录、商户查询、博客探店、关注 Feed、优惠券秒杀等功能，并在此基础上补充了缓存优化、异步削峰、分布式锁、失败补偿、接口限流和自动化测试等工程化能力。

这个仓库不仅是一个业务练手项目，也适合作为高并发、缓存、消息队列、限流和测试能力的综合实践项目。

一、项目亮点

- 基于 Redis 实现短信验证码登录、Token 会话缓存、签到统计
- 基于 Redis 的 Set、ZSet、Bitmap、GEO 等数据结构实现业务能力
- 秒杀链路采用 Redis Lua + Redis Stream + Kafka + MySQL
- 通过 Redisson 分布式锁、数据库唯一索引、乐观扣减库存防止超卖和重复下单
- 通过 Caffeine + Redis 实现二级缓存
- 通过 Redis Pub/Sub 处理多节点本地缓存失效
- 基于 Spring AOP + Redis 实现轻量级接口限流
- 提供 API / UI 自动化测试与 Allure 报告支持

二、核心功能

1. 用户模块

- 手机号验证码登录
- Token 登录态维护
- 用户信息查询
- 退出登录
- 用户签到与连续签到统计

2. 商户模块

- 商户详情查询
- 按类型分页查询商户
- 按名称关键字查询商户
- 商户类型列表查询

3. 博客 / 探店模块

- 发布探店博客
- 查询热门博客
- 博客点赞
- 查询点赞用户列表
- 关注用户动态流

4. 关注模块

- 关注 / 取关用户
- 判断是否已关注
- 查询共同关注

5. 优惠券 / 秒杀模块

- 普通优惠券新增与查询
- 秒杀券新增
- 秒杀下单
- 秒杀订单异步落库
- 失败任务补偿与重试

三、技术栈

后端框架

- Java 8

- Spring Boot 2.3.12
- Spring MVC
- MyBatis-Plus
- Spring Data Redis
- Spring Kafka
- Redisson

存储与中间件

- MySQL
- Redis
- Kafka
- Nginx

缓存 / 并发治理

- Caffeine
- Redis Lua
- Redis Stream
- 分布式锁
- 接口限流

测试

- PyTest
- Selenium
- Allure

四、项目架构设计

1. 登录与会话设计

项目采用手机号验证码登录。

- 验证码缓存到 Redis: `login:code:{phone}`
- 用户登录后, 生成 Token 并将用户会话写入 Redis:
`login:token:{token}`

服务端通过拦截器完成:

- Token 刷新
- 登录态校验
- 用户信息写入 `ThreadLocal`

相关文件:

- [UserController.java](#)
- [UserServiceImpl.java](#)
- [RefreshTokenInterceptor.java](#)
- [LoginInterceptor.java](#)

2. 缓存设计

项目既有基础 Redis 缓存, 也引入了二级缓存方案:

- L1: Caffeine 本地缓存
- L2: Redis
- L3: MySQL

主要优化点:

- 缓存穿透: 缓存空对象
- 缓存击穿: 互斥锁 / 逻辑过期
- 热点数据: 本地缓存加速
- 多节点一致性: Redis Pub/Sub 广播缓存失效消息

相关文件:

- [CacheClient.java](#)
- [LocalCacheConfig.java](#)
- [CacheInvalidationConfig.java](#)

- [CacheInvalidationListener.java](#)

3. 秒杀链路设计

秒杀是项目中最核心的高并发场景，链路如下：

用户请求

- > Controller 入口
- > Redis Lua 原子校验库存 / 一人一单 / 预扣库存
- > XADD 写入 Redis Stream
- > Relay 转发到 Kafka
- > Kafka Consumer 异步消费
- > MySQL 扣减最终库存并创建订单

关键设计点：

- Redis Lua 负责秒杀资格原子校验
- Redis 负责前置预扣库存，不作为最终订单结果
- Kafka 负责削峰、解耦、失败重试和死信处理
- MySQL 使用 `stock > 0` 和唯一索引做最终兜底
- Redisson 分布式锁进一步保护同一用户并发创建订单

相关文件：

- [VoucherOrderServiceImpl.java](#)
- [seckill.lua](#)
- [RedisStreamToKafkaRelay.java](#)
- [VoucherOrderConsumer.java](#)
- [VoucherOrderDeadLetterConsumer.java](#)

4. 限流设计

项目补充了一套轻量级限流方案，当前采用：

- 自定义注解 `@RateLimit`
- Spring AOP 统一拦截

- Redis 固定窗口计数
- 验证码接口附加冷却时间控制

首批限流接口：

- `/user/code`
- `/user/login`
- `/voucher-order/seckill/{id}`

相关文件：

- [RateLimit.java](#)
- [RateLimitAspect.java](#)
- [RateLimitServiceImpl.java](#)
- [RateLimitKeyResolver.java](#)

五、目录结构

```
hm-dianping
├─ src/main/java/com/hmdp
│   ├─ controller          # Controller 层
│   ├─ service             # Service 接口
│   ├─ service/impl        # Service 实现
│   ├─ mapper              # MyBatis-Plus Mapper
│   ├─ entity              # 实体类
│   ├─ dto                 # DTO / 返回对象
│   ├─ config              # Spring 配置、缓存、Kafka、调
度器
│   ├─ mq                  # 消息队列与异步链路
│   ├─ utils               # 工具类
│   ├─ annotation          # 自定义注解
│   ├─ aspect              # AOP 切面
│   ├─ exception           # 异常定义
│   └─ enums               # 枚举
├─ src/main/resources
│   └─ application.yaml
```

```
|   ├── application-local.example.yaml
|   ├── db
|   ├── mapper
|   ├── seckill.lua
|   └── seckill_rollback.lua
├── tests                    # 自动化测试
├── scripts                 # 测试 / 压测脚本
├── heima_qianduan         # 前端静态资源与 Nginx
└── *.md                   # 设计文档、压测记录、面试材料
```

六、快速开始

1. 环境要求

- JDK 8
- Maven 3.6+
- MySQL 5.7+
- Redis 6.x
- Kafka 2.x+

2. 数据初始化

可以使用以下 SQL 文件初始化数据库：

- [src/main/resources/db/hmdp.sql](#)
- [heima_qianduan/hmdp.sql](#)

如果要体验增强版秒杀链路，建议额外执行：

- [src/main/resources/db/voucher_order_unique_index.sql](#)
- [src/main/resources/db/voucher_order_fail_task.sql](#)
- [src/main/resources/db/voucher_order_fail_task_failure_type.sql](#)

3. 配置说明

项目采用“公共配置 + 本地私有配置”方式：

- [application.yaml](#)
可提交到仓库，使用占位符和默认值，不包含真实敏感配置
- [application-local.example.yaml](#)
本地配置示例文件

推荐做法：

1. 复制 `application-local.example.yaml`
2. 重命名为 `application-local.yaml`
3. 填入你本地真实的 MySQL / Redis / Kafka 配置

`application-local.yaml` 已加入 `.gitignore`，不会提交到 GitHub。

你也可以通过环境变量注入配置，例如：

- `DB_URL`
- `DB_USERNAME`
- `DB_PASSWORD`
- `REDIS_HOST`
- `REDIS_PORT`
- `REDIS_PASSWORD`
- `KAFKA_BOOTSTRAP_SERVERS`

4. 启动顺序

推荐启动顺序：

1. MySQL
2. Redis
3. Kafka
4. Spring Boot 后端
5. Nginx 前端

5. 启动后端

```
mvn spring-boot:run
```

或者直接运行启动类：

- [HmDianPingApplication.java](#)

默认后端地址：

```
http://127.0.0.1:8081
```

6. 启动前端

项目自带前端静态资源和 Nginx，可直接本地启动：

```
cd heima_qianduan/nginx-1.18.0  
start nginx.exe
```

默认前端地址：

```
http://127.0.0.1:8080
```

七、测试说明

项目包含 API / UI / 秒杀链路自动化测试。

详见：

- [tests/README.md](#)

常用命令：

```
pytest -m api --base-url http://127.0.0.1:8081
```

```
pytest -m ui --ui-base-url http://127.0.0.1:8080 --  
headless
```

```
pytest --base-url http://127.0.0.1:8081 --ui-base-url http://127.0.0.1:8080 --alluredir reports/allure-results
```

如果安装了 Allure:

```
allure serve reports/allure-results
```

八、适合展开的工程化主题

这个项目比较适合在简历和面试中展开以下内容:

- Redis 缓存设计
- 二级缓存与缓存一致性
- 秒杀高并发链路
- Kafka 削峰、重试、死信与补偿
- Redis 预扣库存与回滚
- 分布式锁与幂等控制
- 接口限流
- 自动化测试平台与回归测试
- 压测与性能优化

九、上传 GitHub 前建议

建议在公开仓库前做以下整理:

- 保留 `application.yaml`, 不要提交真实本地配置
- 不提交 `application-local.yaml`
- 清理日志、报告、临时缓存文件
- 适当补充架构图、接口截图、压测结果图
- 增加 `LICENSE` 文件, 例如 `MIT`

十、后续优化方向

- 将限流规则抽离到配置层
- 为热点接口补充 Nginx / 网关层粗限流
- 增加 Prometheus / Grafana 监控指标
- 为秒杀与缓存链路补充更多集成测试
- 补充英文版 README 与架构图

License

当前仓库尚未补充开源协议。

如果你准备公开到 GitHub，建议新增 `LICENSE` 文件。

?????Kafka秒杀优惠券改造清单.md

Kafka秒杀优惠券改造清单.md

恢复说明：原始 Markdown 已从工作区删除，当前文件为重建入口页，不是原始逐字版本。

建议优先阅读 MASTER_INTERVIEW.md，再按主题阅读现存资料。

建议阅读顺序

1. [MASTER_INTERVIEW.md](#)
 2. 相关主题现存文档
 3. 如需继续细化，可在此文件基础上继续补充整理
-

??????Kafka秒杀链路后续优化实施计划.md

Kafka秒杀链路后续优化实施计划.md

恢复说明：原始 Markdown 已从工作区删除，当前文件为重建入口页，不是原始逐字版本。

建议优先阅读 MASTER_INTERVIEW.md，再按主题阅读现存资料。

建议阅读顺序

1. [MASTER INTERVIEW.md](#)
 2. 相关主题现存文档
 3. 如需继续细化，可在此文件基础上继续补充整理
-

??????Kafka秒杀链路面试准备文档.md

Kafka 秒杀链路面试准备文档

1. 这套 Kafka 改造在项目里是怎么用的
-

这个项目里的 Kafka 不是直接替代 Redis，而是和 Redis 分工配合：

- Redis 负责秒杀入口的高并发原子校验
- Redis Lua 负责库存判断、一人一单判断、预扣库存
- Redis Stream 负责把“已通过资格校验的下单请求”先落成一条本地消息
- Relay 组件负责把 Redis Stream 中的消息转发到 Kafka
- Kafka 负责异步削峰、解耦和更可靠的消费恢复
- Consumer 负责真正创建订单、扣减数据库库存、落 MySQL
- MySQL 负责最终业务事实，以订单表和库存扣减结果为准

你可以把它概括成一句话：

Redis 做前置原子校验，Kafka 做异步削峰，MySQL 做最终一致性落库。

2. 项目里的实际链路

2.1 请求入口

接口是：

- `POST /voucher-order/seckill/{id}`

入口代码在 [VoucherOrderController.java](#) 和 [VoucherOrderServiceImpl.java](#)。

用户发起秒杀请求后，服务端会：

1. 获取当前用户 ID
2. 生成全局订单 ID
3. 执行 Redis Lua 脚本
4. Lua 校验通过后，直接返回“抢购成功，订单已受理”

这里要特别强调一个点：

接口返回成功，表示请求已经通过 `Redis` 原子校验并成功写入 `Redis Stream`，属于“已受理”，不代表订单已经同步落库完成。

真正的订单创建，是后续通过 `Redis Stream -> Relay -> Kafka -> Consumer -> MySQL` 这条异步链路完成的。

所以当面试官问“为什么 `Kafka` 挂了，接口还返回成功”时，你可以回答：

因为我的接口成功语义定义的是受理成功，不是最终完成成功。秒杀入口只对 `Redis` 原子校验和消息入流负责，最终订单落库由后续异步链路完成。这样做的好处是不会把 `Kafka` 可用性直接绑定到接口可用性上。

这里的核心点是：秒杀请求不会在接口线程里直接落库，而是尽量快速返回，把真正的订单处理异步化。

2.2 Lua 脚本做了什么

Lua 脚本见 [seckill.lua](#)。

它做了三件关键事：

1. 判断 `Redis` 里的秒杀库存是否充足
2. 判断用户是否已经下过单
3. 如果校验通过，就预扣 `Redis` 库存、记录下单用户，并 `XADD stream.orders`

这里的意义是：

- 库存校验和一人一单校验都在 `Redis` 内原子执行
- 能在真正访问数据库前拦掉大量无效请求
- 通过 `XADD` 把“成功受理”的下单请求写进 `Redis Stream`，避免接口线程直接做重活

2.3 Redis Stream 到 Kafka 的桥接

桥接组件在 [RedisStreamToKafkaRelay.java](#)。

它的职责是：

- 作为 Redis Stream 消费组消费者，持续读取 `stream.orders`
- 把 Stream 消息转换成 `VoucherOrderMessage`
- 调用 Producer 转发到 Kafka Topic
- 只有 Kafka 发送成功后，才 ACK Redis Stream 消息

这一步非常适合面试讲，因为它解决的是“Redis 已经预扣库存了，但消息怎么可靠地进入 Kafka”这个问题。

你可以把它描述成：

我在 Redis Stream 和 Kafka 之间加了一个 Relay，Kafka 发送成功后才 ACK Stream，用它把秒杀入口和 Kafka 解耦，同时降低 Redis 到 Kafka 之间的丢消息风险。

2.4 Kafka Producer 怎么发消息

Producer 接口和实现分别在：

- [VoucherOrderProducer.java](#)
- [VoucherOrderProducerImpl.java](#)

当前实现里：

- Topic 默认是 `seckill-order`
- 消息体是 `VoucherOrderMessage` 的 JSON
- Kafka message key 使用 `userId`

使用 `userId` 作为 key 的好处是：

- 同一用户的消息更容易落到同一个分区
- 有助于降低同一用户重复请求带来的乱序影响
- 便于面试时解释“按业务维度做分区 key 设计”

2.5 Kafka Consumer 怎么消费

Consumer 在 [VoucherOrderConsumer.java](#)。

它负责：

1. 监听 Kafka Topic
2. 把 JSON 反序列化成 `VoucherOrderMessage`
3. 转成 `VoucherOrder`
4. 调用

```
voucherOrderService.handleVoucherOrderFromMQ(order  
)
```

真正落库逻辑还是在 [VoucherOrderServiceImpl.java](#) 里，核心包括：

- 按用户维度加分布式锁
- 校验是否重复下单
- 扣减数据库库存，条件是 `stock > 0`
- 保存订单

所以整条链路里，Kafka 负责异步传输，真正的业务幂等和最终一致性还是靠 Consumer 侧业务逻辑和数据库约束兜底。

3. 为什么这个项目要引入 Kafka

你不要把它讲成“我把 Redis Stream 替换成 Kafka”，更准确的说法是：

我把秒杀系统升级成了 `Redis` 前置原子校验 + `Kafka` 异步削峰 + `MySQL` 最终落库的链路。

引入 Kafka 的核心动机有四个：

1. 提高异步削峰能力
2. 提高消息积压和消费恢复能力
3. 让消息链路更适合扩展多个 Consumer 实例
4. 让面试表达从“线程池异步”升级成“可讲可靠性和可恢复性的消息架构”

4. Kafka 在这个项目里的职责边界

这是面试里最容易被问的点。

Kafka 负责什么

- 异步化
- 削峰填谷
- 解耦下单入口和订单落库
- 支持消费重试和服务重启后的消息恢复

Kafka 不负责什么

- 不负责秒杀资格校验
- 不负责库存原子扣减
- 不负责业务幂等
- 不直接保证数据库最终一致性

如果面试官问“为什么 Redis 还要保留”，你可以回答：

因为 `Kafka` 擅长异步传输和削峰，但不适合直接做高并发秒杀资格校验。秒杀最核心的是库存判断和一人一单，这两个操作必须在入口阶段快速、原子地完成，所以我保留了 `Redis + Lua`。

5. 这套方案的优点

5.1 性能层面

- 秒杀请求可以快速返回，不阻塞在数据库事务上
- Redis 先拦截无库存和重复下单请求，减少无效流量
- Kafka 把高并发下单请求异步化，降低 DB 峰值写压力

5.2 架构层面

- 入口校验和订单落库解耦
- 更容易横向扩展消费端
- 服务重启后消息恢复能力比本地线程池更强

5.3 面试表达层面

- 能讲清楚 Redis、Kafka、MySQL 三者分工
- 能延伸到幂等、重试、死信、最终一致性
- 比“只是用线程池异步下单”更有工程说服力

6. 当前实现已经具备的亮点

结合现有代码，你可以明确说已经做了这些：

1. Kafka 已经接入 Spring Boot，见 [application.yaml](#)
2. Producer 已经封装，消息模型已独立抽象
3. Consumer 已经可以监听 Topic 并异步执行业务
4. Redis Stream 到 Kafka 的 Relay 已经存在
5. Relay 是“发送 Kafka 成功后再 ACK Redis Stream”
6. 秒杀入口仍然保留 Redis Lua 的原子校验
7. Consumer 侧仍保留 Redisson 锁 + DB 库存条件扣减
8. Consumer 已改成手动 ack，业务成功后才提交 offset
9. Kafka Consumer 已接入有限次重试
(`SeekToCurrentErrorHandler + FixedBackOff`)
10. Relay 启动时会恢复 Redis Stream pending-list
11. 链路日志已经补充 `traceId / orderId / userId / voucherId`
12. 数据库唯一索引脚本已经补充，作为最终幂等兜底
13. 已经验证过：Consumer 失败时不会立即 ack，而是会按 offset 重试
14. 已经验证过：Kafka 不可用时，接口仍可返回“已受理”，但 Relay 不应误判为 Kafka 已成功确认
15. 已完成：重试超限消息进入死信 Topic (`seckill-order-dlt`)

16. 已完成：死信消息落失败任务表

`tb_voucher_order_fail_task`

17. 已完成：失败任务自动重试（定时任务扫描 `RETRY_PENDING` 并回投主 Topic）

18. 已完成：失败任务管理接口（分页查询/手动重试/人工忽略）

7. 当前实现还可以继续补强的点

这部分你面试里反而会加分，因为说明你知道方案边界。

7.1 Kafka Producer 可靠性还可以继续增强

当前配置已经打开了：

- `acks=all`
- `retries=3`
- `enable.idempotence=true`

当前代码层面也已经做了：

- Producer 发送时同步等待 broker ack
- 只有真正拿到 Kafka send result 才返回 success
- 成功日志会打印 `traceId / orderId / userId / voucherId / partition / offset`
- 失败日志会明确打印发送失败

这一步非常关键，因为它避免了“只是把消息提交给客户端发送线程，就误判为 Kafka 发送成功”的问题。

当前还可以继续补：

- 发送失败落本地失败表或重试队列
- 更明确的 Topic 分区规划
- 对 Broker 长时间不可用场景增加失败治理策略

7.2 Kafka Consumer 可靠性还可以增强

当前代码里 Consumer 用的是 `@KafkaListener`，并且已经完成了：

- 手动提交 offset
- 仅在业务成功后 `ack.acknowledge()`
- 使用 `SeekToCurrentErrorHandler + FixedBackOff(1000ms, 3次)` 做有限次重试

这意味着当前已经不是“建议改手动提交”，而是已经落地了基础版本。

进一步的工程化增强还可以包括：

- 区分可恢复异常和不可恢复异常（按异常类型路由）
- 增加消费时延、重试次数、死信量监控
- 增加失败任务重试退避策略（指数退避、熔断窗口）

7.3 数据库幂等还可以更强

当前业务里已经有：

- 用户维度分布式锁
- 查询重复订单
- `stock > 0` 条件扣减

更稳的做法是：

- 在订单表增加唯一索引 `unique(user_id, voucher_id)`

这会让你在回答“怎么防止重复消费导致重复订单”时更硬。

7.4 最终一致性补偿还可以继续做

最典型的问题是：

- Redis 里库存已经预扣

- 但 Kafka 或 Consumer 最终落库失败

当前已经具备的最小恢复能力包括：

- Consumer 失败后不 ack，通过 offset 做重试
- Relay 启动时恢复 Redis Stream pending-list

这说明当前方案已经具备“失败后不立即丢消息”的基础保障。

与最新代码对齐时，可以区分「已落地」和「仍可演进」：

- 死信、失败任务、补偿重试与 Redis 幂等回滚：已落地
- Redis 秒杀库存与 MySQL `tb_seckill_voucher.stock` 定时对账：`SeckillStockReconcileScheduler`（不一致打 WARN）
- Kafka consumer lag 巡检：`KafkaLagMonitorScheduler`（lag 超阈值打 WARN）
- 仍可演进：一人一单与订单事实对账、指标大盘与告警平台对接

7.5 为什么建议把 Kafka Consumer 改成手动确认

这是你面试里非常值得讲的一点，因为它体现的是你对消息可靠性的理解，而不是只会把 Kafka 跑起来。

当前配置是什么

当前项目配置在 [application.yaml](#) 里，可以看到：

- `spring.kafka.consumer.enable-auto-commit=true`

这意味着 Consumer offset 会由 Kafka 客户端自动提交，而不是在业务成功后由我们自己控制提交时机。

自动提交有什么问题

自动提交最大的问题是：

消息是否真正处理成功 和 `offset` 是否已经提交 不是强绑定的。

也就是说，可能出现下面这种情况：

1. Consumer 已经从 Kafka 拉到了消息
2. Kafka 客户端按自动提交策略把 `offset` 提交了
3. 但此时业务代码还没真正执行成功，或者刚执行一半服务就挂了
4. 因为 `offset` 已经提交，Consumer 重启后这条消息不会再被重新消费
5. 最终结果就是：消息丢了，Redis 已经预扣库存，但 MySQL 没有成功创建订单

对于秒杀系统来说，这类问题非常致命，因为它会直接影响最终一致性。

为什么秒杀场景更适合手动确认

因为秒杀链路不是“消费到消息就算成功”，而是必须满足下面条件之后，才算这条消息真正完成：

1. Kafka 消息成功反序列化
2. `VoucherOrder` 对象构建成功
3. 订单业务逻辑执行成功
4. 数据库库存扣减成功
5. 订单写入 MySQL 成功
6. 没有触发重复下单、库存不足等异常分支

只有这些都完成了，才应该提交 `offset`。

你在面试里可以这么说：

秒杀场景里我不会让 `kafka` 自动提交 `offset`，因为消息被拉取不代表订单已经真正落库。更合理的做法是把 `offset` 提交时机后移到业务成功之后，这样才能避免消息已确认但订单未落库的丢单问题。

手动确认的核心思想

手动确认其实很简单，本质就是：

- Kafka 负责把消息投递给你
- 你自己决定什么时候告诉 Kafka“这条消息我处理完了”

也就是：

- 业务成功 -> `ack.acknowledge()`
- 业务失败 -> 不提交 offset, 让 Kafka 后续重试, 或者进入失败处理链路

7.6 手动确认的修改方案

下面这部分建议你面试时按“配置层、消费层、失败处理层”三个层次讲，会显得非常完整。

第一层：修改 Consumer 提交模式

把 Kafka Listener 容器改成手动确认模式。

常见做法是配置 `AckMode.MANUAL` 或 `AckMode.MANUAL_IMMEDIATE`。

两者区别：

- `MANUAL`：调用 `ack.acknowledge()` 后，由容器按批次或时机提交
- `MANUAL_IMMEDIATE`：调用 `ack.acknowledge()` 后立即提交

对于秒杀订单这种更强调处理结果确定性的场景，我更建议：

- `MANUAL_IMMEDIATE`

因为这样更容易讲清楚“业务成功后立刻确认”。

第二层：修改 Consumer 方法签名

当前 Consumer 在 [VoucherOrderConsumer.java](#) 中，大致是：

1. 收到 `payload`
2. 反序列化成 `VoucherOrderMessage`
3. 转成 `VoucherOrder`
4. 调用业务方法处理

改成手动确认后，Consumer 方法要增加一个 `Acknowledgment` `ack` 参数。

逻辑改成：

1. 反序列化消息
2. 调用订单处理逻辑
3. 如果业务成功，则执行 `ack.acknowledge()`
4. 如果失败，则抛异常，不提交 offset

伪代码如下：

```
@KafkaListener(topics = "seckill-order",
containerFactory =
"manualAckKafkaListenerContainerFactory")
public void onMessage(String payload, Acknowledgment
ack) {
    VoucherOrderMessage msg =
    objectMapper.readValue(payload,
    VoucherOrderMessage.class);
    VoucherOrder order = convert(msg);

    voucherOrderService.handleVoucherOrderFromMQ(order)
    ;
    ack.acknowledge();
}
```

核心点就一句话：

只有 `handleVoucherOrderFromMQ(order)` 真正执行成功后，才 `acknowledge`。

第三层：失败时不确认，让 Kafka 重试

如果消费失败，不要调用 `ack.acknowledge()`。

通常做法是：

- 记录错误日志
- 抛出异常
- 让 Spring Kafka 的错误处理机制接管

这样消息不会被当成“已完成”，而是会进入重试逻辑。

你面试时可以这么讲：

我把 `offset` 提交和业务成功绑定起来，如果订单落库失败，我就不确认 `offset`，而是交给 `Kafka` 的重试机制处理，避免消息提前确认导致丢单。

7.7 手动确认之后，还要配什么

只改手动确认还不够，完整方案还要把“重试、死信、幂等”一起讲出来。

1. 有限次重试

不能无限重试，否则一条坏消息会长期阻塞消费。

建议：

- 本地重试 3 次
- 每次退避一段时间
- 超限后进入死信 Topic

2. 死信队列

如果某条消息始终消费失败，比如：

- 数据格式错误
- 业务数据脏数据
- 下游依赖长期异常

就把它投递到死信 Topic，例如：

- `seckill-order-dlt`

这样主消费链路不会一直卡住，同时又保留了后续人工排查和补偿的机会。

3. 数据库唯一索引兜底

就算有了手动确认，也不能假设 Kafka 一定只投递一次。

因为以下情况都可能导致重复消费：

- Consumer 业务成功了，但 ack 提交前进程挂了
- Kafka 在异常边界下重复投递
- 手动补偿时重复重放消息

所以数据库仍然建议增加唯一索引：

- `unique(user_id, voucher_id)`

这样你在面试时就能讲成“三层幂等”：

1. Redis Lua：入口防重复
2. Consumer 业务校验：消费侧防重复
3. DB 唯一索引：最终兜底

7.8 面试时怎么回答“为什么要手动确认”

简版回答

因为自动提交 `offset` 会导致消息是否确认和业务是否成功解耦。对于秒杀订单来说，消息被拉取不代表订单已经成功落库，所以我会改成手动确认，只有在订单创建成功后才提交 `offset`，避免丢单。

进阶版回答

我这个秒杀链路是 `Redis` 预扣库存、`Kafka` 异步削峰、`MySQL` 最终落库。这里最怕的问题是 `Redis` 已经扣了库存，但 `Kafka` 消息却因为消费失败没有成功落库。如果 `offset` 还是自动提交，就会出现消息已经确认但订单没创建成功的情况。为了把消息确认时机和业务成功强绑定，我会把 `Consumer` 改成手动 `ack`，业务成功再提交 `offset`；失败则不确认，让 `Kafka` 重试，重试超限再进入死信队列。同时再用数据库唯一索引兜底重复消费。

7.9 面试官可能继续追问的问题

1. 手动确认是不是就绝对不会重复消费了

不是。

手动确认解决的是“过早确认导致丢单”的问题，不是“绝不重复投递”的问题。

`Kafka` 的设计本身更接近至少一次投递，所以消费侧仍然要做幂等。

2. 为什么不在消费前先 `ack`，成功后再补偿

因为这样会把风险提前释放。

对于秒杀订单，核心目标是：

- 宁可重复消费后被幂等拦住
- 也不能提前确认后造成真实丢单

所以更合理的是“先执行业务，后确认消息”。

3. 手动确认会不会降低吞吐

会有一点点影响，但这是值得的。

因为秒杀系统最怕的不是少一点吞吐，而是订单数据不一致、库存已经扣了但订单没创建成功。

在这种场景下，可靠性优先级通常高于极致吞吐。

7.10 推荐落地顺序

如果后面真的继续改代码，建议按下面顺序实施：

1. Consumer 改手动 ack
2. 业务成功后再提交 offset
3. 增加 Kafka 消费失败重试
4. 增加死信 Topic
5. 增加数据库唯一索引 `user_id + voucher_id`
6. 增加补偿任务，处理 Redis 预扣成功但 MySQL 落库失败的场景

你可以把这一组改造总结成一句话：

我后续会把 `kafka` 消费从自动提交升级成业务成功后手动确认，再配合重试、死信和数据库唯一索引，把秒杀订单链路从“能跑”提升到“可靠可讲”。

7.10 数据库唯一索引、死信队列与补偿方案

如果你面试只讲“我用了 Kafka”，其实很容易停留在会用框架的层面。

真正能体现工程深度的，是你有没有把下面三个问题讲清楚：

1. 重复消费怎么办
2. 多次消费失败怎么办

3. Redis 已预扣库存但 MySQL 没落库怎么办

这三个问题分别对应：

- 数据库唯一索引
- 死信队列
- 一致性补偿

7.10.1 为什么还要加数据库唯一索引

很多同学会以为已经有了：

- Redis Lua 一人一单
- Consumer 侧重复下单校验
- Redisson 锁

就已经足够了。

但从“最终一致性”和“消息重复投递”的角度看，这还不够。

因为 Kafka 消费模型本质上更接近至少一次投递，以下情况都可能导致同一条订单消息被重复处理：

1. Consumer 业务成功，但 ack 前进程挂了
2. Kafka 发生重复投递
3. 死信消息被人工重放
4. 补偿任务重复执行
5. 多实例并发消费边界下出现重复进入业务逻辑

如果这时候只有应用层判重，没有数据库兜底，那么最坏情况仍然可能出现重复订单。

所以数据库唯一索引的作用不是替代 Redis 和 Consumer 判重，而是做最终防线。

7.10.2 唯一索引应该怎么设计

推荐在订单表 `tb_voucher_order` 上增加联合唯一索引：

- `unique(user_id, voucher_id)`

它表达的业务含义非常明确：

- 同一个用户
- 对同一张优惠券
- 最多只能生成一笔订单

面试时你可以这样说：

我把一人一单做成三层保障：Redis Lua 在入口拦截，Consumer 业务逻辑里再判重，数据库层再通过 `unique(user_id, voucher_id)` 做最终兜底。这样即使 Kafka 发生重复消费，也不会生成重复订单。

7.10.3 唯一索引落地 SQL

你可以准备下面这段 SQL：

```
ALTER TABLE tb_voucher_order
ADD CONSTRAINT uk_user_voucher UNIQUE (user_id,
voucher_id);
```

如果表里已经有历史脏数据，正式执行前要先排查重复数据。

7.10.4 有了唯一索引后，Consumer 应该怎么办

加了唯一索引之后，不代表应用层判重就不要了。

更合理的做法是：

1. 先做业务判重，减少无意义数据库异常
2. 如果极端情况下还是插入重复
3. 捕获唯一索引冲突异常
4. 把它识别成“幂等命中”，而不是系统故障

也就是说，在面试里你要体现一个思想：

唯一索引不是为了替代应用逻辑，而是为了在异常边界下兜底。

7.11 为什么需要死信队列

手动确认 + 重试已经能解决一部分问题，但还有一种场景：

- 某条消息就是处理不了

比如：

- 消息体字段缺失
- JSON 结构错误
- 对应优惠券数据已经损坏
- 下游依赖异常持续很久
- 消费代码遇到不可恢复异常

如果这类消息一直重试，就会带来两个问题：

1. 反复报错，占用消费资源
2. 阻塞后续正常消息

所以要引入死信队列，把“短期无法成功处理的消息”隔离出去。

7.11.1 死信队列在这套秒杀架构里的作用

死信队列不是为了让失败消息消失，而是为了：

- 不阻塞主消费链路
- 保留失败消息现场
- 支持后续人工排查
- 支持补偿任务或人工重放

推荐死信 Topic：

- `seckill-order-dlt`

其中 `dlt` 表示 dead letter topic。

7.11.2 推荐的消费失败处理顺序

你可以把失败处理流程讲成下面这样：

1. Kafka Consumer 收到秒杀订单消息
2. 执行业务处理
3. 如果失败，先进行有限次重试
4. 重试仍失败，则不再阻塞主链路
5. 将消息投递到死信 Topic
6. 死信消费者落失败任务表并标记 `RETRY_PENDING`
7. 定时任务自动重试，成功后更新为 `DONE`
8. 超过阈值更新为 `MANUAL_HANDLE_REQUIRED`，走人工处理

这个表达会让面试官觉得你不是只会“失败就打日志”，而是有完整失败治理思路。

7.11.2.1 当前项目已落地的处理动作

当前版本已经具备以下治理闭环，可直接在面试中陈述：

- 自动链路：`主消费失败 -> DLT -> 失败任务表 -> 定时重试 -> DONE`
- 人工链路：`MANUAL_HANDLE_REQUIRED -> 手动重试/人工忽略`
- 管理接口：
 - `GET /voucher-order-fail-task/page`
 - `POST /voucher-order-fail-task/{id}/retry`
 - `POST /voucher-order-fail-task/{id}/ignore`

7.11.2.2 面试官追问：你怎么观测 Kafka 堆积？Redis 和 DB 库存怎么对账？

Kafka lag: 用 `AdminClient` 拉取主 Topic 各分区的 **latest offset**，再读消费组已提交 offset，分分区做 `end - committed` 后汇总。项目里由 `KafkaLagMonitorScheduler` 周期执行，超过 `app.kafka.monitor.lag-threshold` 打 **WARN**，便于接入日志告警或后续接 Prometheus。

库存对账: 秒杀场景以 `tb_seckill_voucher` 为库存事实源之一，定时比对 Redis 键 `seckill:stock:{voucherId}` 与表中 `stock`；DB 仍大于 0 但 Redis 无键、或两边数值不一致，打 **WARN**。注意异步窗口内有短暂不一致可能，对账更适合做「持续巡检」而非毫秒级强一致证明。

7.11.3 重试次数怎么设计

推荐思路：

- 即时重试 3 次
- 指数退避或固定退避
- 超过阈值进入死信 Topic

为什么不无限重试？

因为无限重试会放大坏消息的破坏力，让整个消费线程长期被一条异常消息拖住。

你可以这样回答：

我会把异常分成可恢复异常和不可恢复异常。可恢复异常做有限次重试，不可恢复异常或者重试超限则进入死信 Topic，避免一条坏消息拖垮整个消费链路。

7.12 Redis 已预扣库存但订单落库失败，怎么补偿

这是秒杀项目里最能体现你理解深度的问题之一。

因为你的链路是：

1. Redis Lua 先预扣库存
2. Redis Stream 记录受理消息
3. Relay 转发 Kafka
4. Kafka Consumer 最终落 MySQL

这意味着：

- Redis 上的变化发生在前
- MySQL 上的最终业务落库发生在后

如果后半段失败，就会出现数据不一致。

7.12.1 典型不一致场景

最典型的是：

1. Lua 校验通过
2. Redis 库存已减 1
3. 用户购买标记已写入 Redis
4. 消息已经进入 Stream/Kafka
5. 但 Consumer 落库失败
6. 最终 MySQL 没有订单，数据库库存也没真正对应扣减成功

这时如果不补偿，就会出现：

- Redis 库存比真实库存更少
- 用户在 Redis 中被标记成“已购买”
- 但数据库实际上并没有订单

这会直接影响后续业务正确性。

7.12.2 补偿的目标是什么

补偿不是简单“重试一下”，而是要恢复系统业务一致性。

目标一般有两个方向：

方向一：重试落库成功

即：

- 保留当前 Redis 状态不变
- 继续把失败消息重试或重放
- 最终让 MySQL 成功创建订单

这种方案适合：

- 失败原因是暂时性的
- 比如数据库短时抖动、网络闪断、下游依赖瞬时异常

方向二：回滚 Redis 侧预扣结果

即：

- Redis 库存加回去
- 删除用户购买标记
- 把这次秒杀资格恢复掉

这种方案适合：

- 明确无法成功落库
- 比如数据脏了、消息非法、业务规则判定必须失败

7.12.3 生产上更常见的思路

生产里更常见的是：

- 先尽量重试落库
- 重试超限后再进入补偿逻辑

原因很简单：

- 秒杀请求本来就已经向用户返回“受理成功”
- 从用户体验上，优先保证订单最终成功通常更合理
- 只有明确无法完成时，才考虑回滚 Redis 资源

所以你可以面试里这么讲：

对于 Redis 预扣成功但 MySQL 落库失败的情况，我会优先走重试和死信补偿，尽量让订单最终创建成功；如果确认无法成功，再执行库存和购买标记的回滚补偿。

7.13 补偿方案怎么设计

7.13.1 补偿数据来源

补偿任务至少要能拿到这些信息：

- orderId
- userId
- voucherId
- traceId
- 失败原因
- 失败次数
- 首次失败时间
- 最后一次失败时间

这些数据可以来自：

- 死信 Topic
- 失败任务表
- 消费错误日志归集

更推荐的是：

- 死信 Topic + 失败任务表结合

这样既有消息链路，也有可查询可运营的数据视图。

7.13.2 补偿任务的执行方式

可以设计一个定时任务，周期性扫描失败任务或死信消息，然后：

1. 判断是否还能重试
2. 如果可以，重新投递 Kafka 或直接执行业务补偿
3. 如果确认不可恢复，则回滚 Redis 预扣库存和购买标记
4. 记录补偿结果

7.13.3 Redis 回滚要回滚什么

如果最终决定回滚，就至少要处理两部分：

1. 库存 Key
2. 用户已购买标记

也就是把 Lua 中成功写入的内容反向撤销。

你面试时可以这样说：

补偿不是只补 MySQL，还要把 Redis 里的预扣库存和一人一单标记一起考虑，否则会出现库存和购买资格被长期错误占用。

7.13.4 补偿时最容易忽略的问题

补偿流程本身也要幂等。

比如：

- 同一条死信被重复处理
- 补偿任务执行到一半中断
- 回滚 Redis 后又被再次回滚

所以补偿也建议有状态机，例如：

- INIT
- RETRYING
- SUCCESS
- ROLLBACK_SUCCESS
- MANUAL_HANDLE_REQUIRED

这样你在面试里就能体现“补偿任务自身也需要可控、可恢复、可幂等”。

7.14 这三件事放在一起，面试怎么讲

你可以把“唯一索引 + 死信 + 补偿”总结成一套完整可靠性方案：

`Kafka` 只是把消息异步化了，但真正让秒杀链路可靠，还要补三层能力：第一层是 `Consumer` 成功后手动提交 `offset`，避免提前确认；第二层是数据库唯一索引兜底重复消费，避免重复订单；第三层是失败重试、死信队列和补偿任务，处理 `Redis` 已预扣但 `MySQL` 落库失败的最终一致性问题。这样整条链路才从“能跑”升级成“可恢复、可兜底、可解释”。

7.15 面试官追问示例

1. 已经有 `Redis` 一人一单了，为什么还要数据库唯一索引

答：

`Redis` 的一人一单主要解决入口高并发拦截，但 `Kafka` 消费是异步链路，可能出现重复投递、重复消费和补偿重放。数据库唯一索引是最终防线，用来兜住所有应用层漏网情况。

2. 为什么需要死信，而不是一直重试

答：

因为一直重试会让坏消息长期阻塞主链路。更合理的做法是有限次重试，超过阈值进入死信 `Topic`，把失败消息隔离出来，后续再人工排查或自动补偿。

3. Redis 预扣成功但 MySQL 失败，为什么不立刻回滚

答：

因为很多失败是暂时性的，比如数据库抖动、网络瞬时异常，直接回滚会让订单链路反复震荡。更合理的是先重试和死信补偿，尽量让订单最终成功，只有确认不可恢复时再回滚 Redis。

7.15 Kafka、RabbitMQ、RocketMQ 怎么对比

这部分在面试里非常常见，尤其当你已经讲了“我在秒杀项目里用了 Kafka”之后，面试官很容易追问：

- 为什么选 Kafka，不选 RabbitMQ？
- 为什么不是 RocketMQ？
- 如果换成 RocketMQ / RabbitMQ 行不行？

你要注意，这类问题不是在考你背八股，而是在考你有没有“结合业务场景做技术选型”的能力。

7.15.1 先给一句总回答

你可以先用一句总括回答把方向定住：

这三种消息队列都能做异步削峰，但如果结合秒杀这种高吞吐、可堆积、可恢复的场景，我更倾向于 **Kafka**；如果更强调灵活路由和传统业务系统集成，**RabbitMQ** 会更合适；如果更强调消息可靠性、顺序、延时和事务消息能力，**RocketMQ** 也很适合做订单场景。我的这个项目之所以选 **Kafka**，是因为我想突出高吞吐削峰、消费恢复和可扩展消费能力。

这句话好处是：

- 先承认三者都能用

- 再强调是“结合场景做选择”
- 最后把话题拉回你当前项目的技术决策

7.16 Kafka、RabbitMQ、RocketMQ 的核心差异

7.16.1 Kafka 的特点

Kafka 最突出的标签是：

- 高吞吐
- 分区模型清晰
- 对消息堆积和消费恢复很强
- 适合日志、埋点、异步削峰、大流量消息处理

Kafka 的优势在于：

1. 吞吐量高，特别适合高并发大流量场景
2. 消息持久化和顺序写磁盘机制比较强
3. Consumer 组扩展能力强，适合横向扩容
4. 对消息积压的承受能力通常比较好
5. 很适合做“高峰流量先接住，再慢慢消费”的模型

Kafka 的短板也要知道：

1. 它不是以复杂路由见长
2. 业务语义不像 RabbitMQ 那么灵活
3. 传统业务里如果要做很多精细投递控制，开发体验不一定最好
4. 事务消息、延迟消息这类能力不是它最突出的一面

7.16.2 RabbitMQ 的特点

RabbitMQ 最突出的标签是：

- 协议成熟

- 路由灵活
- 交换机模型丰富
- 适合传统业务系统异步解耦

RabbitMQ 的优势在于：

1. Exchange + Queue + Binding 模型很灵活
2. 路由能力强，适合复杂业务分发
3. 社区成熟，很多企业业务系统都在用
4. 对低延迟、小规模业务异步场景很友好

RabbitMQ 的短板：

1. 极高吞吐场景下一般不如 Kafka
2. 面对超大规模消息堆积时，扩展性和处理模型通常不如 Kafka 自然
3. 如果你要讲秒杀削峰和海量消息承压，RabbitMQ 的说服力一般不如 Kafka 或 RocketMQ

7.16.3 RocketMQ 的特点

RocketMQ 最突出的标签是：

- 业务消息能力强
- 顺序消息、延迟消息、事务消息都比较适合电商场景
- 对订单、交易、通知类业务有很强工程适配性

RocketMQ 的优势通常包括：

1. 更贴近业务型消息队列
2. 顺序消息支持更容易讲
3. 延时消息、事务消息在业务系统里很有代表性
4. 在订单、支付、库存类场景里很常见

RocketMQ 的短板也要会说：

1. 社区生态和通用性上，很多场景下 Kafka 更常见

2. 如果场景重点是超大吞吐、日志流、埋点流、海量异步削峰，Kafka 往往更自然
3. 如果你的项目本身并没有用到事务消息和复杂延时语义，RocketMQ 的优势不一定完全发挥出来

7.17 结合秒杀场景，三者怎么选

你面试时最好不要只横向背特性，而是结合秒杀业务来讲。

对秒杀最关键的几个点

秒杀系统最关注的通常是：

1. 高并发流量削峰
2. 消息积压能力
3. 服务重启后的恢复能力
4. 消费侧扩容能力
5. 幂等与最终一致性配合能力

如果按这几个维度去看：

Kafka

更适合强调：

- 高吞吐削峰
- 消息积压
- 可扩展消费
- 服务重启后继续消费

RabbitMQ

更适合强调：

- 路由灵活
- 业务模块解耦

- 中小规模业务异步通知

RocketMQ

更适合强调：

- 电商业务消息
- 顺序消息
- 延时消息
- 事务消息

7.18 为什么我这个项目更适合讲 Kafka

你这个项目当前链路是：

- Redis Lua 前置原子校验
- Redis Stream 本地承接受理消息
- Relay 转发 Kafka
- Kafka Consumer 异步落库

从这个设计出发，Kafka 在这里最适合强调的是：

1. 高并发削峰能力

秒杀流量会在短时间内集中打进来，Kafka 可以把瞬时高峰流量先承接住，再由 Consumer 平滑处理。

2. 消息积压和恢复能力

如果消费端一时处理不过来，Kafka 更适合承接积压。

这点面试里很好讲，因为它比“单机线程池异步下单”更有工程深度。

3. Consumer 组扩展能力

如果后续订单量进一步提升，可以通过增加 Consumer 实例和分区数来扩展消费能力。

4. 更适合讲可靠性增强

例如：

- 手动 ack
- 死信
- 重试
- 补偿
- 幂等

这些都能和 Kafka 消费模型自然结合起来讲。

所以你可以这么总结：

我选 kafka，不是因为它唯一能做秒杀，而是因为这套项目的重点想放在高吞吐削峰、消息积压恢复和可扩展消费能力上，这些点和 kafka 的特点是相匹配的。

7.19 如果面试官问：为什么不用 RabbitMQ

这是高频追问。

你可以这样回答：

RabbitMQ 也能做异步削峰，但它更强的是灵活路由和传统业务解耦。我的这个秒杀项目重点不在复杂路由，而在高峰流量承接、消息积压和消费恢复，所以我更倾向于 kafka。尤其是秒杀这种突发流量场景，用 kafka 来承接异步下单请求，更容易讲高吞吐和可扩展消费。

如果想再进一层，可以补一句：

如果项目是很多业务模块之间做事件通知、延迟不高、消息量也没那么极端，RabbitMQ 其实会是很合理的选择。

这样回答会显得你不是在“踩 RabbitMQ”，而是在理性选型。

7.20 如果面试官问：为什么不用 RocketMQ

这个问题也非常常见，尤其是 Java 岗和电商业务岗。

你可以这样回答：

RocketMQ 其实也很适合订单类业务，尤其是顺序消息、延时消息、事务消息这些能力，在电商系统里很有代表性。如果我这个项目重点想突出事务消息或者订单链路里的业务语义能力，RocketMQ 也是不错的选择。之所以这里用 Kafka，是因为我这版改造更想强调高并发削峰、消息积压承接和消费扩展能力。

如果还想再补强一句：

换句话说，不是 RocketMQ 不合适，而是我这个项目当前讲法更偏高吞吐异步削峰，所以 Kafka 更贴近我想表达的架构重点。

7.21 如果面试官继续追问：那到底谁更好

正确回答一定不是“某一个绝对更好”，而是：

没有绝对更好的消息队列，关键看业务侧重点。

你可以按下面方式回答：

- 如果我要强调高吞吐、积压能力、可扩展消费，我倾向 Kafka
- 如果我要强调灵活路由、业务模块解耦，我倾向 RabbitMQ
- 如果我要强调订单业务能力、顺序消息、延迟消息、事务消息，我倾向 RocketMQ

这就是一个非常成熟的面试回答了。

7.22 一张面试可直接背的对比表

Kafka

- 优势：高吞吐、可堆积、易扩展、适合削峰
- 短板：复杂业务路由能力一般
- 适合：秒杀削峰、日志流、埋点流、大流量异步场景

RabbitMQ

- 优势：路由灵活、协议成熟、业务解耦友好
- 短板：极高吞吐和超大积压场景通常不如 Kafka
- 适合：中小规模业务异步、通知、复杂路由分发

RocketMQ

- 优势：业务消息能力强，顺序/延时/事务消息更适合电商
- 短板：如果不需要这些业务能力，优势不一定能完全体现
- 适合：订单、支付、库存、交易型业务链路

7.23 你这个项目最推荐的回答模板

简版

RabbitMQ、RocketMQ、Kafka 都能做秒杀异步化，但我这个项目重点想突出高并发削峰、消息积压承接和消费扩展，所以我选了 **Kafka**；如果更强调复杂路由，我会考虑 **RabbitMQ**；如果更强调订单业务里的顺序、延时和事务消息，我会考虑 **RocketMQ**。

进阶版

我不会把 MQ 选型讲成谁绝对更强，而是看业务重点。我的秒杀项目入口已经用 Redis Lua 解决了原子校验问题，后面的核心诉求是把瞬时高峰流量异步化并平滑消费，所以我更关注吞吐、堆积和消费恢复能力，这些点和 Kafka 更匹配。RabbitMQ 更适合复杂路由型业务，RocketMQ 更适合强调顺序、延时、事务消息的订单链路。如果后续我想把订单链路进一步往事务消息方向升级，RocketMQ 也是很值得考虑的方案。

7.23 这次项目真实排障过程怎么讲

这一部分其实非常有面试价值。

因为很多时候，面试官不只想听“你做了什么功能”，还想知道：

- 项目遇到问题时你怎么定位
- 你能不能区分表象和根因
- 你是怎么一步步把系统从“起不来”修到“能跑通”的

这次你这个项目的排障过程，完全可以整理成一个比较完整的工程案例。

7.24 排障背景

在把秒杀链路升级成：

- Redis Lua 原子校验
- Redis Stream 承接受理消息
- Relay 转发 Kafka
- Kafka Consumer 异步落库

之后，项目在启动阶段连续报错，表现为多个 Bean 创建失败。

一开始看上去像是：

- Redis 有问题
- Kafka 有问题
- Bean 注入有问题

- 甚至像是代码逻辑写坏了

但真正排下来，问题是分层出现、连续暴露的。

这正好适合你在面试里讲“我怎么做系统化排障”。

7.25 第一阶段：先识别这是编译产物问题，不是业务逻辑问题

最开始的报错集中在这些类：

- `RedisStreamToKafkaRelay`
- `VoucherOrderProducerImpl`
- `VoucherOrderConsumer`
- `SeckillStockPreheatRunner`
- `VoucherOrderMessage`
- `VoucherOrder`
- `SeckillVoucher`

典型异常是：

- `log cannot be resolved`
- `The blank final field ... may not have been initialized`
- `setXxx(...)` is undefined
- `getXxx(...)` is undefined

这类报错有一个非常典型的特征：

- 不是运行期业务异常
- 而是“编译时应该生成的方法和字段没有生成”

我当时的判断是：

这不是 `Kafka` 本身的问题，也不是 `Redis` 逻辑问题，而是 `Lombok` 相关代码生成在当前构建链路里没有生效，导致 `target/classes` 里的编译产物不完整。

为什么能判断是 Lombok 问题

因为这些缺失项都高度集中在 Lombok 自动生成的内容上：

- `@Slf4j` 生成的 `log`
- `@RequiredArgsConstructor` 生成的构造器
- `@Data` 生成的 `getter/setter`

而且错误不是偶发出现在单个类，而是成片出现在多个新接入或依赖 Lombok 的类上。

所以第一步不是去怀疑 `Kafka` 消息没发出去，而是先把项目编译产物修到可启动。

7.26 第二阶段：先把启动链上的 Lombok 依赖类“显式化”

因为当时最紧急的目标不是“研究 IDE 为啥 Lombok 失效”，而是先让项目恢复可运行。

所以处理策略是：

- 对启动链路上直接报错的类
- 去掉关键 Lombok 依赖
- 改成显式构造器、显式 `logger`、显式 `getter/setter`

具体做法包括：

1. `RedisStreamToKafkaRelay`

处理内容：

- 去掉 @Slf4j
- 去掉 @RequiredArgsConstructor
- 手写 Logger
- 手写构造器

2. VoucherOrderProducerImpl

处理内容：

- 去掉 @Slf4j
- 去掉 @RequiredArgsConstructor
- 手写 Logger
- 手写构造器

3. VoucherOrderConsumer

处理内容：

- 去掉 @Slf4j
- 去掉 @RequiredArgsConstructor
- 手写 Logger
- 手写构造器

4. VoucherOrderMessage

处理内容：

- 去掉 @Data
- 手写 getter/setter

5. VoucherOrder

处理内容：

- 去掉 @Data
- 补齐关键 getter/setter，避免 Consumer 组装订单时报错

6. SeckillVoucher

处理内容：

- 去掉 `@Data`
- 补齐关键 getter/setter，避免库存预热逻辑报错

7. SeckillStockPreheatRunner

处理内容：

- 去掉 `@Slf4j`
- 手写 `Logger`

面试时这一步怎么讲

你可以这么说：

当时我没有先陷在工具链问题里，而是先按启动链路优先级把关键类显式化，先恢复应用可启动性。这样做的目的是快速止血，让系统先从“完全起不来”恢复到“可以继续验证主链路”。

这句话会显得你有工程优先级意识。

7.27 第三阶段：分清“真正主因”和“伴随异常”

项目修到能继续往下启动之后，又出现了一波异常：

- `RejectedExecutionException: event executor terminated`
- `RedisConnectionException: Unable to connect to Redis server`
- `OutOfMemoryError: Unable to allocate 16777216 bytes`

如果经验不足，很容易把 `RejectedExecutionException` 当主因。

但我当时做了一个判断：

`RejectedExecutionException` 只是 `Netty/Redisson` 在线程池收尾阶段抛出的伴随异常，真正的主因还是 `Redisson` 创建阶段连 `Redis` 失败，并且伴随直接内存分配压力。

也就是说，要学会区分：

- 表面上最先刷出来的异常
- 真正导致 Spring 容器启动失败的根因

这一步怎么判断

我主要看了异常链的落点：

- Spring Bean 创建失败最终落到了 `redissonClient`
- `redissonClient` 创建失败最终落到了 `Unable to connect to Redis server`
- 同时异常底层还有 `OutOfMemoryError: Unable to allocate 16777216 bytes`

所以这一阶段的结论不是“Kafka 有问题”，而是：

- `Redisson` 配置是刚性写死的
- `Redis` 连接创建阶段存在资源压力
- `Netty` 报错只是连带现象

7.28 第四阶段：优化 `Redisson` 配置，降低资源压力

当时 `RedissonConfig` 里是硬编码：

- `127.0.0.1:6379`

- 固定密码

而且连接参数没有做更细粒度控制。

所以我做了两个调整：

1. Redisson 改成读取 `application.yaml`

这样做的目的：

- 避免配置散落两处
- 避免 Spring Redis 和 Redisson 配置不一致
- 后续环境切换更方便

2. 缩小连接池并调低启动期资源占用

例如控制：

- 连接池大小
- 最小空闲连接数
- 订阅连接池大小
- 超时和重试参数

另外还把：

- `logging.level.com.hmdp`

从 `debug` 降到了 `info`，减少启动阶段日志和资源消耗。

面试时怎么讲

你可以这么说：

我没有只停留在“Redis 连不上”这个表层，而是继续看异常链，发现 Redisson 创建阶段存在连接失败和直接内存压力，所以我把配置收口到 application.yaml，并且缩小了连接池规模，降低了启动期资源占用。

7.29 第五阶段：验证 Kafka 链路是否真正生效

项目能启动之后，还不能直接说 Kafka 改造成功。

因为“应用启动成功”和“Kafka 链路真的打通”是两回事。

所以我继续做了链路验证。

验证点 1：看 Kafka Producer 初始化日志

启动日志里出现了类似信息：

- Instantiated an idempotent producer
- Cluster ID ...
- ProducerId set ...

这说明：

- Producer 已经成功初始化
- 已经连上 Kafka Broker
- 幂等 Producer 也生效了

验证点 2：看 Kafka Consumer 是否成功订阅 Topic

日志里能看到：

- 成功加入 consumer group
- 分区分配成功
- 成功订阅 seckill-order

这说明消费侧也已经起来了。

验证点 3：看 Redis Stream Relay 是否正常启动

日志里出现：

- `Stream group already exists, stream=stream.orders, group=relay-g1`

说明 Relay 线程也正常启动并接入了 Redis Stream 消费组。

验证点 4：看库存预热是否正常

日志里出现：

- `Seckill stock preheat finished, loaded 2 records to Redis.`

说明预热逻辑也打通了。

这一阶段可以怎么总结

你可以面试里说：

我不是只看应用起没起来，而是把 Producer、Consumer、Relay、库存预热四个关键节点都验证了一遍，确认 Redis -> Relay -> Kafka -> Consumer 这条链路已经真实打通。

7.30 第六阶段：从“能启动”继续推进到“可讲可靠性”

在项目恢复运行后，我没有停在“系统能跑”这个状态，而是继续往可靠性层面思考。

后续重点规划了几件事：

1. 把 Kafka Consumer 从自动提交 offset 改成手动确认
2. 为订单表增加 `unique(user_id, voucher_id)` 唯一索引
3. 增加死信 Topic
4. 增加消费失败补偿和库存回滚补偿
5. 补充 traceId 和链路日志

这部分很适合面试时表达成：

我做的不是一次简单的功能接入，而是先把项目从编译失败和启动失败里救起来，再继续往消息可靠性和最终一致性方向补强。

7.31 这次排障过程的面试表达模板

简版回答

这次改 kafka 秒杀链路时，项目一开始其实并不是业务逻辑错了，而是启动链上多个类依赖 Lombok 自动生成代码，但当前构建产物里这些代码没有生效，导致 log、构造器、getter/setter 大量缺失。我先按启动优先级把这些关键类改成显式代码，让项目恢复可启动。之后又继续排查 Redisson 初始化阶段的 Redis 连接和内存压力问题，把配置收口并缩小连接池，最终把 Redis 预热、Redis Stream Relay、Kafka Producer、Kafka Consumer 整条链路都跑通了。

进阶版回答

这次排障我分了三个层次。第一层是识别问题类型，我发现报错集中在 log cannot be resolved、final field not initialized、getter/setter undefined，这些都是 Lombok 失效的典型特征，所以我先判断这是构建产物问题，不是业务逻辑问题。第二层是按启动链路止血，我把 RedisStreamToKafkaRelay、Producer、Consumer、消息模型和关键实体改成显式构造器、显式 logger、显式 getter/setter，让应用先恢复启动。第三层是继续往下排真正运行期问题，我定位到 Redisson 创建阶段的 Redis 连接失败和直接内存压力，调整了 Redisson 配置和日志级别。最后我没有只停在应用启动成

功，而是进一步验证了 `Kafka Producer`、`Consumer`、`Redis Stream Relay` 和库存预热都已经生效，并继续规划手动 `ack`、唯一索引、死信和补偿，把链路往可靠性方向再推进。

7.32 这次排障能体现哪些能力

如果面试官愿意深挖，你还可以顺手点出自己体现的能力：

1. 能区分工具链问题和业务问题

不是一看到 MQ 报错就认为 Kafka 有问题，而是能从异常特征判断这是 Lombok 代码生成失效。

2. 有启动优先级意识

不是一上来大改所有代码，而是先修启动链上直接阻塞的类，让系统尽快恢复可运行。

3. 能区分根因和伴随异常

没有被 `RejectedExecutionException` 这种表象异常带偏，而是继续顺着异常链找到 Redisson 初始化失败和资源压力。

4. 有链路验证意识

项目启动成功之后，还继续验证 `Producer`、`Consumer`、`Relay`、预热是否都真正生效。

5. 有工程补强意识

不是“跑通就结束”，而是继续规划手动 `ack`、死信、唯一索引和补偿方案。

9. 10 分钟长版面试讲稿

下面这版适合你在面试里系统讲述这个 Kafka 秒杀链路改造。

它不是单纯背八股，而是按照一个比较自然的项目表达顺序展开：

- 先讲业务背景
- 再讲原始问题
- 再讲技术方案
- 再讲链路细节
- 再讲可靠性
- 最后讲排障与优化

你可以把它理解成一版“项目复盘式表达模板”。

如果让我系统介绍这次 Kafka 秒杀链路改造，我一般会从业务背景开始讲。

我这个项目本质上是一个类似黑马点评的本地生活服务后端，其中有一个比较典型的高并发场景，就是优惠券秒杀。秒杀的核心难点其实不是普通 CRUD，而是要同时解决几个问题：

- 第一，高并发下库存不能超卖；
- 第二，同一个用户不能重复下单；
- 第三，接口不能把所有压力直接打到数据库；
- 第四，整个链路要尽量做到异步化、可恢复和可扩展。

所以我在这个项目里做的，不是简单把一个同步下单接口改成“发个 MQ”而已，而是把整条秒杀链路升级成了一套 Redis + Kafka + MySQL 分工明确的架构。

我通常会先用一句话概括这套方案：

Redis 负责秒杀入口的原子校验，Kafka 负责异步削峰和消费恢复，MySQL 负责最终订单落库和业务事实。

具体来说，请求入口是在 `POST /voucher-order/seckill/{id}`。用户发起秒杀之后，接口线程不会直接去做数据库事务，而是先进入 Redis Lua 脚本。Lua 里主要做三件事：

- 第一，判断库存是否充足；
- 第二，判断用户是不是已经下过单，也就是一人一单；
- 第三，如果校验通过，就在 Redis 里预扣库存，并且把这次请求写入 Redis Stream。

这里 Redis 的作用非常关键，因为秒杀这种场景如果把库存判断和一人一单校验放在数据库层，会导致数据库在高并发下很快成为瓶颈。而 Lua 的好处是它能把库存校验、一人一单校验和消息写入 Redis Stream 这几个操作做成一个原子过程。也就是说，在入口阶段，我就可以先用 Redis 快速拦掉大量无效请求，把真正有资格下单的请求保留下来。

不过我这里并没有停留在 Redis Stream 本身，而是继续往后做了一层扩展。我新增了一个 Relay 组件，专门负责从 Redis Stream 里读取消息，再转发到 Kafka Topic。这个设计是我觉得这个项目里比较值得讲的一点。

因为如果只做到 Redis Stream，其实更多像是“本地异步化”；而我引入 Kafka 之后，整条链路就升级成了一个更适合讲削峰、积压、扩展和消费恢复的消息架构。

所以现在的链路变成了：

用户请求 -> Redis Lua 原子校验 -> Redis Stream 落受理消息 -> Relay 转发 Kafka -> Kafka Consumer 异步消费 -> MySQL 扣减库存并创建订单。

其中 Relay 的职责是这样的：

它会作为 Redis Stream 消费组消费者持续读取 `stream.orders`，把消息转换成统一的 `VoucherOrderMessage`，然后调用 Kafka Producer 发到 Topic `seckill-order`。这一层我专门做成了“Kafka 发送成功后再 ACK Redis Stream 消息”。

这个点面试里很好讲，因为它体现的是中间桥接层的可靠性设计。也就是说，不是从 Redis Stream 读到了消息就立刻确认，而是要等 Kafka 真正收到了消息之后，才去 ACK Redis Stream，尽量降低 Redis 到 Kafka 这一段的丢消息风险。

再往后就是 Kafka Consumer。Consumer 监听 `seckill-order` Topic，拿到消息后会先把 JSON 反序列化成 `VoucherOrderMessage`，然后组装成 `VoucherOrder` 领域对象，再调用订单服务去执行业务落库逻辑。

这里真正的业务落库不是“直接插一条订单”这么简单，而是还要做消费侧的业务兜底。主要包括几个点：

- 第一，按用户维度加分布式锁，避免同一用户并发重复创建订单；
- 第二，再查一次数据库，确认用户是不是已经下过单；
- 第三，扣减数据库库存时必须带 `stock > 0` 条件，防止并发下库存扣成负数；
- 第四，库存扣减成功后才真正保存订单。

所以整条链路里，Redis 是入口原子校验，Kafka 是异步削峰和可恢复消费，MySQL 才是最终业务结果。这个职责边界我在面试里一定会讲清楚，因为很多面试官就爱问一句：“既然用了 Kafka，为什么还要 Redis？”

我的回答一般是：

`Kafka` 擅长异步传输和削峰，但不擅长做高并发库存校验和一人一单这种入口原子逻辑。秒杀场景最关键的是在请求入口快速完成资格判断，所以 `Redis` 不能去掉。

接下来我一般会继续讲，为什么这个项目要引入 Kafka，而不是继续停留在本地线程池或者纯 Redis Stream。

我觉得主要有四个原因：

第一，Kafka 更适合承接秒杀这种高峰流量，把瞬时洪峰先接住，再让消费端慢慢处理；

第二，Kafka 对消息积压和消费恢复能力更强，服务重启后也更容易继续消费；

第三，Kafka 的 Consumer Group 模型更适合后续做横向扩容；

第四，从面试表达上，它能让我进一步讲到手动 ack、重试、死信、补偿和最终一致性，而不是只停留在“我用了异步线程池”。

当然，我也不会把 Kafka 讲成唯一正确答案。如果面试官追问为什么不用 RabbitMQ 或 RocketMQ，我会说：

RabbitMQ 更适合灵活路由和传统业务模块解耦；RocketMQ 更适合讲顺序消息、延迟消息、事务消息这些偏交易型业务能力；而我这个项目当前更想强调的是高吞吐削峰、消息积压和消费恢复，所以我选 Kafka 会更贴合我要表达的架构重点。

然后我会继续往可靠性上讲，因为秒杀项目里只把消息发出去还远远不够。

我现在这个版本虽然已经把主链路打通了，但从工程可靠性角度，我后续还会继续补几件事。

第一件事，是把 Kafka Consumer 从自动提交 offset 升级成手动确认。

因为当前如果配置的是自动提交，那么消息只要被 Consumer 拉到了，offset 就可能先提交，但这并不代表订单已经真正落库成功。最危险的情况是：Kafka 这边 offset 提交了，但 MySQL 订单还没创建成功，或者业务执行到一半服务挂了，那这条消息重启后就不会再消费，最终会出现 Redis 已经预扣库存，但数据库没有订单的丢单问题。

所以更合理的做法是把确认时机后移到业务成功之后。也就是 Consumer 处理成功后再 `ack.acknowledge()`，如果业务失败就不确认，让 Kafka 后续重试。这个点我会明确表达成：

对于秒杀订单这种场景，我更关注业务成功和消息确认时机的强绑定，所以会把 Consumer 改成手动 ack，只有订单真正落库成功后才提交 offset。

第二件事，是数据库唯一索引兜底。

虽然入口已经有 Redis Lua 做一人一单，消费侧也有 Redisson 锁和业务判重，但 Kafka 消费模型本质还是至少一次投递，极端情况下仍然可能重复消费。所以我建议在 `tb_voucher_order` 上增加 `unique(user_id, voucher_id)` 联合唯一索引，作为最终防线。

这样我在面试里就能讲成“三层幂等”：

- Redis Lua：入口防重复
- Consumer 业务逻辑：消费侧防重复
- DB 唯一索引：最终兜底

第三件事，是补死信队列和补偿方案。

因为秒杀链路里最麻烦的场景其实不是普通失败，而是 Redis 这边已经预扣库存并记录了购买标记，但 Kafka Consumer 最终落库失败了。这个时候如果不补偿，就会出现 Redis 库存变少了，用户也被标记成买过，但数据库里根本没有订单。

对于这类情况，我的思路是分两步：

先尽量通过有限次重试和死信补偿，让订单最终落库成功；
如果确认不可恢复，再执行 Redis 侧的回滚补偿，也就是把库存加回去，把用户购买标记删除掉。

这里我会强调一个观点：

补偿不是简单重试一下，而是为了恢复 Redis 和 MySQL 之间的业务一致性。

如果面试官继续问“为什么不直接回滚 Redis”，我也会说：

因为很多失败其实是暂时性的，比如数据库抖动、网络闪断，如果一失败就立刻回滚，反而会让链路来回震荡。更稳妥的做法是先重试，重试超限后再补偿。

接下来，我通常会把这次项目里的真实排障过程也讲出来，因为这个部分很容易体现工程能力。

我这次改造过程中，项目一开始其实不是 Kafka 逻辑写坏了，而是启动阶段连续报 Bean 创建异常。最开始的报错看起来很乱，有 `log cannot be resolved`、`final field may not have been initialized`、`getter/setter undefined` 之类。

我当时判断，这类错误不是普通业务逻辑报错，而是 Lombok 自动生成代码没有进入当前编译产物导致的。因为缺失的正好都是 `@Slf4j` 生成的 log、`@RequiredArgsConstructor` 生成的构造器、`@Data` 生成的 getter/setter。

所以我第一步不是去怀疑 Kafka 或 Redis 本身，而是先按启动链路优先级，把 `RedisStreamToKafkaRelay`、`VoucherOrderProducerImpl`、`VoucherOrderConsumer`、`VoucherOrderMessage`、`VoucherOrder`、`SeckillVoucher` 这些直接阻塞启动的类改成显式代码，先让项目恢复可启动。

项目继续往下启动后，又出现了 Redisson 初始化失败、Redis 连接异常和直接内存压力相关的问题。这里我没有被表面的 Netty 线程池异常带偏，而是顺着异常链继续看，发现真正落点是在 `redissonClient` 创建阶段，所以又去调整了 Redisson 配置，把配置统一收口到 `application.yaml`，并且缩小连接池、降低日志级别，减少启动期资源占用。

最后项目能启动之后，我也没有直接宣布“改造完成”，而是继续验证了四个关键节点：

- 第一，Kafka Producer 是否已经正常初始化；
- 第二，Kafka Consumer 是否已经成功加入消费组并订阅分区；
- 第三，Redis Stream Relay 是否已经正常启动；
- 第四，秒杀库存预热是否已经成功写入 Redis。

这些都验证通过之后，我才认为这条 Redis -> Relay -> Kafka -> Consumer -> MySQL 的链路已经真正打通。

所以如果让我总结这次项目改造，我不会只说“我在项目里用了 Kafka”，我会说：

我把原来的秒杀异步下单链路升级成了 Redis 前置原子校验 + Kafka 异步削峰 + MySQL 最终落库的架构。在实现上，我做了 Redis Stream 到 Kafka 的桥接、Kafka Producer/Consumer 的接入，以及消费侧业务幂等和库存安全控制；在可靠性上，我进一步规划了手动 ack、数据库唯一索引、死信和补偿方案；在落地过程中，我还处理了 Lombok 构建产物失效、Redisson 启动资源压力等问题，最终把整条链路从“起不来”修到“能跑通、能验证、能继续补强”。

如果面试时间足够，我最后还会补一句：

我觉得这次改造最大的价值，不只是把 Kafka 接进来了，而是把这个秒杀系统从“会跑”往“可恢复、可解释、可扩展”的方向推进了一步。

这就是一版 10 分钟左右、比较完整的项目讲法。

11. 本次实际落地改造摘要

这一节可以理解成对当前项目真实改造状态的一次收口总结。

和前面偏“设计思路、面试表达、方案规划”不同，这一节强调的是：

- 这次到底改了什么
- 哪些已经真正落地了
- 哪些已经验证过了

- 哪些还可以继续增强

如果面试官问你：

你这个 `Kafka` 秒杀链路，最后真正做到了什么程度？

你就可以直接按这一节回答。

11.1 当前已经真实落地的改造点

1. 秒杀入口仍然保留 Redis Lua 原子校验

当前接口仍然是：

- `POST /voucher-order/seckill/{id}`

入口逻辑仍然保留：

- 库存判断
- 一人一单判断
- Redis 预扣库存
- `XADD stream.orders`

这保证了秒杀最核心的原子资格校验仍然在 Redis 里完成。

2. Redis Stream 到 Kafka 的桥接已经完成

我新增了 Relay 组件，把 Redis Stream 中的秒杀订单消息继续转发到 Kafka。

这一步把原来“只在 Redis 体系内异步”的链路，升级成了：

- Redis 前置原子校验
- Kafka 异步削峰
- MySQL 最终落库

3. Kafka Producer 已经改成“真正发送成功才算成功”

这一点是本次改造里很关键的一个修正。

我一开始发现一个问题：

- 即使 Kafka Broker 不可用
- Producer 也可能因为只是把发送任务提交给客户端线程而提前返回 success
- 这会导致 Relay 误以为消息已经进 Kafka，从而错误 ACK Redis Stream

后来我把 Producer 改成了：

- 调用 `kafkaTemplate.send(...).get(...)`
- 同步等待 broker 返回确认
- 只有真正拿到 Kafka send result 才返回 success

这一步解决的是：

避免“客户端已提交发送任务”被误判成“Kafka 已成功接收消息”。

4. Relay 已经做到“Kafka 成功后再 ACK Redis Stream”

当前 Relay 的关键保证是：

- 只有 Producer 返回 true
- 才会 ACK Redis Stream

因为 Producer 现在已经改成同步等待 broker ack，所以这一步的语义也就更可靠了。

你可以把这句话直接拿去面试说：

我不是简单调用 `kafkaTemplate.send` 就算成功，而是等 broker 真正确认成功后，才 ACK Redis Stream，从而降低 Redis 到 Kafka 之间的丢消息风险。

5. Kafka Consumer 已经改成手动 ack

当前 Consumer 已经不是自动提交 offset，而是：

- 业务处理成功后才 `ack.acknowledge()`
- 业务失败则不确认 offset

这一步已经实际落地，不是停留在方案层。

6. Kafka Consumer 已经接入有限次重试

当前使用的是：

- `SeekToCurrentErrorHandler`
- `FixedBackOff(1000ms, 3次)`

也就是消费失败时会按 offset 重试，而不是直接把消息吞掉。

7. Redis Stream Relay 已经支持 pending-list 恢复

当前 Relay 在启动时会先执行 pending-list 恢复逻辑。

它的作用是：

- 如果之前有 Redis Stream 消息已经被读到，但还没成功转发 Kafka
- 那么应用重启后还能继续捞出这批 pending 消息再处理

这一步是当前补偿链路里的一个最小恢复能力。

8. 订单表唯一索引兜底已经落地

我已经补了唯一索引脚本：

- `src/main/resources/db/voucher_order_unique_index.sql`

对应索引是：

- `unique(user_id, voucher_id)`

并且数据库里也已经执行成功。

这意味着现在一人一单已经形成了三层兜底：

1. Redis Lua 入口拦截
2. Consumer 业务判重
3. DB 唯一索引最终兜底

9. 业务日志已经补了 traceId

当前链路日志已经能串起：

- Relay 转发成功/失败
- Kafka send 成功/失败
- Kafka consume 成功/失败
- 订单创建成功

并且日志里已经带：

- `traceId`
- `orderId`
- `userId`
- `voucherId`

所以现在已经具备比较完整的链路追踪能力。

11.2 当前已经验证过的结果

1. traceId 全链路已经打通

我已经实际验证过同一个 traceId 会依次出现在：

- `Relay forward success`
- `Kafka send success`
- `Kafka consume success`

这说明：

- Redis Stream -> Relay

- Relay -> Kafka
- Kafka -> Consumer

三段链路都已经打通。

2. 手动 ack 已经验证生效

我做过一次故意消费失败验证：

- Consumer 第一次消费时人为抛异常
- 这时消息不会 ack
- Kafka 会 seek 回原 offset
- 然后同一条消息再次被消费并成功

这说明：

- 当前确实不是自动提交 offset
- 而是业务失败不确认，业务成功才确认

3. Kafka 不可用时，接口“受理成功”语义已经明确

当前接口返回成功，不代表订单已经同步落库。

它代表的是：

- Redis Lua 校验成功
- Redis Stream 写入成功
- 请求已经被系统受理

所以 Kafka 挂掉时，接口依然可能返回成功，这是符合当前架构设计的。

这个点也已经在实际测试里验证和澄清过了。

4. 订单创建成功日志已经验证可见

当前日志中已经可以看到：

- `Create voucher order success`

说明 Consumer -> Service -> MySQL 这一步也已经真实落地，而不是只停留在消息投递层。

5. 基线压测已经做过

我已经做过几轮本地基线测试，包括：

- RedisIdWorker 吞吐测试
- 单用户高并发秒杀测试
- 多用户高并发秒杀测试
- **HTTP 层 JMeter 压测**（秒杀抢购.jmx + 多 token CSV + 响应断言白名单，见 秒杀链路测试结果记录.md 4.4）

结果已经整理在：

- 秒杀链路测试结果记录.md

其中已经验证：

- 单用户并发下，一人一单拦截生效
- 多用户并发下，当前服务层入口具备千级 QPS 的基础能力
- **HTTP 接口**在本地一轮 **3000 请求** 的聚合报告中，**Throughput 约 1900/s 量级、断言失败率 0%**（单机环境，用于面试与相对对比，非生产承诺）

11.3 当前这版方案的“接口成功”到底是什么意思

这个问题非常重要，因为它关系到你能不能把秒杀链路讲清楚。

当前这版接口成功的含义是：

请求已经通过 Redis 原子校验，并成功写入 Redis Stream，属于“已受理成功”。

它不等价于：

- Kafka 已经完成消费
- MySQL 订单已经创建成功
- 用户最终一定已经拿到订单

真正的最终成功，要看后续异步链路是否完成。

这个设计的好处是：

- 不把 Kafka 可用性直接绑死在接口响应上
- 让秒杀入口可以更快返回
- 提高高并发场景下的入口承载能力

你在面试里可以这么表达：

我的秒杀接口返回成功，表示请求已经被受理，不代表订单同步落库成功；最终订单创建由 Redis Stream、Kafka、Consumer 这条异步链路继续完成。

11.4 当前还没有完全做完，但已经明确下一步方向的内容

虽然这次已经把主链路和基础可靠性补强了不少，但还没有把所有增强都做完。

后续仍然可以继续补：

1. 死信 Topic

当前已经有：

- 手动 ack
- 有限次重试

但重试超限后还没有单独落到死信 Topic。

2. 失败任务表

当前失败日志已经有了，但还没有把失败消息统一落库做运营化管理。

3. 库存与购买标记回滚补偿

当前已经具备最小恢复能力：

- Kafka 重试
- Redis Stream pending-list 恢复

但如果最终确认订单无法落库，Redis 的预扣库存和购买标记回滚还可以继续做得更完整。

4. Redis 与 MySQL 对账任务

后续还可以增加库存对账任务，进一步补最终一致性治理能力。

11.5 如果面试官问“你现在这套方案做到哪一步了”

你可以直接用下面这段回答：

我现在已经把秒杀链路从 Redis Lua + 本地异步，升级成了 Redis Lua + Redis Stream + Kafka + MySQL 的异步下单架构。入口仍然由 Redis Lua 做库存判断和一人一单原子校验，校验通过后写入 Redis Stream。后面我新增了 Relay，把 Stream 消息转发到 Kafka，并且 Producer 已经改成同步等待 broker ack，只有 Kafka 真正确认成功后，Relay 才会 ACK Redis Stream。Consumer 这边我已经改成手动 ack，业务成功后才提交 offset，并接入了有限次重试。数据库层我也补了 unique(user_id, voucher_id) 唯一索引做最终幂等兜底。日志层面已经补了 traceId，可以串起 Relay、Producer、Consumer 和订单创建日志。实际验证上，我已经验证过 traceId 全链路打通、手动 ack 生效、Consumer 失败后会按 offset 重试、订单可以正常落库，以及

kafka 不可用时接口成功语义是“已受理”而不是“已完成”。目前还可以继续补死信 Topic、失败任务表和更完整的补偿任务。

11.6 一句话总结当前状态

当前这版 Kafka 秒杀链路，已经从“能跑”升级到了：

- 主链路打通
- 消息确认时机更合理
- 幂等更完整
- 日志更可追踪
- 具备最小恢复能力

也就是说，现在已经不是一个“只会发 MQ”的 demo 了，而是一条具备一定工程可靠性的秒杀异步链路。

8.1 为什么要在秒杀项目里引入 Kafka

答：

因为秒杀流量是突发高并发，数据库不适合直接扛住所有写请求。我保留 Redis + Lua 做前置原子校验，再把真正的订单创建异步化，通过 Kafka 做削峰、解耦和更可靠的消息消费恢复，这样秒杀接口可以更快返回，后端也更稳。

8.2 为什么不是直接用 Kafka，而是 Redis + Kafka 一起用

答：

Kafka 更擅长异步传输和削峰，不擅长做库存判断和一人一单这种原子校验。秒杀最核心的问题必须在入口阶段快速、原子地解决，所以我保留 Redis Lua 做前置校验，再用 Kafka 承接后续异步订单处理。

8.3 Redis Stream 和 Kafka 在这里分别做什么

答：

Redis Stream 更像本地事务消息桥，保证 Lua 校验通过后的请求先落到 Redis 里；Kafka 负责后续更标准的消息队列能力，比如异步削峰、消费扩展、消息积压和重启恢复。

8.4 为什么要加 Relay，而不是 Lua 直接发 Kafka

答：

Lua 运行在 Redis 内部，不能直接可靠地调用 Kafka。我的做法是先在 Lua 里原子地写入 Redis Stream，再由 Java 侧 Relay 组件读取 Stream 转发 Kafka，只有 Kafka 发送成功后才 ACK Stream，这样能把秒杀入口和 Kafka 解耦，也降低中间断点带来的丢消息风险。

8.5 Kafka message key 为什么选 userId

答：

我用 userId 作为 key，主要是为了让同一用户相关消息尽量进入同一分区，减少同一用户请求乱序的影响，也便于从业务维度解释分区设计。

8.6 Kafka 能保证不丢消息吗

答：

严格说不能只靠一句“kafka 不丢消息”来回答。我的项目里做了几层保障：Producer 开启 acks=all、重试和幂等；Redis Stream 到 Kafka 之间通过 Relay 做“发送成功再 ACK”；Consumer 侧通过幂等和数据库约束兜底。真正工程上追求的是尽量不丢、可恢复、可补偿。

8.7 Kafka 会不会重复消费

答：

会，所以 `Consumer` 不能假设消息只来一次。我的处理思路是业务幂等：先做重复下单校验，再通过数据库唯一索引和条件扣库存做最终兜底。

8.8 你怎么保证一人一单

答：

我做了三层控制。第一层，`Redis Lua` 在入口阶段做一人一单校验；第二层，`Consumer` 处理时按用户维度加分布式锁；第三层，数据库层建议加 `user_id` 和 `voucher_id` 的唯一索引作为最终兜底。

8.9 你怎么防止超卖

答：

入口层在 `Redis Lua` 里先校验库存并预扣减，`Consumer` 真正落库时又通过 `stock > 0` 条件更新扣减库存，所以是 `Redis` 预扣减加数据库最终扣减双重控制。

8.10 如果 Kafka 消费失败怎么办

答：

短期可以依靠 `Kafka` 重试和 `Consumer` 再消费；更完整的方案是有限次重试后进入死信 `Topic`，再配合补偿任务人工或自动处理失败订单。

8.11 如果 Redis 已经预扣库存，但订单没有落库成功怎么办

答：

这是最终一致性问题。标准做法是通过死信和补偿机制回滚 Redis 库存和用户购买标记，或者重新投递订单消息，并通过定时对账保证 Redis 和 MySQL 最终一致。

8.12 这套方案是强一致吗

答：

不是强一致，是最终一致。秒杀入口为了吞吐和响应速度，先在 Redis 层做资格校验和预扣减，再异步落库，所以我接受短时间内的状态不一致，但会通过幂等、重试、死信和补偿把最终结果修正回来。

8.13 为什么 Consumer 还要再做业务校验，不是 Lua 校验过了吗

答：

因为消息系统天然要考虑重复消费、服务重启和异常重试。Lua 只负责入口阶段挡住大部分无效请求，但最终订单创建仍然必须在 Consumer 侧做幂等和库存安全控制。

8.14 Kafka 比你原来线程池异步方案好在哪里

答：

线程池异步只能解决当前实例内的削峰，服务重启后任务容易丢，也不方便横向扩展。Kafka 方案更适合高并发链路，因为它有更成熟的消息持久化、消费恢复、扩展性和监控能力。

8.15 这套 Kafka 方案还有什么风险

答：

我认为主要风险是三类：Redis Stream 到 Kafka 的桥接断点、Consumer 重复消费、Redis 预扣和 DB 最终结果不一致。所以我会把重点放在 Relay 成功后再 ACK、Consumer 幂等、唯一索引、死信和补偿机制上。

9. 一段适合面试直接说的表述

我在这个项目里没有简单地把秒杀改成“Kafka 下单”，而是设计成 Redis + Kafka + MySQL 分层协作。秒杀请求先经过 Redis Lua 做库存校验和一人一单校验，校验通过后原子写入 Redis Stream。然后由一个 Relay 组件消费 Redis Stream，把消息转发到 Kafka，并且只有 Kafka 发送成功后才 ACK Stream。Kafka Consumer 再异步消费消息，执行真正的订单创建、数据库库存扣减和订单落库。这样做的好处是：Redis 负责高并发原子校验，Kafka 负责异步削峰和消费恢复，MySQL 负责最终业务事实。我同时考虑了重复消费、超卖、幂等和最终一致性问题，所以这套方案比单纯线程池异步更有工程可讲性。

10. 一段适合被追问时的补充回答

如果继续增强这套方案，我会补三件事。第一，订单表增加 `unique(user_id, voucher_id)` 唯一索引，让数据库成为最终幂等兜底。第二，Consumer 改为手动提交 `offset`，并增加有限次重试和死信 Topic。第三，增加 Redis 和 DB 的库存对账与补偿任务，解决预扣库存成功但最终订单失败的问题。这样整条秒杀链路会更完整。

11. 进展追加记录（2026-04-20）

本节为 append-only 追加，不覆盖原有面试稿，仅补充当前已落地能力，面试可直接引用。

11.1 当前已落地的失败治理闭环

- 主链路失败：有限次重试 (`SeekToCurrentErrorHandler + FixedBackOff`)
- 重试超限：进入死信 Topic `seckill-order-dlt`
- 死信落库：写入 `tb_voucher_order_fail_task`
- 自动补偿：定时重试 `RETRY_PENDING` 任务并回投主 Topic
- 最终状态：
 - 成功 -> `DONE`
 - 超阈值仍失败 -> `MANUAL_HANDLE_REQUIRED`

11.2 当前人工处理入口

- 查询任务： `GET /voucher-order-fail-task/page`
- 手动重试： `POST /voucher-order-fail-task/{id}/retry`
- 人工忽略： `POST /voucher-order-fail-task/{id}/ignore`

11.3 面试一句话表达（可直接背）

我现在把失败治理做成了闭环：主消费失败先有限重试，超限进 `DLT`，死信落失败任务表，再由定时任务自动重试，最终失败才转人工处理。这样既不阻塞主链路，又保证失败消息可追踪、可恢复、可治理。

Kafka 秒杀模拟面试 20 问 20 答

使用方式

- 第一轮：先只看“问题”，自己口述答案。
- 第二轮：对照“标准答法”，补齐关键词。
- 第三轮：每题控制在 30-60 秒，练面试节奏。

1) 你这个项目为什么要引入 Kafka?

标准答法:

核心原因是削峰和可靠异步。秒杀流量是突发的，DB 不适合直接承接峰值写入。

我把链路升级为 `Redis Lua + Kafka + MySQL`：Redis 做入口原子校验，Kafka 做异步削峰和恢复消费，MySQL 做最终落库。这样吞吐更稳、故障恢复能力更强。

2) 你是把 Redis Stream 替换成 Kafka 吗?

标准答法:

不是替换，而是分层协作。入口仍由 Lua 写 `stream.orders`，然后通过 Relay 转 Kafka。

这样保留了 Redis 在入口原子校验上的优势，同时利用 Kafka 做更成熟的消费恢复与重试管理。

3) 完整链路说一下?

标准答法:

请求 -> Redis Lua 校验并预扣库存 -> `XADD stream.orders` -> Relay 转发 Kafka -> Consumer 消费落库 -> 成功 ack / 失败重试 / 超限死信。

最终由消费侧幂等和数据库约束保证正确性。

4) 为什么不直接请求进来就发 Kafka?

标准答法：

因为秒杀最关键的是入口阶段快速且原子的资格校验（库存和一人一单），这类判断更适合 Redis + Lua。

Kafka 适合异步传输，不适合做高并发入口资格判定。

直接发Kafka的问题

```
用户请求 → Kafka → Consumer → DB
```

问题1：Lua脚本的原子性没了

秒杀的核心逻辑在Lua里：

```
-- 原子操作：校验库存 + 预扣 + 记录用户
if (库存充足 and 未下单) then
    预扣库存 + 记录用户 + 发消息
end
```

如果不用Lua：

- Java代码需要多次Redis调用
- 并发下无法保证原子性 → **超卖**

问题2：Kafka不可用时，接口直接失败

```
用户请求 → Kafka发送 → 失败 → 返回"系统繁忙"
```

- 瞬间高并发，Kafka可能扛不住
- 接口失败率飙升
- 用户体验差

问题3：没有入口缓冲

10000个请求同时到

- |
- ├ 9000个通过校验 → 发Kafka
- └ 1000个库存不足 → 直接返回

如果直接发Kafka:

10000个请求 → Kafka (瞬间压力)

用Redis Stream的好处

用户请求 → Lua (原子校验) → Redis Stream (写入成功就返回)

|



Kafka → Consumer → DB

优势	说明
原子校验	Lua保证库存和一人一单原子操作
快速响应	Stream写入成功就返回，不等Kafka
入口缓冲	10000请求瞬间到，Stream先扛住，Kafka慢慢消费
解耦	入口和落库独立，Kafka挂了不影响入口

一句话

Lua + Redis = 入口快、校验准 **Kafka = 异步稳、落库可靠**

两者结合，既保证秒杀的核心逻辑（原子性），又保证高可用（入口不依赖Kafka）。

5) 接口返回成功，代表订单一定成功了吗？

标准答法：

不代表。我的成功语义是“已受理”，即 Lua 校验成功并写入 Stream。

最终订单成功要看后续异步链路是否完成落库。这个语义设计能把入口可用性和后端落库解耦。

6) 那 Kafka 挂了是不是就丢单？

标准答法：

不会直接丢。关键在 Relay 的 ACK 时序：只有 Kafka 发送成功才 ACK Redis Stream。

Kafka 不可用时消息会留在 Stream pending，服务恢复后可继续转发，避免链路中间丢失。

7) 你为什么强调“Kafka 成功后再 ACK Stream”？

标准答法：

这是防断点丢消息的关键。

如果先 ACK 再发 Kafka，一旦发送失败，消息既不在 Stream，也不在 Kafka，形成永久丢失。

现在这个顺序至少保证失败可恢复。

8) Producer 做了哪些可靠性配置？

标准答法：

我设置了 `acks=all`、发送重试、幂等开启，并且发送端同步等待 broker 回执后才认为成功。

避免“只是交给客户端缓冲就误判发送成功”。

9) Consumer 为什么用手动 ack？

标准答法：

因为“拉到消息”不等于“业务成功”。

自动提交 offset 可能导致订单落库失败但位点已提交，造成消息丢失。

手动 ack 可以把位点提交和业务成功绑定，失败就不确认，让重试机制接管。

10) Consumer 失败后会怎样？

标准答法：

抛异常后进入错误处理器，先做有限次重试（固定退避）。

重试耗尽后写入死信 Topic，再由死信消费者入失败任务表，后续重试或人工补偿。

11) Kafka 会重复消费，怎么保证不重复下单？

标准答法：

我做了三层幂等：

1. Redis Lua 入口判重；
2. Consumer 业务判重；

3. DB 唯一索引 `user_id + voucher_id` 兜底。

所以即使重复投递，也不会生成重复订单。

12) 怎么防超卖？

标准答法：

Redis 侧先预扣库存拦截高并发，DB 落库时更新语句带 `stock > 0` 条件二次保护。

双层控制下，最终以 DB 成功结果为准，避免超卖。

13) 既然 Redis 预扣了，为什么还要 DB 再扣一次？

标准答法：

Redis 是高并发优化层，不是最终业务事实层。

MySQL 才是最终账本，必须在 DB 层再做库存扣减和订单落库，才能保证业务闭环与可审计。

14) 如果 Redis 预扣成功，但最终落库失败怎么办？

标准答法：

会进入补偿链路：失败任务重试、必要时回滚 Redis 库存与购买标记，并做对账修复。

我接受短暂不一致，但必须保证可追踪、可重试、可修复。

15) Topic 分区和 key 你怎么设计？

标准答法：

Topic 用 `seckill-order`，分区先按目标吞吐和消费者并发给初值（比如 3/6），后续按 lag 调优。

消息 key 用 `userId`，让同用户请求尽量落同分区，减少局部乱序影响。

16) 你如何观测这条链路健康？

标准答法：

我重点看四类指标：

1. Kafka lag/堆积；
2. Relay 转发成功率和 pending 积压；
3. 消费成功率、重试次数、死信量；
4. 业务指标：下单成功率、重复下单率、超卖率。

日志统一带 `traceId/orderId/userId/voucherId` 做全链路排查。

17) 为什么还要死信，不无限重试吗？

标准答法：

无限重试会拖垮消费能力，也会卡住后续正常消息。

死信把“短暂故障”和“脏数据/逻辑异常”分开处理，保证主消费通道持续可用。

18) 这套改造的最大代价是什么？

标准答法：

代价是系统复杂度上升：组件更多、排障难度更高。

我通过标准化日志、失败任务表、死信处理、监控告警来控制复杂度。

用复杂度换取高峰稳定性和恢复能力，这是值得的 trade-off。

19) 你如何证明这套方案确实更好？

标准答法：

做三组压测对比：同步下单、Redis Stream 异步、Kafka 异步。

对比 QPS、TP99、成功率、超卖率、重复率、lag、DB CPU/慢 SQL。

Kafka 方案通常在峰值稳定性和故障恢复上优势明显。

20) 如果让你继续优化，下一步做什么？

标准答法：

我会优先做三件事：

1. 完善自动补偿（失败任务分级重试 + 回滚策略）；
2. 建立库存/订单对账任务；
3. 增加告警和压测回归基线，形成工程闭环。

目标是把“可运行”升级到“可长期稳定运营”。

结尾 15 秒（可背诵）

这次改造的重点不是“上了 Kafka”，而是把秒杀链路做成了分层协作：

Redis 保证入口原子正确，Kafka 保证异步削峰和可恢复，MySQL 保证最终业务事实。

我重点解决了 ACK 时序、重复消费、超卖控制和失败补偿，所以这套方案既能扛流量，也能扛故障。

秒杀并发与 Kafka 面试说明 (项目扫描版)

1. 30 秒项目概述（可直接背）

这个项目是一个 Spring Boot + MyBatis-Plus + Redis + Kafka 的本地生活服务后端。

秒杀链路上，我把请求处理拆成三层：入口用 Redis Lua 做库存和一人一单的原子校验，消息通过 Redis Stream + Kafka 异步削峰，最终由 Consumer 落 MySQL 并做幂等兜底。

同时我把失败治理做成闭环：主消费有限次重试，超限进 DLT，死信落失败任务表并回滚 Redis 预扣，支持人工重试和人工忽略，系统还有库存对账和 Kafka lag 巡检。

2. 项目全局技术栈（面试开场 10 秒）

- Java 8, Spring Boot 2.3.12, MyBatis-Plus 3.4.3
- Redis (String/Set/Stream/Geo/Bitmap) 、Redisson、Caffeine 本地缓存
- Kafka (Spring Kafka)
- MySQL

核心目录分层：

- `controller`：HTTP 接口层
- `service/service.impl`：业务编排
- `mapper` + `entity`：数据访问
- `config`：Kafka/Redis/调度/监控配置

- `mq`: Stream-Kafka 中继、生产消费、死信处理
- `utils`: 缓存、ID 生成、拦截器、常量

3. 秒杀高并发链路（重点）

3.1 请求入口为什么快

入口是 `POST /voucher-order/seckill/{id}`，核心在 `VoucherOrderServiceImpl.seckillVoucher()`：

1. 先生成全局订单号（`RedisIdWorker`）
2. 执行 `seckill.lua` 原子脚本
3. 成功就快速返回 `orderId`，失败立即返回原因（库存不足/重复下单）

关键点：**不在入口做 DB 落单**，所以接口在高并发下更稳。

3.2 Lua 脚本做了什么

`seckill.lua` 一次性完成：

- 检查 `seckill:stock:{voucherId}` 库存
- 检查 `seckill:order:{voucherId}` 是否已下单（Set）
- 扣 Redis 库存、写用户下单标记
- `XADD stream.orders` 写入受理消息

这保证了“库存校验 + 一人一单 + 入队”在 Redis 侧是原子动作。

3.3 异步化为什么是 Stream + Kafka

链路是：

```
Lua -> Redis Stream(stream.orders) -> Relay ->
Kafka(seckill-order) -> Consumer -> MySQL
```

`RedisStreamToKafkaRelay` 的关键语义是：

- Kafka 发送成功才 `XACK` Stream
- 发送失败不 ack, 消息留在 pending-list
- 启动时会补扫 pending-list

这解决了“Redis 已受理但中继异常导致消息丢失”的风险。

3.4 最终落库如何防并发与重复

`VoucherOrderServiceImpl.createVoucherOrder()` 里做了 3 层保护:

- Redisson 用户锁: `lock:order:{userId}`
- 业务判重: 按 `user_id + voucher_id` 查重复
- DB 约束兜底: 唯一索引 `uk_user_voucher(user_id, voucher_id)`

库存扣减用条件更新: `stock = stock - 1 and stock > 0`, 防超卖。

4. Kafka 在项目中的职责与可靠性

4.1 Producer 侧

配置 (`application.yaml`) :

- `acks=all`
- `retries=3`
- `enable.idempotence=true`

实现 (`VoucherOrderProducerImpl`) :

- `send(...).get(5s)` 同步等待 broker 回执
- 成功日志输出 `traceId/orderId/partition/offset`

4.2 Consumer 侧

配置 (`KafkaConfig + application.yaml`) :

- 手动 ack: `manual_immediate`
- 自动提交关闭: `enable-auto-commit=false`
- 错误处理: `SeekToCurrentErrorHandler + FixedBackoff(1000,3)`
- 超限进入 DLT: `{topic}-dlt`

实现 (`VoucherOrderConsumer`) :

- 业务成功才 `acknowledgment.acknowledge()`
- 可恢复异常交给重试链路
- 不可恢复异常会先回滚 Redis 预扣, 再 ack, 避免毒消息无限重试

5. 死信与失败任务治理 (你现在已落地)

5.1 DLT 闭环

`VoucherOrderDeadLetterConsumer` 监听 `seckill-order-dlt`, 收到消息后:

1. 解析消息
2. 调用 `saveDeadLetterTaskAndRollback`
3. 落表 `tb_voucher_order_fail_task`
4. 同步执行 Redis 预扣回滚 (`seckill_rollback.lua`)

5.2 失败任务表用途

这张表把“日志事件”转成“可治理任务”, 支持:

- 状态追踪 (如 `ROLLBACK_DONE`、`MANUAL_HANDLE_REQUIRED`、`DONE`)

- 错误归因 (`error_msg`、`failure_type`)
- 人工介入与二次处理

5.3 已有处理入口

- 查询: `GET /voucher-order-fail-task/page`
- 手动重试: `POST /voucher-order-fail-task/{id}/retry`
- 人工忽略: `POST /voucher-order-fail-task/{id}/ignore`

另外有自动任务 `VoucherOrderFailTaskRetryScheduler` 定时扫描待重试任务。

6. 运维与可观测（面试加分点）

- Kafka 堆积巡检: `KafkaLagMonitorScheduler`
- 秒杀库存对账: `SeckillStockReconcileScheduler` (Redis vs DB)
- 启动预热库存: `SeckillStockPreheatRunner`
- 全链路 trace 字段: `traceId/orderId/userId/voucherId`

7. 高频面试追问（直接用）

Q1: 为什么不用“接口直接写 DB”，而要异步？

A: 秒杀场景峰值高，接口层最怕慢 SQL 和锁竞争。我把入口收敛成 Redis 原子校验 + 入流，先保证“快速受理”，再异步落库，这样吞吐和稳定性都更好。

Q2: Redis 已扣库存但 DB 失败怎么办？

A: 我有失败治理闭环。可恢复异常走 Kafka 重试；不可恢复异常或重试超限进入 DLT，落失败任务表并回滚 Redis 预扣，避免长期不一致。

Q3: Kafka 至少一次投递会重复消费，怎么防？

A: 业务层查重 + DB 唯一索引双保险。就算重复消息到达，最终也不会产生重复订单。

Q4: 为什么要手动 ack？

A: 自动提交会出现“消息已提交但业务未成功”风险。手动 ack 可以把位点提交和业务成功强绑定，失败就不确认，让重试机制接管。

Q5: DLT 的价值是什么？

A: 不是把坏消息丢掉，而是把问题消息隔离出来，避免阻塞主链路，同时给出可追踪、可修复、可人工处理的入口。

8. 你可以主动说的改进路线（下一步）

1. 异常分级更细：可恢复/不可恢复按异常类型路由
2. 重试策略升级：固定间隔改退避（带 `next_retry_time`）
3. 失败任务看板：状态统计、重试成功率、人工积压量
4. Redis-DB 对账后自动补偿策略完善

9. 关键代码定位（面试时可引用）

- 秒杀入口：

```
src/main/java/com/hmdp/controller/VoucherOrderContr
```

oller.java

- 秒杀服务:

src/main/java/com/hmdp/service/impl/VoucherOrderServiceImpl.java

- Lua 原子脚本: src/main/resources/seckill.lua

- Stream 中继:

src/main/java/com/hmdp/mq/RedisStreamToKafkaRelay.java

- Kafka 配置:

src/main/java/com/hmdp/config/KafkaConfig.java

- 主消费者:

src/main/java/com/hmdp/mq/VoucherOrderConsumer.java

- 死信消费者:

src/main/java/com/hmdp/mq/VoucherOrderDeadLetterConsumer.java

- 失败任务服务:

src/main/java/com/hmdp/service/impl/VoucherOrderFailTaskServiceImpl.java

- 失败任务接口:

src/main/java/com/hmdp/controller/VoucherOrderFailTaskController.java

- 失败任务表:

src/main/resources/db/voucher_order_fail_task.sql

Kafka 秒杀链路面试场景与追问作答手册

1. 一句话总述 (开场 10 秒)

我的秒杀系统不是把 Redis Stream 简单替换成 Kafka，而是升级成了 Redis 前置原子校验 + Kafka 异步削峰 + MySQL 最终落库。Redis 保证高并发入口正确性，Kafka 保证异步传输与恢复能力，MySQL 保证最终业务一致性。

2. 30 秒版本（面试常用主回答）

秒杀请求进来后，我先在 Redis Lua 里做库存判断和一人一单判断，校验通过后预扣 Redis 库存，并写入 `stream.orders`。然后通过一个 Relay 组件把 Redis Stream 消息转发到 Kafka Topic。这里我做了关键约束：**只有 Kafka 发送成功才 ACK Redis Stream**，避免中间链路丢消息。

Kafka Consumer 收到消息后执行业务下单，成功才手动提交 offset；失败走重试，重试耗尽进死信，再由失败任务链路做补偿。

最终通过消费侧判重 + 数据库唯一索引兜底，保证幂等和一致性。

3. 3 分钟版本（结构化展开）

3.1 为什么引入 Kafka

- 高并发秒杀下，数据库不适合直接承接突发写流量，需要异步削峰。
- 仅靠同步链路会把接口 RT 和后端落库强绑定，系统脆弱。
- Kafka 在消息堆积、重试恢复、消费位点管理方面更适合做异步主干。

3.2 为什么不直接用 Kafka 取代 Redis

- 秒杀最核心是“库存判断 + 一人一单”，要求入口阶段原子完成。
- Redis + Lua 适合高并发原子校验，Kafka 不适合作资格校验。

- 所以职责分工是：Redis 负责“快且准”的入口校验，Kafka 负责“稳且可恢复”的异步传输。

3.3 具体链路

请求 -> Lua 校验/预扣 -> XADD stream.orders -> Relay 转
Kafka -> Consumer 落库 -> ACK/重试/死信

3.4 可靠性关键点

- Producer 侧：acks=all、重试、幂等开启。
- Relay：Kafka 成功后才 ACK Stream，且支持 pending-list 恢复。
- Consumer：手动 ack，业务成功才提交 offset。
- ErrorHandler：有限次重试，超限进入 DLT。

3.5 幂等与一致性

- Redis 层：Lua 原子判断避免入口重复下单。
- 消费层：业务判重，不能假设“只消费一次”。
- DB 层：user_id + voucher_id 唯一索引兜底。
- 最终一致性：失败消息通过重试/死信/失败任务补偿闭环。

4. 面试高频追问与标准作答

4.1 追问：接口为什么能快速返回成功？Kafka 挂了怎么办？

答法：

我定义的接口成功语义是“请求已受理”，即 Lua 校验成功并写入 Redis Stream，不代表订单已经最终落库。

如果 Kafka 不可用，Relay 不会误 ACK Stream，消息会留在

pending 或待恢复链路中，后续恢复后继续转发。
这样可以把接口可用性和 Kafka 瞬时可用性解耦。

加分点：

主动补一句“我在日志和监控上会重点看 relay-forward-fail、pending 积压、consumer lag”。

4.2 追问：为什么要“Kafka 成功后再 ACK Redis Stream”？

答法：

这是为了避免“Redis 侧消息先确认，但 Kafka 实际没收到”的链路断点。

如果先 ACK 再发 Kafka，一旦发失败就可能永久丢单。

我现在的顺序是：先发 Kafka，拿到成功确认后再 ACK Stream，至少能保证失败消息留在 Redis 可恢复。

4.3 追问：手动 ack 和自动提交 offset 有什么区别？

答法：

自动提交只表示“消息被拉取过”，不代表“业务处理成功”。

秒杀场景里如果自动提交了 offset，但订单落库失败，就会出现消息已确认、订单没落库的丢单风险。

所以我使用手动 ack，把 offset 提交时机后移到业务成功之后。

4.4 追问：Kafka 会重复投递，怎么避免重复下单？

答法：

我按“三层幂等”做的：

- Redis Lua：入口一人一单校验；
 - Consumer 业务：消费侧判重；
 - MySQL：唯一索引最终兜底。
- 就算消息重复到达，也不会生成重复订单。
-

4.5 追问：如何避免超卖？

答法：

- 入口 Redis Lua 先预扣库存，快速拦截大部分无效请求；
 - 落库时 SQL 带 `stock > 0` 条件更新，防止并发穿透；
 - Redis 与 DB 双层控制，最终以 DB 结果为准。
-

4.6 追问：Consumer 失败后具体怎么走？

答法：

业务异常时不手动 ack，抛出异常交给 Kafka Listener 错误处理器。

我配置了固定退避重试，重试耗尽后转入死信 Topic。

死信消息会记录到失败任务表，由定时重试或人工干预补偿，确保最终可追可修。

4.7 追问：为什么链路里既有 Redis Stream 又有 Kafka，不会重复吗？

答法：

Redis Stream 在我的方案里是“入口接收缓冲层”，Kafka 是“异步主干总线”。

二者定位不同：前者贴近 Lua 原子写入、便于入口快速落消息；后者更擅长消费组扩展、重平衡、长链路可靠投递。

Relay 把两者连接起来并做 ACK 时序控制。

4.8 追问：你这套方案最大的 trade-off 是什么？

答法：

核心 trade-off 是系统复杂度提升：链路更长、组件更多、排障要求更高。

我通过标准化日志字段（orderId/userId/voucherId/traceld）、失败任务表、死信和监控告警来控制复杂度。

换来的是高峰期吞吐与稳定性显著提升，以及更好的故障恢复能力。

5. 面试官可能继续深挖的“进阶追问”

5.1 追问：Kafka 分区数怎么定？

答法：

先按目标吞吐和消费者并发估算一个初始值（例如 3 或 6），后续根据 lag 和 CPU 调整。

消息 key 用 `userId`，让同一用户请求尽量落同分区，降低同用户乱序影响。

我不强依赖全局顺序，依赖幂等和约束保证正确性。

5.2 追问：如果 Redis 预扣成功但最终落库失败怎么办？

答法：

会进入补偿链路：失败任务重试、必要时回滚 Redis 库存和购买标记，并做对账修复。

原则是承认分布式场景会有短暂不一致，但必须提供可追踪、可重放、可修复机制。

5.3 追问：如何证明方案有效？

答法：

做三组压测对比：同步直写 DB、Redis Stream 异步、Kafka 异步。

看 QPS、TP99、成功率、超卖率、重复下单率、consumer lag、DB CPU/慢 SQL。

在我的实践中，Kafka 链路在峰值稳定性和恢复能力上明显更好。

5.4 追问：你怎么观测 Kafka 堆积和消费是否健康？

答法：

我会盯 **consumer lag**：用 Kafka 管理接口读出每个分区 end offset 和消费组已提交 offset，**lag = end - committed**，再按分区汇总。

工程里用定时任务拉一次 lag，超过阈值打 **WARN** 日志，先把「堆积恶化」暴露出来，后续可以再接到 Grafana。

这和只看「Consumer 有没有在跑」不一样，lag 直接对应「欠了多少条消息没消化」。

5.5 追问：Redis 和 MySQL 库存会不会不一致，你怎么发现？

答法：

秒杀里 Redis 是预扣、MySQL 是最终扣减，中间又有异步消息，不能假设永远毫秒级一致。

我做的是**定时对账**：按券把 `seckill:stock:{voucherId}` 和 `tb_seckill_voucher.stock` 比对，不一致就 **WARN**，必要时再结合失败任务/死信去修。

更严格的「一人一单/订单事实对账」可以作为下一步演进。

6. 容易扣分的说法（避坑）

- 不要说“我把 Redis Stream 替换成 Kafka”。
 - 不要把“接口返回成功”说成“订单已落库成功”。
 - 不要说“Kafka 保证绝对不重复消费”。
 - 不要只讲组件名，不讲 ACK 时机、重试、死信、幂等。
 - 不要只讲吞吐，不讲一致性和补偿。
-

7. 可直接背诵的收尾话术（20 秒）

这次改造的重点不是“用了 Kafka”，而是把秒杀链路拆成了职责清晰的三层：

Redis 做前置原子校验，Kafka 做异步削峰和可恢复消费，MySQL 做最终业务事实。

我重点解决了 ACK 时序、重复消费、超卖控制和失败补偿，所以方案既能扛高并发，也能在异常下保证最终一致性。

8. 面试前 1 分钟速记卡

- 关键词：Lua 原子校验、Relay、Kafka 成功后 ACK Stream、手动 ack、重试+DLT、三层幂等、最终一致性补偿、lag 巡检、库存对账
- 主链路：Lua -> Stream -> Relay -> Kafka -> Consumer -> MySQL
- 成功语义：接口成功 = 已受理，不等于已完成
- 风险控制：失败可重试、可追踪、可补偿

?????Kafka秒杀链路面试题.md

Kafka秒杀链路面试题.md

恢复说明：原始 Markdown 已从工作区删除，当前文件为重建入口页，不是原始逐字版本。

建议优先阅读 MASTER_INTERVIEW.md，再按主题阅读现存资料。

建议阅读顺序

1. [MASTER INTERVIEW.md](#)
 2. 相关主题现存文档
 3. 如需继续细化，可在此文件基础上继续补充整理
-

?????S1-shop查询基线压测报告.md

S1-shop查询基线压测报告.md

恢复说明：原始 Markdown 已从工作区删除，当前文件为重建入口页，不是原始逐字版本。

建议优先阅读 MASTER_INTERVIEW.md，再按主题阅读现存资料。

建议阅读顺序

1. [MASTER INTERVIEW.md](#)
 2. 相关主题现存文档
 3. 如需继续细化，可在此文件基础上继续补充整理
-

?????二级缓存改造后验证报告.md

二级缓存改造后验证报告.md

恢复说明：原始 Markdown 已从工作区删除，当前文件为重建入口页，不是原始逐字版本。

建议优先阅读 MASTER_INTERVIEW.md，再按主题阅读现存资料。

建议阅读顺序

1. [MASTER INTERVIEW.md](#)
 2. 相关主题现存文档
 3. 如需继续细化，可在此文件基础上继续补充整理
-

??????二级缓存改造方案.md

二级缓存改造方案.md

恢复说明：原始 Markdown 已从工作区删除，当前文件为重建入口页，不是原始逐字版本。

建议优先阅读 MASTER_INTERVIEW.md，再按主题阅读现存资料。

建议阅读顺序

1. [MASTER INTERVIEW.md](#)
 2. 相关主题现存文档
 3. 如需继续细化，可在此文件基础上继续补充整理
-

??????二级缓存改造清单.md

二级缓存改造清单.md

恢复说明：原始 Markdown 已从工作区删除，当前文件为重建入口页，不是原始逐字版本。

建议优先阅读 MASTER_INTERVIEW.md，再按主题阅读现存资料。

建议阅读顺序

1. [MASTER INTERVIEW.md](#)
 2. 相关主题现存文档
 3. 如需继续细化，可在此文件基础上继续补充整理
-

?????压测对比测试计划.md

压测对比测试计划.md

恢复说明：原始 Markdown 已从工作区删除，当前文件为重建入口页，不是原始逐字版本。

建议优先阅读 MASTER_INTERVIEW.md，再按主题阅读现存资料。

建议阅读顺序

1. [MASTER INTERVIEW.md](#)
 2. 相关主题现存文档
 3. 如需继续细化，可在此文件基础上继续补充整理
-

?????本地缓存架构图.md

本地缓存架构图.md

恢复说明：原始 Markdown 已从工作区删除，当前文件为重建入口页，不是原始逐字版本。

建议优先阅读 MASTER_INTERVIEW.md，再按主题阅读现存资料。

建议阅读顺序

1. [MASTER INTERVIEW.md](#)
 2. 相关主题现存文档
 3. 如需继续细化，可在此文件基础上继续补充整理
-

?????秒杀并发与Kafka面试话术-1_3_8分钟版.md

秒杀并发与Kafka面试话术-1_3_8分钟版.md

恢复说明：原始 Markdown 已从工作区删除，当前文件为重建入口页，不是原始逐字版本。

建议优先阅读 MASTER_INTERVIEW.md，再按主题阅读现存资料。

建议阅读顺序

1. [MASTER INTERVIEW.md](#)
 2. 相关主题现存文档
 3. 如需继续细化，可在此文件基础上继续补充整理
-

?????秒杀链路测试结果记录.md

秒杀链路测试结果记录.md

恢复说明：原始 Markdown 已从工作区删除，当前文件为重建入口页，不是原始逐字版本。

建议优先阅读 MASTER_INTERVIEW.md，再按主题阅读现存资料。

建议阅读顺序

1. [MASTER INTERVIEW.md](#)
 2. 相关主题现存文档
 3. 如需继续细化，可在此文件基础上继续补充整理
-

?????自动化测试平台实现方案-面试版.md

HM-DianPing 自动化测试平台实现方案（面试版）

1. 项目背景与目标

本项目是典型的高并发秒杀业务，技术栈包含 Spring Boot、Redis、Kafka、Nginx，前端为静态页面 + Axios 调用后端接口。业务链路中最关键的是：

- 登录鉴权（验证码 + token）
- 秒杀预扣与异步下单
- 死信补偿与 Redis 回滚

自动化测试平台目标不是“把所有用例一次性写完”，而是先构建一套可持续演进的测试底座：

- 框架标准化：PyTest + Selenium + Page Object + fixture
- 稳定性：失败重试、环境预检、自动鉴权
- 可观测：Allure 统一报告（API + UI）
- 可执行：一键脚本跑全链路

2. 测试平台总体设计

2.1 技术选型

- 测试框架： `pytest`
- UI 自动化： `selenium`
- 页面模型： `Page Object`
- HTTP 测试： `requests`
- 报告： `allure-pytest`
- 重试： `pytest-rerunfailures`
- 鉴权辅助： `redis`（自动读取验证码）

2.2 分层结构（解耦思路）

三层解耦如下：

- 元素层（Page）
 - 只维护页面元素定位和原子操作
 - 目录： `tests/ui/pages/`
- 业务动作层（Page 方法）
 - 把“登录”“点击秒杀”封装成业务动作
 - 仍放在 Page 类，避免用例直接操作元素
- 用例层（Case）
 - 只关注业务断言，不关心 DOM 细节
 - 目录： `tests/ui/cases/`、 `tests/api/`

2.3 运行拓扑（与项目实际一致）

- 前端: `127.0.0.1:8080` (Nginx)
- 后端: `127.0.0.1:8081`、`127.0.0.1:8082`
- Redis: `127.0.0.1:6379`
- Nginx `/api` 反向代理后端双节点

这也是为什么测试脚本加入了端口检查和 `-Require8082` 选项：避免 upstream 半可用造成随机失败。

3. 代码级实现细节

3.1 配置与依赖

- `requirements-test.txt`
 - 固化测试依赖，降低环境差异
- `pytest.ini`
 - 统一 marker (`api`、`ui`)
 - 默认重试 (`--reruns 1`)

3.2 fixture 设计（核心）

文件: `tests/conftest.py`

关键 fixture:

- `api_session`: 复用 HTTP 会话
- `driver`: Selenium 浏览器实例
- `auth_token`: 自动获取 token (可被手工 token 覆盖)
- `token_factory`: 可生成多个独立用户 token, 解决“一人一单”互相污染
- `seckill_voucher`: 自动扫描可用秒杀券

- `active_seckill_voucher`: 按 begin/end 时间过滤活动有效券

关键设计点:

- **自动鉴权链路**
 - 调 `/user/code` 发码
 - 从 Redis 读取 `login:code:{phone}`
 - 调 `/user/login` 获取 token
- **驱动稳定性回退**
 - 先走 `webdriver-manager`
 - 失败则尝试本地 `chromedriver`
 - 再失败走 Selenium 默认发现
 - 仍失败则 skip (避免整套因外网驱动下载失败崩掉)
- **失败证据采集**
 - UI 失败自动附加截图与 page source 到 Allure

3.3 Page Object 实现

文件:

- `tests/ui/pages/base_page.py`
- `tests/ui/pages/login_page.py`
- `tests/ui/pages/shop_detail_page.py`

关键实践:

- 基类统一封装 `open/click/type/text_of`
- 登录页处理 ElementUI 单选按钮在 headless 下“可见但不可点击”的问题, 增加 JS 兜底
- 秒杀页增加“按钮就绪等待”, 按钮不可用时输出可读性 skip 原因

3.4 API 用例实现

文件: `tests/api/test_hmdp_api_smoke.py`

覆盖重点:

- `/shop-type/list` 基础可用性与数据非空
- `/user/me` 未登录 401
- 自动登录后 `/user/me` 成功
- 自动发现秒杀券并验证券列表可见
- 同用户重复秒杀被拦截
- 登出后 token 失效
- 非法 token 秒杀被拒绝

3.5 UI 用例实现

文件:

- `tests/ui/cases/test_login_page_smoke.py`
- `tests/ui/cases/test_seckill_flow_ui.py`

覆盖重点:

- 登录页基础可访问性
- 未登录点击秒杀跳登录页
- 已登录点击秒杀出现业务反馈 (成功/重复/库存不足)

说明:

- `test_login_action_template` 默认关闭 (需 `RUN_UI_LOGIN_E2E=1`) , 防止环境不稳定时影响主回归。

3.6 一键执行脚本

文件: `scripts/run-tests.ps1`

功能:

- 启动前端/后端前的端口预检 (8080/8081/6379)

- 可选强校验 8082 (`-Require8082`)
- 支持模式：
 - `all`
 - `api`
 - `ui`
 - `seckill`
- 自动生成 `reports/allure-results`
- 可选直接 `allure serve`

4. 当前“10条通过”的解释（面试可用）

最近稳定回归结果是：

- `10 passed, 1 skipped`

这不是“只会写 10 条”，而是刻意分阶段：

- 第一阶段先做 **高价值冒烟集（Smoke Baseline）**
- 目标是先把链路跑通、结果稳定、可持续执行
- 在此基础上再扩用例数量，比一开始堆很多脆弱用例更工程化

当前 10 条覆盖已经包含：

- 登录鉴权正反向
- token 生命周期（登录/登出）
- 秒杀核心约束（一人一单）
- UI 关键入口（未登录拦截、已登录下单反馈）
- 业务可用性（券查询、类目查询）

5. 为什么会有

`tb_voucher_order_fail_task`

这张表是本项目“异步失败补偿”的关键治理能力，不是异常本身：

- 当 Kafka 消费端多次重试后仍失败
- 消息进入 DLT（死信队列）
- 系统把失败记录落入 `tb_voucher_order_fail_task`
- 同时触发 Redis 回滚（库存 + 一人一单标记）

`status=ROLLBACK_DONE` + `failure_type=NON_RECOVERABLE` 的含义：

- 本条消息主链路最终失败
- 但补偿已经执行完成
- 数据一致性已尽量对齐

这在高并发系统中是“可观测、可追踪、可补偿”的正确设计，不是简单报错。

6. 扩展方案（从 10 条到 30+）

建议按优先级迭代，而不是一次铺太多：

P0（先补 8~10 条）

- 秒杀活动未开始/已结束/库存为 0 的边界断言
- 同券不同用户并发抢购结果分布校验
- DLT 落库字段完整性校验（`traceId/orderId/errorMsg`）
- 手动重试任务接口与状态流转校验

P1（再补 8~12 条）

- 店铺列表分页/排序（距离、人气、评分）
- 详情页券展示与库存变更可见性
- 登录页异常手机号、错误验证码场景
- token 过期刷新链路（若接入网关/拦截器续期策略）

P2 (持续建设)

- CI 定时回归 (白天增量 + 夜间全量)
- 压测后冒烟 (loadtest + regression)
- Allure 历史趋势看板

7. 面试表达模板 (可直接说)

可以按下面结构讲 2~3 分钟：

1. 我先做了测试框架底座，而不是直接堆用例。
2. 框架采用 PyTest + Selenium，按 Page Object + fixture 解耦元素、动作、断言三层。
3. 针对这个项目，我实现了自动鉴权：发码、读 Redis 验证码、自动登录拿 token。
4. 接口和 UI 共用鉴权能力，并统一输出 Allure 报告。
5. 我加了一键脚本做端口预检和全链路执行，避免 8082 未启动导致 upstream 随机失败。
6. 当前稳定通过 10 条高价值用例，覆盖登录、秒杀、一人一单、登出失效、未登录拦截等核心路径。
7. 同时保留了死信补偿场景观测，通过 `tb_voucher_order_fail_task` 追踪失败并验证 Redis 回滚。
8. 后续我会按 P0/P1/P2 扩到 30+ 条并接入 CI 趋势分析。

8. 常见追问与回答思路

Q1：为什么不是一开始就 50 条？

答：先做稳定基线。测试平台是工程，不是脚本堆叠。先把 10 条高价值链路跑稳，再规模化扩展，维护成本最低。

Q2：你怎么处理 UI 用例脆弱问题？

答：Page Object 隔离定位变化；关键动作加等待与兜底；环境不满足时 skip 并给出原因，不让假红影响回归判断。

Q3：为什么要接 Redis？

答：这个系统登录验证码存 Redis，接入 Redis 后可自动鉴权，避免人工介入，使回归可无人值守。

Q4：死信日志算失败吗？

答：看状态。ROLLBACK_DONE 说明主链路失败但补偿完成；是系统韧性机制的一部分。关键是监控频率与根因分类。

9. 执行命令（面试演示可用）

全量（推荐）

```
powershell -ExecutionPolicy Bypass -File  
scripts/run-tests.ps1 -Mode all -Require8082
```

秒杀专项

```
powershell -ExecutionPolicy Bypass -File  
scripts/run-tests.ps1 -Mode seckill -Require8082
```

UI + 开启真实登录动作

```
powershell -ExecutionPolicy Bypass -File  
scripts/run-tests.ps1 -Mode ui -EnableUiLoginE2E
```

查看报告

```
allure serve reports/allure-results
```

10. 结论

这套平台已经具备“可落地、可复现、可扩展、可面试展示”的四个特征：

- 可落地：和当前项目真实链路打通（Nginx + Redis + Kafka + 秒杀）
 - 可复现：一键脚本、端口预检、统一依赖
 - 可扩展：分层清晰，新增用例成本低
 - 可展示：Allure 报告 + 死信补偿可观测，能讲清工程价值
-

?????自动化测试平台面试讲稿-校招版.md

自动化测试平台面试讲稿（校招风格）

1. 60秒自我介绍版（开场）

在这个 `hm-dianping` 项目里，我做的是一个 API + UI 一体化自动化测试平台。

我用 `PyTest + Selenium` 搭了框架，用 `Page Object + fixture` 做了解耦，把页面元素、业务动作、用例断言分开。

为了让回归可无人值守，我实现了自动鉴权：调用发码接口、从 Redis 取验证码、自动登录拿 token。

最后用 Allure 统一输出报告，并做了一键脚本用于端口预检和全量执行。

目前稳定基线是 10 条高价值用例，后续按优先级扩展。

2. 3分钟主讲版（校招推荐）

这个项目的业务重点是秒杀链路，涉及 Redis 预扣、Kafka 异步消费、死信补偿。

我做测试平台时，先明确目标不是“写很多脚本”，而是“搭一个能长期跑、团队可复用的测试底座”。

我主要做了四件事：

1. 框架搭建

用 PyTest 做统一入口，UI 用 Selenium，按 Page Object 分层。

Page 层负责元素定位和动作，用例层只写业务断言，后续维护更轻。

2. 自动鉴权

这个项目登录是验证码模式，我做了自动化流程：

`/user/code` 发码 -> Redis 读 `login:code:{phone}` ->

`/user/login` 拿 token。

这样 API 和 UI 都能复用 token，不需要人工干预。

3. 稳定性治理

- 用例失败重试
- UI 失败自动截图和保存页面源码
- 一键脚本先做端口检查，避免服务没起来就盲跑
- 对不稳定环境（如 upstream 节点未起）给出可读的 skip 原因

4. 结果可视化

所有结果统一写入 `reports/allure-results`，可直接 `allure serve` 查看。

目前全量稳定结果是 `10 passed, 1 skipped`，覆盖登录、鉴权、秒杀核心约束和 UI 主路径。

我这次最大的收获是：

不仅写了测试脚本，更把测试做成了“可执行、可复现、可扩展”的工程能力。

3. 面试官常问（校招简洁回答）

Q1：为什么只做了10条？

我先做的是高价值稳定基线。

先把核心链路跑稳，再扩展数量，比一开始堆很多不稳定脚本更工程化。

Q2：你这套怎么体现业务理解？

我不只测页面是否可点，而是测秒杀关键约束：

未登录拦截、一人一单、登出后 token 失效、异常链路补偿可观测。

Q3：看到

`tb_voucher_order_fail_task` 是不是系统有问题？

不一定。

如果是 `ROLLBACK_DONE` 且 `redisRollback=OK`，说明主链路失败后补偿闭环完成。

要看是否高频出现，高频才需要专项治理。

Q4：你个人在这个项目里最有价值的点是什么？

我把“手工验证”变成了“自动化回归能力”：

自动鉴权 + 一键执行 + 报告输出 + 可扩展分层，这是团队可持续使用的。

4. 校招风格结尾（可直接背）

我在这个项目里不是单纯补了几条测试，而是搭了一个可以长期演进的自动化测试平台。

现在已经能稳定覆盖核心链路，后续我会继续按优先级扩到 30+ 条，并接入 CI 做持续质量保障。

?????自动化测试平台面试讲稿.md

自动化测试平台面试讲稿

1) 3分钟口述版（先讲这个）

我在 `hm-dianping` 这个项目里，搭建了一套 API + UI 一体化自动化测试平台，目标是把秒杀核心链路做成可回归、可观测、可扩展。

技术上我选了 `PyTest + Selenium`，并按 `Page Object + fixture` 做分层：

页面元素和业务动作放在 Page 层，用例层只写业务断言，避免 DOM 变动直接影响用例。

这个项目的难点是登录依赖验证码，我做了自动鉴权链路：

先调用 `/user/code` 发码，再从 Redis 读取 `login:code:{phone}`，然后调用 `/user/login` 自动拿 token。

这样 API 和 UI 用例都不需要人工输入验证码，回归可以无人值守执行。

我还把稳定性做了两块增强：

第一是失败重试（`pytest-rerunfailures`），第二是一键执行脚本会先做端口预检，避免 Nginx upstream 半可用导致随机失败。报告统一用 Allure，API 和 UI 结果合并到一个报告目录。

当前稳定基线是 `10 passed, 1 skipped`，覆盖了登录鉴权、未登录拦截、一人一单、登出失效、秒杀 UI 主流程等高价值路径。不是只会 10 条，而是先做稳定冒烟基线，再按优先级扩到 30+ 条，这样工程上更可持续。

2) 10分钟深入版（面试官追问时展开）

2.1 需求拆解

我把需求拆成四块：

- 测什么：核心业务链路，而不是纯页面点点点
- 怎么测：API + UI 协同，不做单腿走路
- 怎么稳：重试、环境预检、自动鉴权、失败附件
- 怎么用：一键脚本 + 报告输出，团队可复用

2.2 架构与目录

- `tests/api/`：接口用例
- `tests/ui/pages/`：页面对象
- `tests/ui/cases/`：UI 用例
- `tests/conftest.py`：全局 fixture
- `scripts/run-tests.ps1`：一键执行

核心思想是三层解耦：元素、动作、断言。

2.3 自动鉴权关键设计

这个项目登录是短信验证码模式，传统 UI 自动化依赖人工输入，不稳定且低效。

我改为：

1. `/user/code?phone=...` 发码

2. Redis 读取 `login:code:{phone}`
3. `/user/login` 获取 token
4. token 注入 API header 或 UI `sessionStorage`

这样把“人手操作”变成“系统动作”，大幅提升可回归性。

2.4 稳定性治理

我主要处理了三类不稳定源：

- **环境不一致**：脚本先查 8080/8081/6379，可选强校验 8082
- **浏览器驱动问题**：driver 初始化做多级回退（在线下载 -> 本地 -> 默认发现）
- **UI时序波动**：关键动作加显式等待和兜底 skip，避免假红

并且失败自动附带截图和 page source，方便直接定位。

2.5 业务覆盖策略

先做高价值冒烟：

- `/user/me` 正反向鉴权
- 秒杀券自动发现
- 同用户重复秒杀拦截
- 登出后 token 失效
- UI 未登录秒杀跳登录
- UI 已登录秒杀业务反馈

这套先跑稳，再按优先级扩容。

2.6 与死信补偿联动

我在讲测试平台时会提到系统韧性：

当 Kafka 消费失败进 DLT，`tb_voucher_order_fail_task` 会落失败任务并触发 Redis 回滚。

像 `ROLLBACK_DONE`、`redisRollback=OK` 表示补偿完成，是可观

测可治理，而不是单纯“报错”。

这能体现我不仅会写脚本，也能结合业务链路判断系统状态。

2.7 结果与价值

- 形成了“一键执行 + 报告输出”的团队化能力
- 回归从人工验证转为可重复执行
- 面对复杂链路（Redis/Kafka/秒杀）能提供可观测闭环
- 后续扩展成本低，新增用例只需补 Page 或断言

3) 高频追问与回答模板

Q1：为什么不用 JMeter/只做接口压测？

压测和回归不是替代关系。

JMeter解决容量问题，PyTest+Selenium解决功能正确性和回归效率。

我这套平台是“功能基线”，可和压测串联使用。

Q2：为什么现在只有 10 条？

这是有意做“稳定冒烟基线”。

先把核心链路跑稳，解决环境和框架稳定性，再规模化扩用例；否则一开始堆很多不稳定脚本，维护成本很高，团队很快弃用。

Q3：怎么保证不是假通过？

- API 用例覆盖正反向断言（401、重复下单、登出失效）
- UI 覆盖关键入口行为（跳转、业务提示）
- 自动鉴权走真实后端链路，不是mock token
- 失败有截图和页面源码，便于复盘

Q4: `tb_voucher_order_fail_task` 出现记录是不是系统有问题？

不一定。

关键看状态：`ROLLBACK_DONE` + `redisRollback=OK` 表示补偿闭环已完成。

如果持续增长才需要根因治理（消费端异常分类、重试策略、下游稳定性）。

Q5: 如何扩展到 30+？

- P0: 秒杀边界 + 失败任务重试链路
- P1: 店铺列表分页/排序 + 登录异常输入
- P2: CI 定时回归 + Allure 历史趋势

4) 你可以直接照读的结尾

我做这个平台的核心不是“写了几条脚本”，而是把测试能力工程化：

能自动鉴权、能统一报告、能一键执行、能结合死信补偿判断真实系统状态。

当前 10 条是稳定基线，接下来会按业务优先级扩到 30+，并接入 CI 做持续质量监控。

?????自动化测试平台面试速 记卡-校招.md

自动化测试平台面试速记卡 (校招)

30秒版（电梯自我介绍）

我在 `hm-dianping` 项目里做了一套 API+UI 一体化自动化测试平台。

技术上用 `PyTest + Selenium`，并用 `Page Object + fixture` 解耦页面元素、业务动作和用例断言。

我还做了自动鉴权：发验证码、从 Redis 读验证码、自动登录拿 token。

目前全量稳定结果是 `10 passed, 1 skipped`，并且支持一键执行和 Allure 报告。

60秒版（标准回答）

这个项目秒杀链路比较复杂，涉及 Redis、Kafka、死信补偿。

我搭平台时重点做了三件事：

1. **框架分层**：PyTest + Selenium，Page Object 把定位和断言解耦，后续维护成本低。
2. **自动鉴权**：调用 `/user/code` 发码，从 Redis 读 `login:code:{phone}`，再调 `/user/login` 拿 token，API 和 UI 共用。
3. **工程化落地**：加一键脚本做端口预检和全量执行，结果统一输出到 Allure。

产出上，已经覆盖登录鉴权、未登录拦截、一人一单、登出失效、UI 秒杀主流程等高价值场景，回归可重复执行。

3分钟版（建议主讲）

我在这个项目里做的不是“写几条测试脚本”，而是搭了一个能长期演进的测试底座。

背景是秒杀业务链路复杂，单靠手工回归效率低、漏测风险高。

我先做了技术方案：

- pytest 做统一执行入口
- Selenium 做 UI 自动化
- Page Object + fixture 做三层解耦
- Allure 统一报告，失败重试提升稳定性

然后我做了项目特定能力：

- 自动鉴权链路： `/user/code` -> Redis 取验证码 -> `/user/login` 拿 token
- token 同时给 API 和 UI 用例复用
- 自动发现可用秒杀券并校验活动时窗

为了让这套东西真正“能用”，我加了一键脚本：

- 先做 8080/8081/6379（可选 8082）端口预检
- 再按模式执行 API/UI/全量
- 自动产出 Allure 原始结果目录

最终稳定基线是 `10 passed, 1 skipped`，覆盖了核心链路：

- 鉴权正反向
- 秒杀一人一单
- token 生命周期（登录/登出）
- UI 未登录拦截与已登录下单反馈

后续扩展计划是先到 30+ 条，优先补边界和异常路径，再接 CI 做持续回归。

高频追问一句话

- **为什么只有10条？**

先做高价值稳定基线，再规模扩展，这是工程上更可持续的路线。

- **死信是不是系统挂了？**

不一定，像 `ROLLBACK_DONE + redisRollback=OK` 说明补偿闭环完成，重点看是否高频。

- 你的亮点是什么？

把手工验证变成可复用的工程化回归能力：自动鉴权 + 一键执行 + 报告统一。

?????限流设计方案.md

限流设计方案.md

恢复说明：原始 Markdown 已从工作区删除，当前文件为重建入口页，不是原始逐字版本。

建议优先阅读 MASTER_INTERVIEW.md，再按主题阅读现存资料。

建议阅读顺序

1. [MASTER INTERVIEW.md](#)
 2. 相关主题现存文档
 3. 如需继续细化，可在此文件基础上继续补充整理
-

?????项目专题面试题-校招版-参考回答.md

?????????-????-???????PDF?????

????????? PDF ??????????????????????????????

hm-dianping 项目专题面试 题库（校招 / 实习版，含参考 回答）

说明：这份文档保留了专题题库的结构，并在每道题后补充了适合校招 / 实习场景的参考回答。回答风格尽量贴近“这是我做过、能展开讲细节”的口径。

一、项目总览专题

1. 先用 1 分钟介绍一下你的 hm-dianping 项目，你主要负责了哪些部分？

参考回答：

我的 hm-dianping 是一个基于 Spring Boot + Redis + MySQL

- Kafka + Nginx 的本地生活服务项目，业务上参考了大众点评和小红书的部分场景，主要包括用户登录、商户查询、探店博客、关注 Feed、优惠券和秒杀下单等功能。我负责的是后端整体开发和改造，不只是接口实现，还包括缓存体系、秒杀高并发链路、分布式锁、失败补偿、限流和测试补充这些工程化能力。如果面试官继续展开，我一般会重点讲商户缓存和秒杀链路，因为这两块最能体现我在后端设计和细节实现上的工作。

2. 这个项目和普通 CRUD 项目相比，最大的技术亮点是什么？

参考回答：

我觉得最大的区别是这个项目不只是增删改查，而是加入了比较完整的高并发和缓存设计。比如商户查询做了 Caffeine + Redis 两级缓存，秒杀链路做成了 Redis Lua + Redis Stream + Kafka + MySQL 的异步方案，还处理了一人一单、幂等、库存回

滚和失败补偿。相比普通 CRUD，这个项目更能体现我对性能、并发和稳定性的考虑。

3. 你为什么选这个项目作为实习 / 校招阶段的代表项目？

参考回答：

因为它业务不空，能讲清真实场景；技术也不浅，能覆盖 Redis、消息队列、缓存、高并发这些面试高频点。对校招来说，这种项目比单纯管理系统更能体现后端能力。另一方面，这个项目我自己后续做了不少增强改造，所以很多实现细节我能讲得比较扎实。

4. 这个项目里有哪些核心业务模块？它们之间大致是怎么关联的？

参考回答：

核心业务模块有用户模块、商户模块、博客模块、关注模块、优惠券模块和秒杀订单模块。用户登录之后可以查商户、发博客、关注别人、看 Feed，也可以参与优惠券秒杀。技术上这些模块共享一套登录态、缓存、Redis 工具类和数据库访问层，秒杀模块还额外用到了 Lua、Redis Stream、Kafka 和补偿机制。

5. 如果面试官只让你讲一个最能体现你能力的模块，你会讲哪一个？为什么？

参考回答：

我会优先讲秒杀模块。因为它同时覆盖了 Redis、Lua、Kafka、MySQL、分布式锁、幂等、异常补偿这些后端高频点，技术密度

最

高，也最能体现我不是只会写接口。另外如果面试官更偏缓存，我也可以切到商户缓存这块，因为我做过两级缓存和失效同步的改造。

6. 这个项目是你从零搭起来的，还是在现有基础上做增强？你做了哪些改造？

参考回答：

项目基础业务框架是现有的，但后端很多关键能力我是按自己的理解

做增强和改造的。比如秒杀链路从简单异步方案逐步改成了

Redis Stream + Kafka 的链路；缓存从单 Redis 缓存改成了两级

缓存；另外还补了限流、失败补偿、配置脱敏和一些回归测试。可

以理解成我是在已有业务骨架上，把它往更工程化的方向做了升级。

7. 你在项目里改造过哪些原有方案？为什么觉得原方案不够好？

参考回答：

我改造过的重点有两块。第一块是缓存，原来更多是单 Redis 缓

存，查询热点数据时 Redis 压力会更集中，所以我补了本地缓存和

失效同步。第二块是秒杀链路，原来链路能跑，但在幂等、失败补

偿和异步削峰上不够完整，所以我把它做成了更清晰的消息流转和

异常处理方案。

8. 这个项目的技术栈为什么选了 Spring Boot + Redis + MySQL + Kafka ？

参考回答：

Spring Boot 适合快速搭后端服务，MySQL 负责业务数据持久化，Redis 主要解决缓存、会话、地理位置和高并发预处理的问题，Kafka 主要用来做秒杀订单的异步削峰和可靠消费。这套技术栈覆盖了校招后端最常见的核心能力，也比较适合讲清楚从业务到工程的设计过程。

9. 如果不看代码，只从业务角度讲，用户一次秒杀下单会经过哪些步骤？

参考回答：

用户点击秒杀之后，请求先到后端接口；后端会先通过 Redis 和

Lua 脚本快速判断库存够不够、用户是不是重复下单，并在 Redis

里先做预扣；如果通过，就把订单信息写进消息流，接口先快速返

回；后面由消息消费端异步落库，真正扣数据库库存并创建订单；

如果落库失败，再走补偿或回滚逻辑。这样设计是为了让接口快返

回，同时把高并发压力从数据库前移到 Redis 和消息队列。

10. 你自己觉得这个项目里最难讲清楚的一段链路是哪一段？为什么？

参考回答：

我觉得最难讲的是秒杀链路。因为它不是单点功能，而是 Redis 、

Lua 、 Stream 、 Kafka 、数据库、分布式锁和补偿逻辑串起来的一

整条链路。面试时如果不按顺序讲，很容易说乱，所以我一般会先

讲“为什么要这么设计”，再按“资格校验、异步削峰、数据库兜底、

失败补偿”这个顺序展开。

二、登录与用户模块专题

1. 你的登录功能是怎么做的？为什么选择验证码登录而不是账号密码登录？

参考回答：

登录是手机号验证码登录，流程是先发送验证码到 Redis，再由用户提交手机号和验证码进行校验，校验通过后生成 token，并把用户简要信息写到 Redis 中。选择验证码登录一方面是因为这种项目更贴近移动端业务场景，另一方面它能自然引出 Redis 的使用，比如验证码过期、登录态刷新、拦截器校验这些点。

2. 验证码为什么放 Redis，不直接放数据库？

参考回答：

验证码本质上是一个短期有效的临时数据，生命周期很短，而且读写频率高，不适合落数据库。放 Redis 可以直接设置 TTL，到期自动失效，读写也更快。相比数据库，这种场景 Redis 更轻量、更合适。

3. 你的验证码 key 是怎么设计的？过期时间为什么这么定？

参考回答：

验证码 key 设计成 `login:code:{phone}`，这样按手机号维度隔离，查的时候也很直接。过期时间我设置得比较短，一般是 2 分钟左右，因为验证码本身就是高时效数据，时间太长不安全，太

短又

可能影响用户体验，所以 2 分钟是比较常见的折中。

4. 用户登录成功后，为什么把用户信息放 Redis，而不是直接放 Session？

参考回答：

因为 Redis 更适合做分布式登录态共享，如果后面服务做多实例部署，不同节点都能读到同一份登录信息。传统 Session 更适合单体或需要粘性会话的场景，而 Redis 存 token 更容易扩展，也更常见。对我这个项目来说，用 Redis 登录态和整体技术方案更统一。

5. 你项目里的 Token 是怎么生成和校验的？

参考回答：

登录成功后，我会生成一个随机 token，一般可以理解成 UUID 风

格的值，然后把用户信息以 Hash 或类似结构存到

login:token:

{token} 对应的 Redis key 里。后续请求带上 token，拦截器就去

Redis 查这条登录态是否存在，存在就说明登录有效。这个方案本

质上是“token 做索引，Redis 存会话数据”。

6. 为什么要设计 login:token:{token} 这种 key？

参考回答：

这样设计主要是为了分层清晰和方便管理。前缀 login:token: 能

直接表示这是登录态相关数据，后面拼 token 能保证唯一性，也方

便后面做批量清理和排查。Redis 里 key 有明确业务前缀是一个很常见的习惯。

7. 登录拦截器和刷新 Token 的拦截器分别

做了什么？顺序为什么不能乱？

参考回答：

刷新 token 的拦截器主要负责从请求里拿 token，如果 Redis 里存

在这条登录态，就把用户信息刷新到当前线程上下文，并顺便续

期。登录拦截器则负责判断当前接口是否要求登录，如果要求登录

但上下文里没有用户，就拦截掉。顺序不能乱，因为必须先尝

试解

析和刷新 token，后面登录校验才能知道当前用户到底有没有登录。

8. 你为什么要用 ThreadLocal 保存用户

信息？它解决了什么问题？

参考回答：

因为用户信息一旦在拦截器阶段拿到了，后面的 controller、service 都可能要用。如果每一层都手动传 userId，代码会很啰

嗦，所以我用 ThreadLocal 把当前请求的用户信息放到线程上下

文里，后面直接取就行。它解决的是“请求内用户上下文透传”的问

题。

9. ThreadLocal 在请求结束后如果不清

理，会有什么问题？

参考回答：

最大的风险是线程复用导致脏数据串请求，尤其在线程池环境里更

明显。比如上一个请求的用户信息残留到下一个请求里，就可能带

来严重的权限问题。所以请求结束后一定要清理 ThreadLocal
,
这是使用它时必须注意的点。

10. 如果面试官问你这个登录方案和 JWT 的区别，你怎么回答？

参考回答：

我这个方案更偏 “Redis 会话型 token”，核心是 token 只是一个凭

证，真正的用户状态在 Redis 里。JWT 则是把用户信息和签名放在

token 自身里，服务端可以不查库也不查 Redis 就完成校验。

前者

的优点是可以主动失效、续期灵活，后者的优点是更无状态，

但要

做强制下线和续期会相对麻烦一些。

三、Redis 与缓存专题

1. 你项目里哪些地方用了 Redis ？

参考回答：

Redis 在项目里用得比较多，主要包括验证码、登录态、商户缓存、商户 GEO、博客点赞、关注集合、Feed 流、签到、秒杀库

存、一人一单集合，还有消息流转入口。也就是说它既承担了缓存

作用，也承担了一部分高并发场景下的业务状态处理。

2. 商户查询为什么适合用缓存？

参考回答：

因为商户详情和商户列表属于读多写少的数据，用户访问非常频

繁，但更新频率没有那么多高。这种典型的高频读场景特别适合缓

存。把热点查询前移到 Redis 或本地缓存，能明显减少数据库

压
力。

3. 你缓存商户信息时，Redis 的 key 是怎么设计的？

参考回答：

商户详情一般设计成 `cache:shop:{id}`，这样业务含义很清晰，也方便按 id 精确失效。商户类型列表则是固定 key，比如 `cache:shop-type:list`。我比较重视 key 命名，因为后面排查线

上问题时，规范的 key 会省很多时间。

4. 什么是缓存穿透？你在项目里是怎么解决的？

参考回答：

缓存穿透就是查询一个数据库里根本不存在的数据库，请求每次都打不到缓存，最终全落到数据库。我的处理方式是空值缓存，也就是数据库查不到时，在 Redis 里也写一个短 TTL 的空标记。这样下一

次同样的请求就不会再打到数据库。

5. 什么是缓存击穿？你在项目里用了哪两种思路？

参考回答：

缓存击穿一般是某个热点 key 过期瞬间，大量请求同时打到数据库

库。我在项目里用了两种常见思路：一种是互斥锁，也就是只允许

一个线程去查库并重建缓存；另一种是逻辑过期，也就是先返回旧

值，同时后台异步重建缓存。两种方案我都保留了，方便根据场景

选择。

6. 什么是缓存雪崩？你的项目里做了哪些预防？

参考回答：

缓存雪崩是大量 key 在同一时间失效，导致请求同时涌向数据库。

我的项目里主要通过合理 TTL、空值缓存、热点数据本地缓存以及

不把所有 key 设置成同一时刻过期这些方式做预防。如果是更大的

线上系统，还可以进一步做随机过期时间或者多级缓存。

7. 为什么空值缓存能解决缓存穿透？它有没有副作用？

参考回答：

因为空值缓存把“不存在”这个结果也缓存起来了，后续同样的请求

就不会再打数据库。副作用主要有两个，一个是会占一点 Redis 内

存，另一个是如果这个数据后来真的新建了，短时间内可能还会读

到空标记。不过因为空值 TTL 通常比较短，所以这个副作用是可接受的。

8. 逻辑过期和互斥锁方案分别适合什么场景？

参考回答：

互斥锁更适合要求数据一致性更高、而且可以接受少量线程等待的

场景。逻辑过期更适合热点数据，优点是大部分请求都能快速返

回，不会一起阻塞，但短时间内可能返回旧数据。我的理解是，逻

辑过期更偏“性能优先”，互斥锁更偏“数据新鲜度优先”。

9. 你后来为什么又加了本地缓存

Caffeine ?

参考回答:

因为商户详情这类热点数据如果全部压在 Redis 上, Redis 还是会

承受很高读取压力。加了 Caffeine 之后, 同一个实例上的热点请求

很多时候可以直接命中本地内存, 不需要再走网络访问 Redis。这

样可以进一步降低延迟, 也减轻 Redis 压力。

10. 本地缓存和 Redis 同时存在时, 会带来什么一致性问题?

参考回答:

最大的问题是某个节点更新了数据之后, Redis 虽然删掉了, 但其

他节点的本地缓存可能还是旧值。也就是说, 分布式共享缓存和单

机本地缓存之间会出现短暂不一致。这也是为什么我后面又补了失

效通知机制。

11. 你的项目里本地缓存失效是怎么做的?

参考回答:

更新商户信息时, 我会先更新数据库, 再删 Redis, 同时删掉当前节点本地缓存, 然后通过 Redis Pub/Sub 发一条失效消息。其他节

点订阅到这条消息后, 再把各自本地缓存清掉。这样做是“主动失效

- TTL 兜底”的组合。

12. 为什么要引入 Redis Pub/Sub 来通知本地缓存失效?

参考回答:

因为本地缓存只存在于各个 JVM 里，不共享。如果不做广播，当前

节点删了本地缓存，其他节点还是旧值。用 Pub/Sub 可以把“某个

key 失效了”这个事件广播出去，让所有节点都同步清理。

13. 如果 Pub/Sub 消息丢了，会发生什么？

你是怎么兜底的？

参考回答：

如果消息丢了，最直接的结果就是某些节点短时间内可能继续读到

旧的本地缓存。我的兜底方式是本地缓存本身也设置 TTL，即使广

播失败，过一段时间缓存也会自动失效。也就是说，这块我接受的

是“短暂最终一致”，而不是绝对强一致。

14. 你为什么给商户详情和商户类型分开配

置本地缓存？

参考回答：

因为这两类数据的访问模式和更新频率不一样。商户详情 key 多、

访问分散，但热点商户会非常热；商户类型列表 key 少、变化更

少、适合更长缓存时间。分开配置之后，可以更细地控制容量和

TTL。

15. 本地缓存最大容量和过期时间你是怎么

考虑的？

参考回答：

我主要是根据数据特点和访问频率来定。比如商户详情数量更多，

但单条对象不大，所以我给它更大的容量；商户类型列表总数少，

所以容量更小但 TTL 可以更长。对校招项目来说，这个值不是精确压测调出来的，但我会明确说明它是基于业务特征做的初步配置。

16. 如果面试官问你“为什么不全都用 Redis，不要 Caffeine”，你怎么答？

参考回答：

只用 Redis 当然也能做，但这样所有热点查询都要经过网络访问

Redis，延迟和压力都会更大。加 Caffeine 后，本机热点读可以直

接走内存，会更快。代价是一致性处理更复杂，所以我后面才补了

失效广播和 TTL 兜底。

17. Redis 热 key 会带来什么问题？你的项目里哪里可能产生热 key？

参考回答：

热 key 会导致某些 Redis key 被极高频访问，进而让单点压力过

高，甚至影响整体延迟。项目里最容易出现热 key 的地方是热点商

户详情、热门博客点赞数据、秒杀库存 key。越是用户集中访问的

内容，越容易形成热 key。

18. 如果某个商户特别热门，你现在的缓存方案还能撑住吗？为什么？

参考回答：

比单 Redis 缓存方案更能撑住，因为我加了本地缓存，很多请求可以在 JVM 内部直接命中。不过如果热度再继续上升，还是可能要进

一步做 TTL 优化、热点预热或者更细的监控。也就是说，现在这个

方案对校招项目已经比较完整，但不是无限上限。

四、商户查询与 GEO 专题

1. 你的商户分页查询和按地理位置查询分别是怎么实现的？

参考回答：

普通商户查询是按类型去数据库分页查，如果是商户详情则优先走

缓存。按地理位置查询时，我会先走 Redis GEO，根据坐标和半径

筛出附近商户，再拿这些商户 id 去数据库查详情，并按 Redis 返回

的顺序恢复距离排序。两种查询的核心区别是，一个以数据库分页

为主，一个以 Redis GEO 为主。

2. 为什么普通分页查询不能直接满足“附近商户”场景？

参考回答：

因为“附近商户”本质上不是普通的条件过滤，而是带有地理距离排

序的问题。数据库普通分页只能保证按页取数据，不能天然高效支

持“距离我最近”的排序。所以这个场景更适合交给 Redis GEO 这种

专门的数据结构处理。

3. 你为什么选 Redis GEO 做附近商户查询？

参考回答：

因为 Redis GEO 天然支持经纬度存储、附近搜索和距离计算，使用

成本低，查询速度也比较快。对于我这个项目的体量来说，用 GEO

就能比较自然地实现“附近商户”功能，不需要自己额外设计复杂的地理索引。

4. GEO 查询完之后，为什么还要再查一次数据库？

参考回答：

因为 Redis GEO 里只适合存经纬度和商户 id，不适合把完整商户

对象全塞进去。真正展示给前端时，还需要商户名称、地址、图片

这些完整字段，所以要根据 GEO 返回的商户 id 再查数据库或者缓

存，组装出完整结果。

5. 你为什么要保留商户 id 的顺序，再去数据库里 ORDER BY FIELD？

参考回答：

因为 GEO 返回的结果已经按距离排好了，如果我后面直接按 id 去

数据库查，数据库默认不会保留这个距离顺序。为了保证最终结果

还是“离我最近的在前面”，我需要把 Redis GEO 返回的 id 顺序带过

去，再按这个顺序重新排序。

6. 如果 Redis GEO 查不到数据，你的代码是怎么兜底的？

参考回答：

如果查不到，我会回退到普通分页查询，至少保证接口还能返回结

果，而不是直接空掉或者报错。这样做虽然不再是严格的“附近商

户”，但能提升系统可用性。兜底逻辑更多是保证服务稳定。

7. 附近商户查询为什么要限制半径？半径设大了有什么问题？

参考回答：

限制半径一方面更符合“附近”的业务语义，另一方面也能避免一次

查出太多数据。半径太大时，Redis GEO 返回结果会变多，后面的

数据库查询和排序开销也会明显增加。所以一般都会给一个合理的

搜索范围，比如几公里以内。

8. 如果面试官问你 GEO 的底层原理，你会怎么解释到校招合适的深度？

参考回答：

校招场景下我会说，Redis GEO 底层本质上还是借助有序集合来做

的，只是对经纬度做了特殊编码，方便按地理位置做范围搜索和距

离计算。我不会上来就讲特别底层的数学细节，而是重点讲它适合

“附近搜索”这个业务场景。

9. 如果商户坐标变化了，你的 GEO 数据怎么更新？

参考回答：

如果商户坐标变了，就需要重新写 GEO 数据，相当于更新该商户

在对应 GEO key 里的坐标。实际实现上可以理解成重新 add 一次

同一个商户 id 的经纬度，同时把商户详情缓存一起失效掉，保证数

据一致。

10. 你觉得这个模块还有哪些可以继续优化的点？

参考回答：

可以继续优化的点包括：增加更细粒度的缓存、热点区域预热、坐标更新和缓存更新的一致性处理，以及对 GEO 查询结果做分页优化。再往后如果体量更大，还可以考虑专门的搜索或地理位置服务。

五、博客、点赞、关注 Feed 专题

1. 你的博客点赞为什么用 Redis ZSet，而不是 Set？

参考回答：

因为点赞不只是要去重，有时还要保留顺序，比如展示最近点赞的

前几个人。Set 只能表示“有没有点过赞”，而 ZSet 既能去重，又能

通过 score 记录时间顺序。所以我选了 ZSet。

2. ZSet 里的 member 和 score 分别存了什么？

参考回答：

member 存的是用户 id，score 存的是点赞时间戳。这样我既能判

断某个用户是否点过赞，也能按时间倒序拿到最近点赞列表。

3. 你为什么需要 score？如果没有 score 行不行？

参考回答：

如果只是做点赞去重，理论上 Set 也行。但我项目里还要支持“谁最

近点过赞”这类展示，所以 score 很有意义。没有 score，就没法方

便地做有序展示。

4. 你怎么判断当前用户有没有给某篇博客点过赞？

参考回答：

我会去 Redis 的点赞 ZSet 里查这个用户 id 的 score，如果 score

存在，就说明点过赞；如果不存在，就说明没点过。这个判断比每

次去数据库查更快。

5. 你怎么拿到一篇博客最近点赞的前几个人？

参考回答：

我会从对应博客的点赞 ZSet 里按 score 排序取前几条，再根据用

户 id 去查用户信息。因为 ZSet 本身就是有序的，所以这种“最近点

赞列表”做起来很自然。

6. 点赞功能为什么要同时改数据库和 Redis？

参考回答：

数据库是最终持久化数据，Redis 是为了高效查询和展示。只改数

据库会导致查询慢，只改 Redis 又不能保证持久化。所以我会数据

库和 Redis 一起维护，不过数据库仍然是最终真实来源。

7. 如果数据库更新成功，但 Redis 更新失败了，会发生什么？

参考回答：

这样会导致短时间内点赞展示和真实数据不一致，比如数据库点赞

数已经加了，但 Redis 里的用户点赞集合没更新。这个问题本质上

是缓存和数据库双写一致性问题。对这种项目，我会接受短时不一

致，并通过后续刷新或重新加载来恢复。

8. 关注功能为什么适合用 Redis Set ?

参考回答:

因为关注关系天然是一个集合, 而且最常见的操作是 “是否关注” 和

“共同关注”。Set 的成员唯一、不重复, 特别适合这个场景。

比如

SISMEMBER 判关注、SINTER 求共同关注, 都很方便。

9. 共同关注是怎么做出来的? 为什么这个场

景适合 Redis ?

参考回答:

共同关注就是把当前用户和目标用户的关注集合做交集, 得到双方

都关注的人。Redis Set 直接支持交集运算, 所以这个功能实现起

来非常简单, 而且效率也高。对这类关系型集合场景, Redis 很合

适。

10. 你的 Feed 流是推模式、拉模式还是推拉

结合? 为什么?

参考回答:

我项目里更接近推模式, 也就是用户关注了别人后, 对方发博客时

会把博客 id 推到粉丝的收件箱里。这样用户进首页时查自己的收件

箱就行, 读取会更快。缺点是写扩散更明显, 但对这个项目体量是

可以接受的。

11. 关注推送为什么用 ZSet ?

参考回答:

因为 Feed 流需要按时间排序, ZSet 很适合做 “按时间倒序查看动态

流”。member 可以是博客 id, score 可以是发布时间戳, 这

样后

面滚动分页就比较自然。

12. 滚动分页为什么不能直接用传统的
page/current ?

参考回答:

因为 Feed 流这类按时间变化的数据, 如果用普通页码分页,
前面

插入新数据后, 后面的页内容很容易错乱、重复或遗漏。滚动
分页

更适合这种时间流场景, 它是基于 “上一次查到哪里” 继续往后
翻。

13. 你项目里的滚动分页具体用了哪几个参
数?

参考回答:

一般会带上 lastId 或 maxTime 这类上次查询到的时间分值,
再

配合偏移量 offset 处理同分值数据。这样既能保证连续翻页,
也

能避免重复和漏数据。

14. 如果面试官问你 “点赞和 Feed 这两个
Redis 结构为什么不同”, 你怎么解释?

参考回答:

点赞更关注的是 “某个用户有没有点过赞” 和 “最近谁点过赞”
, 所以用

ZSet 存用户和时间。Feed 则关注的是 “当前用户按时间看到哪
些内

容”, 本质上是另一种按时间排序的收件箱。虽然都用了 ZSet
, 但

它们的业务含义和 member、score 的设计是不一样的。

六、秒杀链路专题

15. 你项目里秒杀链路完整流程是什么? 请按
顺序讲出来。

参考回答：

用户请求先进入秒杀接口，接口拿到用户 id 和优惠券 id 后，先执行 Redis Lua 脚本，脚本会判断库存是否充足、用户是否重复下单，并在 Redis 中先预扣库存、记录一人一单状态；如果通过，就把订单信息写入 Redis Stream；后面 Relay 从 Stream 里读取消息

并转发到 Kafka；Kafka 消费者异步落库，真正扣减数据库库存并

创建订单；如果消费失败，再根据异常类型决定重试、进入死信或者回滚 Redis 状态。这条链路的核心目标是“前端快返回、数据库削峰、最终一致”。

2. 为什么秒杀不能像普通下单一样，直接查库扣库存？

参考回答：

因为秒杀场景并发太高，如果所有请求都直接打数据库，数据库很

容易成为瓶颈，还可能出现锁竞争严重、响应很慢的问题。秒杀的

核心是把资格校验和流量拦截尽量前移，让数据库只处理少量真正

有效的请求。

3. 你为什么把资格校验放在 Redis Lua 里做？

参考回答：

因为 Lua 能在 Redis 里把多步操作合成一个原子操作。秒杀里最关

键的是“查库存、查是否重复下单、扣库存、记录状态、写消息”这

些动作不能被并发打断，所以用 Lua 很合适。这样可以减少多次网

络往返，也能保证一致性。

4. Lua 脚本在你的秒杀链路里到底做了哪几件事？

参考回答：

它主要做了四件事：判断库存够不够；判断用户是不是已经下过单；如果通过，就预扣 Redis 库存；最后把订单消息写进 Redis Stream。也就是说，Lua 负责把秒杀资格判断和消息入队这块一次性做完。

5. 什么叫“一人一单”？你是在哪几层做这个保证的？

参考回答：

一人一单就是同一个用户对同一张秒杀券只能下一次单。我主要在

三层做保证：Redis 层通过 Lua 和用户集合做第一层快速拦截；数

据库层通过唯一索引做最终兜底；业务层还会结合分布式锁降低并

发冲突概率。这样做是多层防线，而不是只依赖单点。

6. 为什么你做了 Redis 校验以后，数据库层还要再做一次兜底？

参考回答：

因为 Redis 预扣和数据库落库之间是有时间差的，而且异步链路里

还有重试、重复消费这些因素。Redis 能解决大部分并发问题，但

不能完全代替数据库最终约束。所以数据库唯一索引和库存条件仍

然必须保留。

7. 秒杀时库存为什么先在 Redis 里预扣，而不是直接更新数据库？

参考回答：

因为 Redis 更适合承接高并发瞬时请求，预扣库存可以在前面快速过滤无效请求，避免每个请求都打数据库。数据库只负责最后的真实落库和强约束，这样整体吞吐量会更高。

8. Redis 预扣库存和数据库真实库存不一致时怎么办？

参考回答：

如果后面数据库落库失败，就需要做 Redis 回滚，把预扣掉的库存补回去，同时把一人一单集合里的用户状态清掉。也就是说，Redis 预扣不是最终结果，最终还是要靠数据库成功落库来确认。

9. 你的项目里 Redis 回滚库存是在哪些场景触发的？

参考回答：

主要是在消息消费端落库失败，而且这个失败被判断为不应该继续

重试时触发，比如数据库库存不足、业务校验失败、唯一约束冲突

之外的一些不可恢复异常。简单理解就是：Redis 先扣了，但后面

订单没成，就要补回去。

10. 什么情况下你会认为这是“可恢复异常”，什么情况下是“不可恢复异常”？

参考回答：

像临时网络波动、数据库短暂不可用、消息系统短时异常，我会倾向认为是可恢复异常，可以重试。像业务数据本身有问题、库存已

经没有、逻辑明确不可能成功的情况，我会认为是不可恢复异

常,

不应该盲目重试。这个区分是为了避免无效重试。

11. 为什么唯一索引也能帮助保证一人一单?

参考回答:

因为唯一索引从数据库层面规定了 (user_id, voucher_id) 组合不能重复。即使前面的 Redis 或消息链路有漏网请求, 数据库插入时也会因为唯一索引冲突失败。它是最终兜底保障。

12. 你的数据库扣库存 SQL 为什么要带 $stock > 0$?

参考回答:

这是最基础的乐观约束, 能避免库存已经是 0 了还继续往下扣。即

使并发请求同时到数据库, 只有满足 $stock > 0$ 的更新才会成功, 这样可以避免超卖。

13. 如果两个请求同时走到数据库扣库存, 会不会超卖?

参考回答:

正常情况下不会, 因为 SQL 本身带了 $stock > 0$, 数据库只会让

真正还有库存的更新成功。再加上前面 Redis 已经先拦掉一大部分

请求, 所以数据库层的并发压力和超卖风险都会小很多。

14. 你为什么还要再加 Redisson 分布式锁?

参考回答:

Redisson 锁不是唯一防线, 但它能进一步减少同一用户并发创建

订单时的竞争, 特别是在消息消费和异步落库阶段。它更多是降低

并发冲突概率, 减轻数据库唯一索引冲突带来的无效操作。

15. 如果面试官问你“有唯一索引了，为什么还要锁”，你怎么答？

参考回答：

唯一索引是最终兜底，但它发生作用时已经走到数据库插入阶段了；锁则是在更前面减少同一用户的并发冲突，能少做很多无效工作。也就是说，一个是“最后挡住错误”，一个是“前面尽量不让错误发生”。

16. 秒杀成功时，接口为什么能很快返回？

订单不是还没真正落库吗？

参考回答：

因为我把真实落库放到了异步链路里。接口只要完成 Lua 校验、预

扣库存和消息入队，就可以先返回订单 id 或成功结果，不需要同步

等待数据库写入。这样就把高并发下最慢的数据库操作从用户请求

线程里拆出去了。

17. 这种“先返回成功、后异步落库”的设计，有什么风险？

参考回答：

风险就是用户感知上的“成功”和数据库最终结果之间会有短暂差

异。比如用户已经收到成功响应，但后面消息消费失败了，订单可

能最终没创建成功。所以这种设计一定要配套补偿、幂等和失败处

理机制，不能只追求接口快返回。

18. 你的项目里是怎么处理“用户看到成功，但最终落库失败”这种情况的？

参考回答：

我这边的思路是把失败分成可恢复和不可恢复两类。可恢复的就继续重试；不可恢复的就做 Redis 回滚，并记录失败任务或死信消息，方便后续补偿和排查。对前端来说，这类问题更适合通过订单状态、补偿处理或后续查询来感知最终结果。

19. 如果面试官让你描述秒杀链路里最核心的三道防线，你会怎么总结？

参考回答：

第一道是 Redis Lua，快速做库存和一人一单校验；第二道是异步消息链路，通过 Stream 和 Kafka 做削峰和解耦；第三道是数据库唯一索引、 $stock > 0$ 和分布式锁做最终一致性兜底。这样三层分别承担“前置拦截、异步缓冲、最终约束”的职责。

七、Kafka 与消息队列专题

20. 你的项目里为什么已经用了 Redis Stream，还要再引入 Kafka？

参考回答：

因为 Redis Stream 更像一个轻量级入口缓冲，而 Kafka 更适合做正式的高吞吐异步消息队列。我的项目里保留了 Stream 作为 Lua 脚本之后的第一站，再由 Relay 转发到 Kafka，这样既能减少改动成本，也能把异步消费能力提升起来。

21. Redis Stream 和 Kafka 在你的项目里分别承担什么职责？

参考回答：

Redis Stream 主要承担的是秒杀接口之后的快速入队缓冲，

Kafka

承担的是正式的异步削峰、可靠消费、重试和死信处理。一个更靠

近入口，一个更靠近正式消息系统。

22. 你项目里的 Relay 中转层是干什么的？

参考回答：

Relay 会从 Redis Stream 里读取订单消息，然后转发到 Kafka，对

Kafka 来说它是一个生产者。它相当于把“Lua 已经写入 Stream”的

现有入口和“后端正式用 Kafka 消费”的新方案桥接起来。

4. 为什么不直接在业务代码里把消息发到

Kafka？

参考回答：

因为我这个项目的秒杀入口已经在 Lua 里完成了原子校验和 Stream 写入，如果直接改成同步发 Kafka，链路改动会更大，而

且入口线程也会更依赖 Kafka 是否可用。加一个 Relay 可以让项目

平滑演进，同时保留入口缓冲能力。

5. 你的 Kafka Topic 是什么？消息体里包含

哪些关键字段？

参考回答：

核心 Topic 是秒杀订单相关的 topic，比如 seckill-order。

消息

体里至少会带订单 id、用户 id、优惠券 id，还会带创建时间和

traceId 这种方便追踪的字段。因为消费者落库时必须依赖这些关

键信息。

6. 你为什么给消息加 `traceId` ? 它有什么用?

参考回答:

主要是为了方便排查链路问题。后面如果需要从 Redis Stream、Relay、Kafka、消费者日志一路往后追, 就可以通过同一个 `traceId` 把一笔订单请求串起来。虽然这是校招项目, 但这个思路更贴近真实工程实践。

7. 消费者为什么要手动 ACK, 而不是自动提交?

参考回答:

因为自动提交有可能在业务还没真正处理成功前就把 `offset` 提交了, 万一后面报错, 就可能出现消息丢失语义。手动 ACK 可以做到“处理成功再确认”, 更适合订单这类不能随便丢的业务。

8. 什么叫消息重复消费? 你的项目里是怎么做幂等的?

参考回答:

重复消费就是同一条消息因为重试、网络抖动、消费者重启等原因, 被处理了不止一次。我的幂等主要靠几层一起做: 前面 Redis 一人一单校验、数据库唯一索引、消费端对重复插单的兜底处理。

这样即使消息重复到达, 也不会重复创建订单。

9. 什么情况下 Kafka 消费失败要重试? 什么情况下不该重试?

参考回答:

像临时网络异常、数据库连接问题这类短时故障可以重试, 因为有

可能下一次就恢复了。像库存已经没有、数据本身非法、业务条件

明确不成立这类情况就不该一直重试，不然只是浪费资源。这个区

别我会结合异常分类来讲。

10. 你的死信队列是用来解决什么问题的？

参考回答：

死信队列主要处理“多次重试后还是失败”的消息，避免这些消息一

直卡在主消费流程里。进入死信后，可以单独记录、排查或后续补

偿。对于秒杀订单这种重要业务，死信队列可以提升问题可观察性。

11. 死信消息到来之后，你项目里后续怎么处理？

参考回答：

死信消息到了之后，我会把关键信息落到失败任务表或对应补偿表

里，记录失败原因、重试次数和原始业务字段，后面再由补偿任务

或人工排查处理。重点是不能让失败消息“消失”。

12. 如果 Kafka 暂时不可用，你的链路会发生什么？

参考回答：

如果 Kafka 暂时不可用，Relay 转发阶段会失败，Redis Stream 里

的消息不会被成功 ACK，后面还可以继续重试处理。也就是说，入

口不会立刻把数据完全丢掉，Stream 能起到缓冲作用。

13. 如果 Relay 成功从 Redis Stream 读到消息，但发 Kafka 失败了，会怎么处理？

参考回答：

这种情况下不能 ACK Redis Stream 的消息，否则消息就丢了。正

确做法是让这条消息保留在 Pending 或未成功确认状态，后续继续

重试发送 Kafka。也就是说，ACK 必须建立在“转发成功”之后。

14. 为什么消费者分组对秒杀场景重要？

参考回答：

因为消费者分组能让同一个 topic 的多个分区并行消费，提高吞吐

量。秒杀场景下消息量大，如果没有消费组并行能力，单消费者容

易成为瓶颈。消费组还能配合 rebalance 提高容错性。

15. 你为什么考虑过 consumer name 不能写死？

参考回答：

因为一旦多实例部署，如果 consumer name 写死，不同实例在

Redis Stream 或消费语义上可能会互相覆盖或混淆，尤其是 Pending List 恢复时更容易出问题。改成实例唯一之后，链路在多

实例场景下会更安全。

16. 如果同一个用户的多条消息落到不同分区，会有什么问题？

参考回答：

最大的问题是顺序性可能丢掉。同一个用户的多次请求如果被不同

消费者并行处理，就可能出现先后的业务顺序错乱。所以如果业务

很依赖同用户顺序，通常会考虑用用户 id 作为 key，让相关

消息尽

量落到同一个分区。

17. 如果面试官问你 offset 是什么，你会怎么结合你的项目解释？

参考回答：

offset 可以理解成 Kafka 里每个分区消息的消费进度标记。消费者

处理到哪里、确认到哪里，都要靠 offset 记录。结合我项目来说，

只有订单消息真正处理成功后，才应该提交对应 offset。

18. 你项目里 Kafka 这块最容易被面试官追问的细节是什么？

参考回答：

我觉得最容易被追问的是“为什么既有 Redis Stream 又有 Kafka”、

“手动 ACK 为什么重要”、“失败消息和死信怎么处理”这三块。因为

这些问题最能看出你是不是真的理解消息链路，而不是只会堆技术名词。

八、分布式锁与一致性专题

1. 你项目里哪些地方用到了分布式锁？

参考回答：

最典型的是秒杀订单落库阶段，用来降低同一用户并发创建订单时

的冲突。除此之外，在缓存击穿的互斥重建场景里，本质上也会涉

及到分布式锁思路。

2. 你为什么选 Redisson，而不是自己手写一个简单 setnx 锁？

参考回答：

Redisson 更成熟，已经帮我处理了很多锁相关细节，比如续

期、

可重入、异常边界等。自己手写简单锁当然也能做，但容易在 owner 校验、过期、误删别人的锁这些细节上踩坑。我项目里后来

也确实专门修过这类问题。

3. 你有没有自己实现过简单 Redis 锁？它和 Redisson 的差距在哪？

参考回答：

有过，简单版本通常就是 setIfAbsent 加过期时间，再在解锁时

配合 Lua 校验 owner 。它能覆盖基础场景，但和 Redisson 比起

来，功能和健壮性都差很多，比如看门狗续期、可重入、封装程度

都不如 Redisson 。

4. 为什么解锁时不能直接 delete key ？

参考回答：

因为锁有可能已经超时，或者当前线程根本不是持锁者。如果直接

删 key ，就有可能把别的线程刚拿到的锁删掉，造成严重并发问

题。所以解锁一定要先确认这把锁是不是自己的。

5. 为什么安全解锁通常要 “ 校验 owner 再删除 ” ？

参考回答：

因为分布式锁的关键不只是 “ 有没有锁 ” ，而是 “ 当前谁持有锁 ” 。只有

校验 owner 成功之后再删，才能避免误删别人持有的锁。这个校

验通常还要保证原子性，所以常配合 Lua 做。

6. Lua 解锁脚本在这里起到了什么作用？

参考回答：

Lua 脚本可以把 “先比较 owner，再删除 key” 合成一个原子操作，
避免中间被其他线程插进来。否则如果先 get 再 delete，中间就可
能发生竞争条件。

7. 如果一个线程没拿到锁，却把别人的锁删掉了，会造成什么后果？

参考回答：

这样后果很严重，可能导致多个线程同时进入本该互斥的逻辑，比
如同时重建缓存或者同时创建订单。表面看只是一个解锁小 bug，
实际上会直接破坏并发保护。

8. 你是怎么发现并修复这个风险的？

参考回答：

我是从代码逻辑里发现这个风险的：如果没拿到锁的线程也执行了 finally 解锁，就存在误删别人锁的可能。修复思路就是让解锁只在真正持锁成功时发生，并且把锁值改成 owner token，再通过 Lua 原子解锁。

9. 如果锁过期时间设置太短，会有什么问题？

参考回答：

锁过期太短的话，业务还没执行完锁就没了，别的线程可能提前进
来，导致本来需要互斥的逻辑失效。特别是高并发或耗时波动较大
的场景，这个问题更明显。

10. 如果锁过期时间设置太长，又会有什么问题？

参考回答：

锁过期太长时，如果持锁线程异常退出，其他线程会等很久，系统吞吐量会下降，用户体验也会变差。所以锁时间要合理设置，既不能太短，也不能太长。

11. Redisson 的看门狗机制你了解吗？校招场景下你怎么简洁回答？

参考回答：

我会简单说：Redisson 有一个看门狗机制，会在业务线程还持有锁且正常运行时，自动帮你续期，避免锁因为时间太短提前过期。

它的作用就是让锁更稳，不需要手动特别精细地估算业务执行时间。

12. 你的项目里哪些一致性问题属于“最终一致性”，哪些属于“强一致性做不到也没必要”？

参考回答：

像本地缓存和 Redis 之间的同步，我接受的是最终一致性，因为通过广播失效和 TTL 兜底就足够了。

像订单落库、库存扣减这种核心业务，我会更强调最终真实结果必须正确，所以要通过数据库约束、幂等和补偿去保证。不是所有地方都需要强一致，但关键链路一定要保证最终正确。

九、限流与稳定性专题

13. 你项目里为什么要做接口限流？

参考回答：

因为像发送验证码、登录、秒杀下单这类接口特别容易被高频调

调

调

因为像发送验证码、登录、秒杀下单这类接口特别容易被高频调

用，既可能被恶意刷，也可能在热门活动时把系统打爆。限流可以

保护后端服务稳定，也是比较真实的工程需求。

14. 哪几个接口最需要限流？为什么？

参考回答：

我觉得最需要的是发送验证码接口、登录接口和秒杀下单接口。验

证码和登录容易被恶意刷，秒杀接口则是天然高并发入口，对系统

冲击最大。

15. 你的限流是基于什么实现的？

参考回答：

我是基于自定义注解、AOP 切面和 Redis 计数来做的。请求进来

后，切面按某种维度生成限流 key，然后在 Redis 里做计数和过期

控制，超过阈值就直接返回限流提示。

4. 自定义注解 @RateLimit 的作用是什么？

参考回答：

它的作用是把限流规则和具体接口绑定起来，让哪些接口要限流、

按什么维度限流、阈值是多少这些信息更清晰。这样做比把限流逻辑

写死在 controller 里更灵活。

5. 你是怎么区分“按 IP 限流”和“按用户限流”的？

参考回答：

本质上是限流 key 的组成不同。按 IP 限流时 key 里会带客户端

IP；按用户限流时 key 里会带用户 id。也就是说，限流逻辑

本身差

别不大，关键在于怎么生成唯一维度。

6. 你的限流算法更接近固定窗口、滑动窗口还是令牌桶？

参考回答：

我现在做的是比较轻量的固定窗口方案，用 Redis 做窗口内计数和

过期时间控制。它实现简单，适合校招项目。后续如果想更平滑，

可以再往滑动窗口或令牌桶升级。

7. 固定窗口限流有什么缺点？

参考回答：

最大的问题是窗口边界突刺，也就是一个窗口最后一秒和下一个窗

口第一秒可能连续放过很多请求，实际限流效果不够平滑。所以它

简单但不算最优。

8. 如果有人在窗口边界疯狂请求，会发生什么？

参考回答：

可能会出现短时间内实际通过的请求数明显超过阈值，因为两个相

邻窗口都各自放行了一部分请求。这也是固定窗口的典型问题。

9. 你项目里的限流返回值是怎么设计的？

参考回答：

一般会返回一个统一的失败结果，比如提示“请求过于频繁，请稍后

再试”。对前端来说，最重要的是知道这是一个可理解的受控失败，

而不是系统异常。

10. 如果面试官问你“后面要继续升级限流，你准备怎么改”，你怎么答？

参考回答：

我会说可以从三个方向升级：第一，把固定窗口改成滑动窗口或令牌桶；第二，把限流规则做成配置化甚至动态化；第三，在应用层之外再增加网关或 Nginx 层的粗粒度限流。这样从算法和部署层都能继续增强。

十、测试与压测专题

11. 你的项目里除了单元测试，还有哪些测试方式？

参考回答：

除了 Java 这边的单元或回归测试，我项目里还有 API 自动化测试、UI 自动化测试，以及一些压测和验证记录。也就是说，我不是只关注功能能不能写出来，也会关注怎么验证它。

2. 为什么你这个项目需要 API 自动化测试？

参考回答：

因为很多核心逻辑最终都是通过接口暴露出来的，比如登录、商户查询、秒杀下单。API 自动化测试可以比较快地验证主流程有没有被改坏，也适合做回归。

3. 为什么还要做 UI 自动化测试？

参考回答：

因为有些业务流程从页面角度更完整，比如登录、商户浏览、秒杀操作这些。UI 自动化虽然比 API 测试更重，但能补上接口测试

覆盖

不到的用户视角。

4. 你项目里哪些模块最值得补单元测试？

参考回答：

我觉得最值得补的是秒杀链路里的事务边界、缓存锁实现、消息消费幂等和配置规范这几块。因为这些地方一旦写错，问题通常不会体现在简单的接口成功失败上，而是会藏在并发和异常场景里。

5. 你最近补的测试主要覆盖了哪些风险点？

参考回答：

主要覆盖了三类风险：订单落库入口有没有事务边界、缓存锁是否会误删别人的锁、Relay consumer name 是否支持多实例唯一，以及示例配置里有没有保留不合适的演示口令。这些都比较偏工程风险，而不是简单业务 happy path 。

6. 为什么“事务边界”也值得专门写测试？

参考回答：

因为事务边界一旦错了，最危险的问题是代码看起来能跑，但数据可能会不一致。比如库存扣了、订单没建成，这种问题非常难通过普通功能测试发现，所以我觉得值得专门验证。

7. 你为什么给配置模板也加了测试？

参考回答：

因为配置模板虽然不是业务代码，但它直接影响项目对外展示和团队使用方式。比如示例文件里保留了 123456 这种演示口令，会让仓库显得不够规范，所以我用一个很轻量的测试把这个规则固

定下
来。

8. 如果面试官问你“什么样的代码最难测”，
你会结合项目里哪个模块来回答？

参考回答：

我会结合秒杀异步链路来答，因为它涉及 Redis、消息队列、
数据

库、锁和异常补偿，是跨组件的，不像纯工具类那么好测。这
类代

码最难的是场景复杂，不是写断言本身难。

9. 你做过压测吗？压测主要压的是哪条链
路？

参考回答：

做过，主要是围绕秒杀链路和部分商户查询链路去看高并发下
的表

现。因为这两块一个是最典型的高并发写场景，一个是典型的高
频

读场景，比较有代表性。

10. 秒杀链路压测时，你最关注哪些指标？

参考回答：

我会重点看 QPS、平均响应时间、失败率、数据库压力、Redis
压

力和消息堆积情况。对秒杀来说，不只是接口快不快，还要看后面
的异步链路能不能接住流量。

11. 如果 QPS 上不去，你会优先排查哪几
层？

参考回答：

我会先看入口是不是卡在 Redis 或 Lua，然后看 Kafka 或
Stream

有没有堆积，再看数据库是不是成为瓶颈，最后看应用线程池
和连

接池配置。基本就是按“入口拦截层、消息层、数据库层、应用资源层”这个顺序排查。

12. 如果面试官追问“你怎么证明自己真的压过”，你会拿哪些结果或现象来讲？

参考回答：

我会讲我关注的指标变化，比如请求量上来之后 Redis 压力先升、

数据库压力相对更稳，或者消息积压什么时候开始出现，以及我做

了哪些链路改造后这些现象有什么变化。这样回答比单说“我压过”

更有说服力。

十一、项目优化与追问专题

13. 你项目里做过哪些“从能跑到更工程化”的优化？

参考回答：

主要有商户两级缓存、本地缓存失效广播、秒杀链路异步化、Kafka 死信和补偿、分布式锁风险修复、配置脱敏、测试补充这些。简单说就是从“功能做出来”往“并发、稳定性、可维护性”继续

补。

2. 你为什么把秒杀链路从简单实现改到现在这个版本？

参考回答：

因为简单实现虽然能跑，但在高并发下数据库压力太大，而且幂

等、失败重试、补偿这些问题考虑得不够完整。我后来逐步把资格

校验前移到 Redis，再通过消息队列做异步削峰，整体链路就

更像

真实业务系统。

3. 你自己最满意的一次改造是什么？具体解决了什么问题？

参考回答：

我最满意的是秒杀链路异步化和多层防线这次改造。它解决了数据

库被高并发直接打爆的问题，也把一人一单、重复消费、落库失败

补偿这些问题一起考虑进来了。这个改造前后，项目的工程感差别

很明显。

4. 你在做改造时，有没有发现原方案看起来能用，但实际上有风险？举个例子。

参考回答：

有，最典型的是缓存锁实现。表面上看加锁、解锁都写了，但如果

没拿到锁的线程也在 finally 里解锁，就可能删掉别人的锁。这种问

题运行时不一定立刻暴露，但一旦高并发就很危险。

5. 如果让你再做一次这个项目，你会最先重构哪一块？

参考回答：

我会优先把文档、配置和消息链路抽象得更统一一些，尤其是缓存和秒杀相关模块。因为业务能跑之后，真正影响后续维护效率的往往是结构清晰度，而不是少写一个接口。

6. 如果让你把这个项目部署成多实例线上版本，你最担心哪几个问题？

参考回答：

我最担心的是本地缓存一致性、Redis 热点压力、消息消费的多实

例协同、分布式锁边界和配置管理。单机和多机最大的区别就是很

多原本简单的问题会变成一致性和协同问题。

7. 你觉得这个项目现在还有哪些不足？不要只说“可以继续优化”，请说具体点。

参考回答：

具体不足有几个：限流算法还比较轻量，没做到更平滑的滑动窗口

或令牌桶；监控和告警比较弱；集成测试还不够完整；另外部分文

档和资料结构还可以继续整理。也就是说它已经有工程化意识，但

离真正生产级还有距离。

8. 如果面试官问你“这个项目里哪块最像真实公司项目”，你会怎么答？

参考回答：

我会答秒杀链路和缓存链路。因为这两块都有很明确的性能目标、

并发问题和工程取舍，不只是“写接口”，而是要解决真实系统里常

见的流量和一致性问题。

9. 如果面试官问你“哪块最不像真实公司项目”，你会怎么答？

参考回答：

我会说监控、告警、灰度、权限体系这些部分还比较轻，不像公司项目那样完整。因为这个项目的重点是业务链路和核心工程点，不是完整地还原企业平台能力。

10. 如果让你只保留三个亮点写在简历上，你会选哪三个？

参考回答：

我会写三个：第一，基于 Redis Lua + Redis Stream + Kafka

- MySQL 的秒杀异步链路；第二，基于 Caffeine + Redis + Pub/Sub 的两级缓存方案；第三，接口限流、失败补偿和回归测试

这些稳定性增强。这样能同时体现并发、缓存和工程化能力。

十二、细节拷打专题

1. 你的秒杀 Lua 脚本入参具体有哪些？返回值有哪些？

参考回答：

核心入参一般包括优惠券 id、用户 id、订单 id 这几个关键字段。

返回值我设计成状态码风格，比如 0 表示成功，1 表示库存不足，

2 表示重复下单。这样 Java 侧判断逻辑会很清晰。

2. 你在 Redis 里给秒杀库存、订单集合分别用的 key 前缀是什么？

参考回答：

库存 key 一般是 seckill:stock:{voucherId}，一人一单集合一般是 seckill:order:{voucherId} 这种风格。前缀设计上我会尽量让 key 一眼能看出用途，便于排查。

3. 你的本地缓存和 Redis 缓存默认 TTL 大概是多少？

参考回答：

商户详情、本地缓存和 Redis 缓存的 TTL 我是区分开的，本地缓存

一般更短，Redis 可以更长一些，商户类型列表还能再长一些。这

里我会根据业务特点回答，而不是死背一个数值，因为不同对象

TTL 设计本身就不完全一样。

4. 你的 Kafka 消费者组名配置在哪里？

参考回答：

是在 application.yaml 里配置的，通过 Spring Kafka 的消费者

配置读取。这样比写死在代码里更规范，也方便后面调整。

5. 你的 Redis Stream 名字是什么？

参考回答：

秒杀订单这条链路的 Stream 名字是类似 stream.orders 这样的

命名。它承担的是 Redis 入口消息流的角色。

6. 你的订单消息里为什么同时带
orderId、userId、voucherId？

参考回答：

因为消费者落库时需要这些最核心的业务字段。orderId 用来标识

订单本身，userId 用于一人一单和分布式锁逻辑，voucherId 用

于库存扣减和业务定位。缺任何一个都不方便做完整处理。

7. 你在代码里是怎么区分“重复下单”和“库存
不足”的？

参考回答：

我是在 Lua 返回值和后端结果转换里区分的，不同状态码对应不同

业务结果。这样接口返回和后续日志都能比较清楚地区分是哪一种失败。

8. 你的幂等主要靠哪几层一起完成？

参考回答：

主要靠 Redis 一人一单集合、数据库唯一索引、消费端重复插单兜

底这几层一起做。不是只靠某一层单独保证，而是组合起来提升可

靠性。

9. 你的 Redis 回滚库存脚本或逻辑，除了加库存，还做了什么？

参考回答：

除了把库存补回去，还要把一人一单集合里的用户状态移除掉，不

然这个用户后续会一直被误判成已经下过单。也就是说，回滚不只

是补库存，还要恢复业务状态。

10. 你的缓存更新时，为什么既要删 Redis，又要清本地缓存？

参考回答：

因为 Redis 和本地缓存都可能保存旧数据，只删其中一个是不够

的。删 Redis 是为了清掉分布式共享缓存，清本地缓存是为了避免

当前节点继续读旧值。再配合广播，才能尽量保证多节点一致。

11. 你的测试里为什么没有把所有东西都做集成测试？

参考回答：

因为集成测试覆盖范围大、依赖多、运行成本高，不适合所有细节都用它来验证。有些逻辑更适合轻量级单元或回归测试，比如配置模板校验、注解存在性、锁调用方式这些。测试也要分层，而不是一股脑全做集成。

12. 如果面试官把代码打开，随机指到一个类让你讲，你最怕他指到哪个类？为什么？

参考回答：

如果一定要说，我最不想被突然指到特别偏工具封装、而不是主业

务链路的类，因为这种类细节很多，但业务成就感没那么强。

不过

即使被指到，我也会尽量先讲这个类解决了什么问题，再讲实现细节，而不是直接陷进代码行数里。

十三、模拟追问链

追问链 1：秒杀

13. 你先讲一下秒杀链路。

参考回答：

我会先按“Redis Lua 资格校验和预扣 -> Redis Stream 入队 -> Relay 转发 Kafka -> Kafka 消费者异步落库 -> 失败重试或补偿”这个顺序讲。重点不是背组件名，而是讲清每一层为什么存在。

14. 为什么不用数据库直接扣库存？

参考回答：

因为数据库抗不住秒杀瞬时高并发，而且大量无效请求没必要都打到数据库。把资格校验前移到 Redis，才能有效削峰。

15. 为什么要 Lua？

参考回答：

因为要把库存判断、重复下单判断、预扣库存和写消息这些动作做成 Redis 内部原子操作。用 Lua 比在 Java 里发多次 Redis 命令更安全。

4. 为什么有了 Lua 还要数据库唯一索引？

参考回答：

因为 Redis 不是最终真相来源，异步链路里仍然可能出现重复消费或边界竞争。唯一索引是最终兜底。

5. 为什么有唯一索引还要 Redisson？

参考回答：

因为唯一索引只能在最后拦住错误，锁可以在更前面减少无效

竞

争。两者不是重复，而是位置不同的两道防线。

6. 如果 Kafka 消费失败怎么办？

参考回答：

先看异常是否可恢复，可恢复就重试；不可恢复就进死信或补偿处

理。不能简单吞掉。

7. 如果 Redis 预扣成功但 MySQL 落库失败怎么办？

参考回答：

需要回滚 Redis 状态，至少把库存和一人一单集合恢复掉。否则会

造成用户不能再次下单，库存也不准。

8. 你怎么证明你考虑过这些异常场景？

参考回答：

我会从代码层和测试层两个角度证明：代码里有异常分类、回滚和

死信逻辑，测试里也补了事务边界和锁实现相关的回归验证。

这样

比单纯说“我考虑过”更可信。

追问链 2：缓存

1. 你讲一下商户缓存怎么做的。

参考回答：

商户详情采用两级缓存，先查本地 Caffeine，再查 Redis，最后才

查数据库。更新时先更新数据库，再删 Redis、本地缓存，并用

Pub/Sub 广播失效。

2. 为什么要加 Caffeine？

参考回答：

因为热点数据如果每次都走 Redis，Redis 压力也会比较大。

Caffeine 能进一步降低读取延迟和 Redis 压力。

3. 本地缓存一致性怎么保证？

参考回答：

主要靠更新时主动删除 + Pub/Sub 广播 + TTL 兜底。它不是强一

致，但能做到比较可接受的最终一致。

4. Pub/Sub 丢消息怎么办？

参考回答：

丢消息时某些节点可能短时间读旧值，但 TTL 到期后会自动恢复。

这个场景我接受最终一致，不追求强一致。

5. 逻辑过期和互斥锁你都用了，分别用在什么地方？

参考回答：

互斥锁更适合普通缓存重建；逻辑过期更适合热点数据快速返回旧

值并异步重建。它们关注点不一样。

6. 你的锁如果写得不严谨，会出什么 bug ？

参考回答：

最典型的的就是未持锁线程误删别人的锁，导致多个线程同时进入临界区。这个 bug 在并发场景下非常危险。

追问链 3：项目真实性

1. 这个项目里哪块是你自己改得最多的？

参考回答：

秒杀链路和缓存链路改得最多。因为这两块从业务正确性到性能优

化都能展开很多。

2. 你改之前是什么样？

参考回答：

改之前更多是“功能能跑”的状态，像消息链路、补偿、两级缓存和

锁安全这些工程细节没现在这么完整。

3. 你为什么想到这样改？

参考回答：

因为我在看链路时会先问自己：如果并发高了会怎样？如果消息丢失了怎么办？如果缓存不一致了怎么办？很多改造就是从这些追问里出来的。

4. 你改完之后怎么验证的？

参考回答：

我会通过代码自查、局部回归测试、接口验证和压测结果去看是否真的改善。重点不是“我改了”，而是“我改完之后有没有证据证明更稳了”。

5. 你踩过什么坑？

参考回答：

比较典型的是锁释放逻辑、消息消费异常分类，还有缓存和数据库双写一致性这些坑。很多坑不是代码跑不起来，而是并发和异常场景下才暴露。

6. 最后你会怎么把这段经历写进简历？

参考回答：

我会强调自己负责后端整体开发，并突出两三个高价值点，比如秒杀异步链路、两级缓存和稳定性增强，而不是只写“完成了某某接口开发”。

十四、自测建议

7. 先只回答每个专题前 3 题，保证能顺下来。

参考回答：

这一步的目标不是讲满，而是先把主线说顺。特别是项目总

览、缓

存和秒杀，必须先做到“别人一问，我能顺着讲下来”。

8. 再补“细节拷打专题”，看自己哪里会卡住。

参考回答：

细节拷打最能检验你是不是真的做过。比如 key 命名、返回值设

计、TTL、补偿逻辑这些，只要卡住就说明那块还得回头补。

9. 最后拿“模拟追问链”做完整演练。

参考回答：

模拟追问链的意义是让你从“会答单选题”提升到“能经得住追问”。校招

面试很常见的情况就是，第一问你答得还行，第二问第三问就乱了，所以这个阶段很重要。

如果你愿意，后面你可以按专题一段一段回答，我再继续扮演面试官追问你。

?????项目专题面试题-校招版.md

项目专题面试题-校招版.md

恢复说明：原始 Markdown 已从工作区删除，当前文件为重建入口页，不是原始逐字版本。

建议优先阅读 MASTER_INTERVIEW.md，再按主题阅读现存资料。

建议阅读顺序

1. [MASTER INTERVIEW.md](#)
 2. 相关主题现存文档
 3. 如需继续细化，可在此文件基础上继续补充整理
-

?????项目技术问答.md

项目技术问答.md

恢复说明：原始 Markdown 已从工作区删除，当前文件为重建入口页，不是原始逐字版本。

建议优先阅读 MASTER_INTERVIEW.md 与 README_ZH.md。

建议阅读顺序

1. [MASTER INTERVIEW.md](#)
 2. [README_ZH.md](#)
 3. 如需继续细化，可在此文件基础上继续补充整理
-

?????项目技术问答详解.md

项目技术问答详解.md

恢复说明：原始 Markdown 已从工作区删除，当前文件为重建入口页，不是原始逐字版本。

建议优先阅读 MASTER_INTERVIEW.md，再按主题阅读现存资料。

建议阅读顺序

1. [MASTER INTERVIEW.md](#)
 2. 相关主题现存文档
 3. 如需继续细化，可在此文件基础上继续补充整理
-

?????项目面试总复习文档.md

项目面试总复习文档.md

恢复说明：原始 Markdown 已从工作区删除，当前文件为重建入口页，不是原始逐字版本。

建议优先阅读 MASTER_INTERVIEW.md，再按主题阅读现存资料。

建议阅读顺序

1. [MASTER INTERVIEW.md](#)
 2. 相关主题现存文档
 3. 如需继续细化，可在此文件基础上继续补充整理
-

?????项目面试题库-详细版.md

hm-dianping 项目面试题库 (详细版)

版本：基于当前代码事实整理

时间：2026-05-05

用途：这份文档适合作为项目主面试文档，覆盖项目介绍、缓存、秒杀、Kafka、补偿、限流、测试与压测等高频问题。

特点：题目更多，答案更细，尽量贴近当前项目真实实现。

一、项目总览专题

1. 请你先用 1 分钟介绍一下你的 `hm-dianping` 项目。

这是一套基于 `Spring Boot + Redis + MySQL + Kafka + Nginx` 的本地生活服务项目，业务上参考了大众点评和小红书的一些常见场景，主要包括用户手机号验证码登录、商户查询、博客探店、关注 Feed、优惠券和秒杀下单。

这个项目和普通 CRUD 最大的区别在于，我后续做了很多工程化增强，而不只是把接口写通。比如：

1. 商户查询引入了 `Caffeine + Redis` 二级缓存，优化热点读请求
2. 秒杀链路做成了 `Redis Lua + Redis Stream + Kafka + MySQL` 的异步架构
3. 通过 `Redisson`、数据库唯一索引和 `stock > 0` 条件更新来保证幂等和防超卖
4. 补了 Kafka 手动 ACK、有限次重试、死信队列、失败任务表和 Redis 回滚
5. 对验证码、登录和秒杀入口加了轻量级限流
6. 还补了 API / UI 自动化测试和压测材料，用来验证改造效果

如果面试官继续展开，我最适合重点讲的是三块：

- 商户缓存

- 秒杀链路
- Kafka 可靠性增强与补偿

2. 这个项目相比普通后台管理系统，最大的技术亮点是什么？

我觉得最大的亮点是两点。

第一点是秒杀链路不是同步落库，而是做成了分层异步架构：Redis 前置资格校验，Kafka 异步削峰，MySQL 负责最终订单结果。这样不仅能抗住高并发，还能继续展开重试、死信、补偿和可恢复性。

第二点是缓存不只是“查 Redis”，而是做成了 Caffeine + Redis + MySQL 的多层结构，并且考虑了缓存穿透、击穿、多节点本地缓存失效和广播清理，这更像真实工程里的热点读优化。

3. 你在这个项目里主要做了哪些改造？

我可以分成 5 类来讲：

1. 缓存改造

给商户详情和商户类型列表加了 Caffeine + Redis 二级缓存，减少热点读直接打 Redis。

2. 秒杀链路改造

保留 Redis Lua 做资格校验，引入 Redis Stream 到 Kafka 的桥梁，再由 Kafka Consumer 异步落库。

3. 可靠性增强

补了 Kafka 生产端幂等、消费端手动 ACK、有限次重试、DLT 死信和失败任务表。

4. 一致性补偿

增加 Redis 回滚脚本、失败任务重试调度器和库存对账任务。

5. 稳定性与测试

补了轻量级限流、Kafka lag 监控、API / UI 自动化测试和压测

材料。

4. 如果让你只保留三个亮点写到简历上，你会写哪三个？

我会写：

1. 设计并实现 Redis Lua + Redis Stream + Kafka + MySQL 的秒杀异步链路
2. 为商户查询引入 Caffeine + Redis 二级缓存，并处理多节点本地缓存失效
3. 为 Kafka 消费链路补充手动 ACK、DLT、失败任务表和 Redis 回滚，增强可恢复性

二、技术栈与模块专题

5. 你的项目技术栈是什么？

后端技术栈：

- Java 8
- Spring Boot 2.3.12
- Spring MVC
- MyBatis-Plus
- Spring Data Redis
- Spring Kafka
- Redisson
- Hutool

中间件和存储：

- MySQL
- Redis
- Kafka

- Nginx

工程化能力：

- Caffeine
- Redis Lua
- Redis Stream
- 分布式锁
- 限流
- API / UI 自动化测试

6. 这个项目的核心业务模块有哪些？

主要有 5 块：

1. 用户模块
验证码登录、Token 会话、用户信息、签到
2. 商户模块
商户详情、商户类型、按类型分页、按名称查询、附近商户
3. 博客模块
博客发布、热门博客、点赞、点赞列表、关注 Feed
4. 关注模块
关注 / 取关、是否关注、共同关注
5. 优惠券与秒杀模块
普通优惠券、秒杀券、秒杀下单、异步落库、失败补偿

7. 如果面试官问你“这个项目最值得讲的模块是什么”，你怎么答？

我会优先讲秒杀模块。因为它同时覆盖了：

- Redis Lua
- Kafka
- Redis Stream
- Redisson 分布式锁

- MySQL 幂等与防超卖
- 死信、补偿、回滚

这一块技术密度最高，也最能体现我不是只会写接口，而是考虑过高并发和系统稳定性。

如果面试官更偏缓存，我再切到商户缓存模块。

三、登录与用户模块专题

8. 你的验证码登录流程是什么？

当前代码里，登录流程主要在 `UserServiceImpl`。

完整流程是：

1. 用户请求发送验证码
2. 后端校验手机号格式
3. 生成 6 位验证码
4. 把验证码写入 Redis，key 是 `login:code:{phone}`，TTL 是 2 分钟
5. 用户携带手机号和验证码发起登录
6. 服务端去 Redis 取验证码并校验
7. 校验通过后，根据手机号查询用户
8. 如果用户不存在，就自动创建用户
9. 生成 Token
10. 把用户信息转成 Hash 写入 Redis，key 是 `login:token:{token}`
11. 后续请求走拦截器做登录校验和 Token 刷新

9. 为什么验证码和登录态都适合放 Redis？

因为这两类数据都属于“临时态、高频读写、需要过期”的数据。

验证码适合 Redis 的原因：

- 写入快
- 读取快
- 天然支持 TTL
- 过期后自动清理

登录态适合 Redis 的原因：

- 多实例服务可以共享会话
- 比 Session 更适合分布式部署
- 可以灵活设置过期时间

10. 你的项目里 Token 是怎么刷新的？

这个逻辑在 `RefreshTokenInterceptor` 里。

拦截器会读取请求头里的 `authorization`，如果能从 Redis 里查到会话数据，就把当前用户放到 `ThreadLocal`，同时刷新 Redis 中该 Token 的 TTL。

所以项目里做的是一种“滑动续期”思路，而不是用户每次都重新登录。

11. 你的签到为什么适合用 Bitmap？

因为签到本质上是一个“每天是否签到”的布尔值问题。

如果直接用数据库逐条写记录，也能做，但成本更高。

用 Redis Bitmap 的好处是：

- 存储紧凑
- 写入是按位写，性能好
- 统计连续签到时可以一次读出多位再计算

在当前项目中：

- key 格式是 `sign:{userId}:yyyyMM`
- 每天签到就是把当月对应的 bit 位置为 1
- 连续签到统计用 `BITFIELD` 读出来再做按位计算

四、缓存专题

12. 你项目里的缓存架构是什么？

项目里商户详情和商户类型列表不是单层 Redis 缓存，而是三层查询路径：

- L1: Caffeine 本地缓存
- L2: Redis 分布式缓存
- L3: MySQL

也就是先查本地缓存，本地未命中再查 Redis，Redis 也未命中再查数据库，然后再回填缓存。

13. 为什么要引入 Caffeine，本来有 Redis 不就够了吗？

Redis 已经很快了，但它仍然是一个网络调用。

对于热点商户详情和商户类型列表这种高频读接口，如果每次都走 Redis，还是有网络往返和 Redis 侧压力。

引入 Caffeine 的价值是：

1. 命中本地缓存时不用网络访问
2. 延迟更低
3. 降低 Redis QPS
4. 对热点数据收益明显

所以 Caffeine 更像是 Redis 前面的一层“就地热点加速层”。

14. 你哪些数据用了本地缓存？

当前代码里主要是两类：

1. 商户详情

本地缓存 Bean 是 `shopLocalCache`

2. 商户类型列表

本地缓存 Bean 是 `shopTypeLocalCache`

这两类数据都有一个共同点：

- 访问频率高
- 变化相对少
- 一致性要求不是强一致到毫秒级

15. 你的缓存是怎么处理穿透的？

通过缓存空值解决。

思路是：

1. 查 Redis 未命中
2. 查 MySQL 也没数据
3. 不直接返回，而是把空字符串写入 Redis
4. 设置一个较短 TTL

这样下次相同的无效请求进来时，就不会继续打数据库。

16. 你的缓存是怎么处理击穿的？

项目里保留了两种常见方案：

1. 互斥锁方案
2. 逻辑过期方案

互斥锁方案

当缓存未命中时，只允许一个线程去查数据库并重建缓存，其它线程等待后重试。

逻辑过期方案

Redis key 本身不过期，而是在 value 里写一个逻辑过期时间。
过期后当前请求先返回旧值，再异步重建缓存。

17. 你项目里为什么两个方案都保留了？

因为它们适合的场景不完全一样。

互斥锁更偏向：

- 保证一致性更严一些
- 允许请求等待一下

逻辑过期更偏向：

- 不阻塞当前请求
- 允许短时间内返回旧数据

所以在一个项目里保留两种通用能力是合理的，后续可以按业务特点选用。

18. 你是怎么处理缓存雪崩的？

项目里主要用了两点：

1. TTL 加随机抖动
2. 热点数据可提前预热

在 `CacheClient` 里，写缓存时会在基础 TTL 上增加一个随机值，这样不同 key 的过期时间不会完全一致，能降低同一时刻大量 key 一起失效的风险。

19. 本地缓存一致性怎么解决？

这也是二级缓存里最容易被问的点。

因为每个服务实例都有自己的 Caffeine，本地缓存不共享，所以更新数据时不能只删 Redis。

当前项目做法是：

1. 更新数据库
2. 删除 Redis 缓存
3. 删除当前节点本地缓存
4. 通过 Redis Pub/Sub 发布失效消息
5. 其它节点收到消息后，也删除自己的本地缓存

对应代码：

- `CacheInvalidationPublisher`
- `CacheInvalidationListener`
- `CacheInvalidationConfig`

20. 如果 Pub/Sub 丢消息怎么办？

这里可以答得稍微工程化一点。

Pub/Sub 的一致性更偏“尽量及时同步”，不是绝对强一致。

如果广播消息偶发丢失，项目里还有本地缓存 TTL 作为兜底，旧缓存不会永久存在。

所以这个方案属于：

- 日常读性能优先
- 一致性做最终收敛

21. 商户详情查询链路你怎么讲？

可以按步骤讲：

1. 先查本地缓存
2. 本地未命中查 Redis
3. Redis 未命中查 MySQL
4. 查到后回填 Redis 和本地缓存
5. 查不到则写空值缓存

这个回答既清楚又贴代码实现。

五、商户 GEO 专题

22. 你项目里的附近商户查询是怎么做的？

当前代码在 `ShopServiceImpl.queryShopByType()`。

逻辑是：

1. 如果前端没有传坐标，直接按类型分页查数据库
2. 如果传了坐标，就优先查 Redis GEO
3. Redis 中 key 是 `shop:geo:{typeId}`
4. 查询半径 5 公里内商户，并带距离
5. Redis 返回的是商户 id 和 distance
6. 再按 id 回表查商户完整信息
7. 最后把距离写回到结果里

23. 为什么 GEO 查询后还要回数据库？

因为 GEO 更适合做“位置索引”和“附近范围筛选”，但它不会直接存完整业务详情。

最终页面展示需要商户名称、图片、评分等完整字段，所以还得根据商户 id 再查一次业务数据。

六、博客、点赞、关注、Feed 专题

24. 你为什么用 Redis ZSet 做点赞？

因为点赞这个场景既需要去重，又可能需要按时间排序查最近点赞用户，所以 ZSet 很合适。

在项目里的设计大概是：

- key: `blog:liked:{blogId}`
- member: `userId`
- score: 点赞时间戳

这样可以支持：

- 判断某用户是否点赞过
- 点赞 / 取消点赞
- 查询最近点赞用户列表

25. 为什么不是 Set?

如果只是做“用户是否点过赞”的去重，Set 也能做。

但如果还希望按点赞时间排序展示最近点赞用户，Set 就不够了，ZSet 更合适。

26. 关注关系为什么适合 Set?

关注关系天然就是集合关系。

在当前项目里：

- `follows:{userId}` 存当前用户关注的人

它适合的原因是：

- 判断是否关注: `SISMEMBER`
- 查询共同关注: `SINTER`
- 关注 / 取关: `SADD` / `SREM`

27. Feed 流是怎么做的?

当前项目里的思路是“推模式”。

用户发完博客后：

1. 查出所有粉丝
2. 把博客 id 推送到每个粉丝的收件箱

3. 收件箱使用 ZSet 保存
4. score 用时间戳
5. 查询 Feed 时做滚动分页

这样能比较自然地支持“我关注的人发了什么”这个场景。

28. 你的 Feed 为什么用滚动分页而不是普通页码分页？

因为 Feed 流是按时间持续插入的新数据流，普通页码分页容易在新数据插入时出现重复或漏数据。

滚动分页用 `max + offset` 这种方式更适合时间流场景。

七、秒杀链路专题

29. 你先完整讲一下你项目里的秒杀链路。

这是我项目里最核心的一条链路。

完整流程是：

1. 用户请求秒杀接口 `/voucher-order/seckill/{id}`
2. 服务端获取用户 ID 和生成全局订单 ID
3. 调用 `seckill.lua`
4. Lua 在 Redis 里原子完成：
 - 校验库存
 - 校验一人一单
 - 预扣库存
 - 记录用户下单标记
 - `XADD stream.orders`
5. 如果 Lua 成功，接口立即返回“受理成功”
6. 后台 `RedisStreamToKafkaRelay` 消费 `stream.orders`

7. Relay 把消息转发到 Kafka

8. `VoucherOrderConsumer` 异步消费 Kafka 消息

9. Consumer 调用

```
voucherOrderServiceImpl.handleVoucherOrderFromMQ()
```

10. 落库前：

- 通过 Redisson 按用户维度加锁
- 再次校验是否重复下单
- 扣减数据库库存，条件是 `stock > 0`
- 保存订单

整体上是：

```
Redis 前置原子校验 + Kafka 异步削峰 + MySQL 最终落库
```

30. 为什么秒杀不能像普通下单一样直接查库扣库存？

因为秒杀的并发量远高于普通下单。

如果让大量请求直接打数据库，会出现：

- 数据库连接和锁竞争压力非常大
- 响应时间变长
- 库存热点行竞争严重
- 容易出现整体吞吐下降

所以秒杀更适合把资格校验前移到 Redis，用内存型组件先挡掉大量无效请求。

31. 你为什么把资格校验放在 Lua 里？

因为库存判断、一人一单判断、扣减库存和记录用户标记，这几个动作必须是原子性的。

如果拆成多条 Redis 命令，在并发下中间可能被别的请求插队，就会导致：

- 超卖
- 重复下单

Lua 的价值就是把多步操作封装成 Redis 内部的一次原子执行。

32. 你的 Lua 脚本具体做了哪几件事？

当前 `seckill.lua` 做了 5 件事：

1. 判断库存 key 是否存在且库存大于 0
2. 判断当前用户是否已下单
3. 预扣库存
4. 记录当前用户已下单
5. 把订单消息写入 `stream.orders`

这 5 步在 Redis 内部原子执行。

33. 你的秒杀接口为什么能很快返回？

因为接口返回的不是“订单已经成功落库”，而是“请求已经通过 Redis 校验并进入异步链路”。

真正的订单落库在后面的 Kafka Consumer 中异步完成。

也正因为如此，接口线程不需要等待数据库事务完成，所以高并发下响应更快。

34. 这种“先返回成功、后异步落库”的设计有什么风险？

有两个典型风险：

1. 用户已经看到成功，但订单最终没落库成功
2. Redis 预扣成功，但 MySQL 失败，造成临时不一致

也正因为有这些风险，后面我又补了 Kafka 重试、DLT、失败任务表和 Redis 回滚来兜底。

35. 什么叫一人一单？你在哪几层做的保证？

一人一单就是同一个用户对同一张秒杀券只能成功下单一次。

我项目里是三层保证：

1. Redis Lua：前置校验用户是否已下单
2. Consumer 落库前：再查一次订单是否已存在
3. 数据库：`user_id + voucher_id` 唯一索引兜底

36. 为什么 Redis 做过一人一单了，数据库还要再兜底？

因为 Redis 的校验属于前置快速拦截，但最终一致性还是以数据库为准。

Redis 预扣和 MySQL 落库之间存在时间差，极端情况下：

- 可能消息重复消费
- 可能并发边界下重复尝试落库

所以数据库唯一索引是最终兜底，不能省。

37. 你的数据库扣库存 SQL 为什么要带 `stock > 0`？

这个条件是乐观控制的一部分。

即使多个请求同时进入到数据库更新，只要 SQL 写成：

```
update ... set stock = stock - 1 where voucher_id = ?  
and stock > 0
```

那库存已经为 0 的情况下，后面的更新就不会再成功，从数据库层防止出现负库存。

38. 如果两个请求同时走到数据库扣库存，会不会超卖？

不会直接因为这条 SQL 超卖。

因为 `stock > 0` 限定了只有库存还大于 0 时才允许成功更新。

也就是说，数据库层本身就具备最后一道库存兜底能力。

39. 为什么你还要再加 Redisson 分布式锁？

因为唯一索引和 `stock > 0` 虽然能兜底，但如果完全依赖数据库兜底，会让大量冲突请求直接打到数据库，数据库压力会更大。

Redisson 的意义是：

- 提前在应用层限制同一用户并发创建订单
- 减少数据库冲突
- 让链路更平稳

40. 有唯一索引了，为什么还要锁？

一句话回答：

唯一索引负责最终兜底，分布式锁负责提前削减冲突请求，它们不冲突，作用层次不同。

八、Kafka 与消息队列专题

41. 你项目里已经用了 Redis Stream，为什么还要再引入 Kafka？

这个问题一定要讲清楚职责边界。

我不是把 Redis Stream 替换成 Kafka，而是把链路升级成：

Redis 前置原子校验 + Redis Stream 本地承接 + Kafka 异步削峰和恢复

Redis Stream 负责：

- 秒杀入口本地落消息
- 靠近 Redis 资格校验链路

Kafka 负责：

- 更强的异步削峰能力
- 消息积压后的恢复能力
- 消费者组扩展能力
- 更成熟的重试和死信机制

42. Redis Stream 和 Kafka 在你的项目里分别承担什么职责？

Redis Stream：

- 入口通过 Lua 校验后，把受理成功的秒杀消息先就地写入 Stream

Kafka：

- 做异步传输
- 做削峰填谷
- 做重试与恢复
- 做死信转移

MySQL：

- 做最终订单和库存结果

43. 你项目里的 Relay 中转层是干什么的？

`RedisStreamToKafkaRelay` 是一个桥接组件。

它负责：

1. 消费 Redis Stream
2. 转换消息结构
3. 转发到 Kafka
4. 只有 Kafka 发送成功后，才 ACK Redis Stream

这个设计的价值在于：

- 入口和 Kafka 解耦
- 降低 Stream 到 Kafka 之间的消息丢失风险
- 支持 pending-list 恢复

44. 为什么不直接在业务代码里把消息发到 Kafka?

因为当前业务入口的核心不是“直接发 Kafka”，而是先走 Redis Lua 原子校验。

既然入口资格校验已经发生在 Redis 里，那么先把消息写入 Redis Stream 更自然，再由 Relay 统一转发到 Kafka，可以把入口线程和 Kafka 彻底解耦。

45. 你的 Kafka Topic 是什么？消息体有哪些关键字段？

默认 Topic 是：

- `seckill-order`

消息体里有这些关键字段：

- `orderId`
- `userId`
- `voucherId`

- `traceId`
- `createTime`

这些字段足够支持：

- 订单创建
- 幂等控制
- 日志追踪
- 补偿定位

46. 你为什么给消息加 `traceId`？

因为秒杀链路是异步的，而且涉及 Redis、Relay、Kafka、Consumer、MySQL 多层组件。

有了 `traceId` 后，可以在日志里把一条订单消息完整串起来，方便排查问题。

47. 消费者为什么要手动 ACK，而不是自动提交？

因为秒杀订单这类消息不能在业务还没处理成功时就提交 offset。如果自动提交或过早 ACK，会出现：

- Kafka 认为消息已经消费成功
- 实际上订单还没落库成功
- 最终导致消息丢失

所以我改成了手动 ACK，只有业务成功后才提交。

一句话就是：

先处理业务，成功后再 ACK；失败就不 ACK，让 kafka 重试。

48. 什么叫消息重复消费？你项目里怎么做幂等？

重复消费就是同一条 Kafka 消息因为重试、服务重启或消费确认时机等原因，被消费者重复处理多次。

我项目里不是试图完全避免重复消费，而是按“至少一次”语义设计，然后做业务幂等。

幂等主要靠三层：

1. Redis 一人一单前置校验
2. Consumer 落库前再次校验
3. 数据库唯一索引兜底

49. 什么情况下 Kafka 消费失败要重试？什么情况下不该重试？

我区分两类：

可恢复异常

例如：

- 临时网络抖动
- 瞬时数据库不可用
- 下游依赖短暂异常

这类适合重试。

不可恢复异常

例如：

- 消息本身有问题
- 业务参数错误
- 明确属于不可恢复的处理异常

这类不适合无限重试，应直接回滚 Redis 并转人工处理或落失败任务表。

50. 你的死信队列是用来解决什么问题的？

当消息重试 3 次仍失败时，如果继续无限重试，会：

- 阻塞正常消息
- 让消费堆积越来越严重
- 无法把顽固异常隔离出来

所以我引入死信队列，把重试超限消息转移到：

- `seckill-order-dlt`

再由死信消费者落失败任务表并触发 Redis 回滚。

51. 死信消息到来之后，你项目里后续怎么处理？

流程是：

1. `VoucherOrderDeadLetterConsumer` 接收死信
2. 反序列化消息
3. 调用失败任务服务
4. 保存失败任务到数据库
5. 同时执行 Redis 回滚
6. 最终把状态标记成：
 - `ROLLBACK_DONE`
 - 或 `MANUAL_HANDLE_REQUIRED`

52. 如果 Kafka 暂时不可用，你的链路会发生什么？

这要分阶段看。

入口阶段：

- Lua 和 Redis Stream 还可能受理成功

中转阶段：

- Relay 从 Stream 读到消息后，发送 Kafka 失败
- 因为没有发送成功，所以不会 ACK Stream
- 消息会留在 Stream / pending-list 里等待恢复

所以链路的恢复点在于：

- Kafka 恢复后，Relay 可以继续转发并 ACK

53. 如果 Relay 成功从 Redis Stream 读到消息，但发 Kafka 失败了，会怎么处理？

因为我的设计是“Kafka 发送成功后再 ACK Redis Stream”，所以如果 Kafka 发送失败：

- 这条 Stream 消息不会被确认
- 它会留在 pending-list 里
- 后续可以通过恢复流程重新处理

这就是 Relay 的关键设计点。

54. 为什么消费者分组对秒杀场景重要？

因为秒杀订单消息量可能比较大，消费者组可以支持后续水平扩展。

如果 Topic 有多个分区，就可以通过多个消费者实例并行消费不同分区，提高吞吐能力。

55. 你的 Kafka key 为什么考虑用 `userId`？

因为同一用户的多条消息如果尽量落在同一分区，就更有利于保持同一用户消息的相对有序性，也更利于从业务维度理解分区设计。

九、分布式锁与一致性专题

56. 你项目里哪些地方用到了分布式锁？

当前最核心的使用点在秒杀订单创建环节。

在 `VoucherOrderServiceImpl.createVoucherOrder()` 里，会按：

- `lock:order:{userId}`

获取 Redisson 锁，避免同一用户并发创建秒杀订单。

57. 你为什么选 Redisson，而不是自己手写 `setnx` 锁？

因为 Redisson 是成熟组件，功能更完整，至少包括：

- 可重入
- 看门狗机制
- 更完善的锁抽象

自己手写 `setnx` 锁虽然也能做一个简单版，但边界和可靠性更难兜。

58. 如果让你解释 Redisson 的价值，一句话怎么说？

我用 Redisson 不是为了替代数据库兜底，而是为了在应用层提前减少同一用户并发下单带来的冲突和数据库压力。

59. 你项目里哪些一致性是最终一致性，哪些不追求强一致？

可以这样答：

- 秒杀订单和库存最终结果：以 MySQL 为准，属于最终一致性治理
- 本地缓存和 Redis 缓存之间：通过广播清理和 TTL 收敛，属于最终一致性
- Redis 预扣库存和 MySQL 落库：通过重试、死信、回滚和对账来保证最终一致性

很多场景没有必要追求强一致，否则成本太高。

十、限流与稳定性专题

60. 你项目里为什么要做接口限流？

虽然项目本身已经有秒杀并发保护，但像：

- `/user/code`
- `/user/login`
- `/voucher-order/seckill/{id}`

这些入口接口容易被高频请求或者恶意刷接口打爆，所以我补了一套轻量级限流，保护系统入口。

61. 你项目里哪些接口最需要限流？为什么？

第一批最需要的有三个：

1. `/user/code`
因为验证码接口最容易被反复刷，浪费资源
2. `/user/login`
防止高频试探和撞库式尝试

3. `/voucher-order/seckill/{id}`

秒杀入口天然是高并发热点接口

62. 你的限流是基于什么实现的？

当前方案是：

- 自定义注解 `@RateLimit`
- Spring AOP 统一拦截
- Redis 固定窗口计数
- 验证码接口增加冷却时间控制

对应代码：

- `RateLimit`
- `RateLimitAspect`
- `RateLimitServiceImpl`
- `RateLimitKeyResolver`

63. 自定义注解 `@RateLimit` 的作用是什么？

它的作用是把限流规则声明在接口上，而不是把限流代码散落在业务逻辑里。

这样做的好处是：

- 低侵入
- 规则清晰
- 便于扩展更多接口

64. 你是怎么区分“按手机号限流”和“按用户限流”的？

这个逻辑在 `RateLimitKeyResolver` 里。

它会根据 `LimitType` 解析不同维度的 key 后缀：

- PHONE
- USER
- IP
- USER_AND_PATH
- IP_AND_PATH

所以限流的关键不是只会“加计数器”，而是要能按业务维度生成合理的限流 key。

65. 你现在的限流算法更接近固定窗口、滑动窗口还是令牌桶？

当前实现更接近：

- 固定窗口限流

因为现在是：

- 请求来了 INCR
- 第一次请求时 EXPIRE
- 超过阈值就拒绝

此外验证码接口还叠加了冷却时间控制。

66. 固定窗口限流有什么缺点？

最大缺点是边界效应。

比如某个窗口最后 1 秒来了很多请求，下一个窗口刚开始又来很多请求，短时间内可能通过过量流量。

67. 为什么你项目现在没直接用滑动窗口？

因为第一版优先考虑：

- 实现简单
- 快速落地

- 低侵入
- 对当前几个高风险接口已经足够

如果后续要继续升级，更安全敏感的接口，比如登录和验证码，可以进一步升级成滑动窗口。

68. 如果面试官问你“后面怎么升级限流”，你怎么答？

我会这样答：

1. 先把规则外置到配置层
2. 给高风险接口增加 IP 维度限流
3. 登录和验证码可升级成滑动窗口
4. 更大规模时再考虑网关层限流或 Sentinel

十一、失败任务、补偿与治理专题

69. 为什么要引入失败任务表？

因为死信消息如果只是打个日志再丢掉，业务上就等于没有后手。失败任务表的价值是：

- 记录失败消息
- 标记失败状态
- 支持自动重试
- 支持人工处理
- 支持后续分析

70. 你的失败任务表状态有哪些？

当前代码里核心状态包括：

- `RETRY_PENDING`

- `RETRYING`
- `DONE`
- `ROLLBACK_DONE`
- `MANUAL_HANDLE_REQUIRED`

这些状态已经可以支持一套比较完整的失败治理流程。

71. 自动重试是怎么做的？

有一个定时任务：

- `VoucherOrderFailTaskRetryScheduler`

它会定期扫描 `RETRY_PENDING` 的任务，再重新发送回 Kafka。

如果超过最大重试次数还失败，就转成：

- `MANUAL_HANDLE_REQUIRED`

72. 你的 Redis 回滚回滚了什么？

回滚的是：

1. Redis 预扣库存
2. 一人一单标记

也就是说不仅要把库存加回去，还要把用户已下单标记清掉，否则用户即使订单没成功，也会被错误地认为已经买过。

73. Redis 回滚为什么要做幂等？

因为重试、死信、人工处理等场景都可能重复触发回滚。

如果回滚逻辑不是幂等的，就可能造成：

- 库存被重复加回
- 状态再次错乱

所以回滚脚本必须考虑重复执行安全。

74. 你的 Kafka lag 监控和库存对账有什么意义？

这是项目里比较像真实工程的地方。

Kafka lag 监控可以发现：

- 消费堆积
- 消费速度跟不上

库存对账可以发现：

- Redis 预扣库存
- MySQL 最终库存

之间是否长期不一致。

这两块说明我不是只把功能做通，还考虑了运行态治理。

十二、测试与压测专题

75. 你项目里除了单元测试，还有哪些测试方式？

当前项目里我补的是：

- API 自动化测试
- UI 自动化测试
- 秒杀链路回归测试
- 压测材料

76. 为什么你这个项目需要 API 自动化测试？

因为项目后续做了很多改造：

- 秒杀链路异步化
- 补偿
- 限流
- 登录和 Token 相关逻辑

这些改动如果没有回归测试，很容易改一块坏一块。
API 自动化可以快速验证关键接口还是否可用。

77. 为什么还要做 UI 自动化测试？

因为有些问题 API 通过了，不代表前端链路就真的通了。
比如：

- 未登录状态秒杀按钮点击后是不是正确跳转登录页
- 已登录状态下点击秒杀，前端是否能拿到正确业务反馈

所以 UI 自动化主要用来覆盖关键入口行为。

78. 你的测试现在有“数据”吗？可以实讲吗？

可以，能讲，而且最好实讲。

从当前代码和测试说明看：

API 用例

当前 `tests/api/test_hmdp_api_smoke.py` 里有 7 条 API 用例：

1. 商户类型列表冒烟
2. `/user/me` 未登录拦截
3. `/user/me` 登录后可访问
4. 查询秒杀券列表
5. 同一用户不能重复下单
6. 登出后 token 失效
7. 非法 token 秒杀被拒绝

UI 用例

当前 `tests/ui/cases` 里主要有 3 条 UI 用例：

1. 登录页冒烟
2. 未登录点击秒杀跳转登录页
3. 已登录点击秒杀出现业务反馈

另外还有 1 条真实 UI 登录动作模板用例，但默认通过环境变量控制是否启用。

当前稳定回归口径

按照现有测试文档里的说法，当前稳定基线是：

- `10 passed, 1 skipped`

这个口径是能讲的，而且和当前代码结构是能对上的：

- 7 条 API
- 3 条稳定 UI
- 1 条真实 UI 登录模板默认 skip

79. 你为什么说不是“只会写 10 条”，而是有计划地扩？

因为当前这 10 条不是随便拼出来的，而是先覆盖高价值主路径：

- 登录鉴权正反向
- 秒杀券查询
- 一人一单
- 登出失效
- UI 未登录拦截
- UI 已登录秒杀反馈

这是一个典型的“稳定冒烟基线”，先把高价值回归链路跑稳，再按优先级扩展到 30+ 条，会比一开始堆很多脆弱脚本更工程化。

80. 你的自动化测试平台技术方案是什么？

主要是：

- `PyTest + Selenium`
- `Page Object + fixture`
- `Allure`
- `pytest-rerunfailures`

还有两个特别适合讲的点：

1. 自动鉴权

通过发验证码 -> 读 Redis -> 登录拿 token, 做到 API / UI 共用鉴权能力

2. 一键脚本

`scripts/run-tests.ps1` 先做端口预检查, 再按模式执行 API / UI / seckill 测试

81. 你的测试为什么要接 Redis?

因为项目登录依赖短信验证码, 而验证码存 Redis。

如果不接 Redis, 就要靠人工输入验证码, 自动化程度很差。

接 Redis 后, 可以做到：

- 发码
- 读验证码
- 自动登录

整个回归链路就能无人值守执行。

82. 你的测试脚本里做了哪些稳定性增强?

我做了几类：

1. 端口预检查

先检查 8080、8081、6379, 必要时检查 8082

2. 失败重试

`pytest.ini` 里默认开启一次失败重试

3. UI 驱动兜底

Driver 初始化做多级回退

4. 失败截图和页面源码

用于排查 UI 失败原因

83. 压测这部分你可以怎么讲？

可以结合 `scripts/LOADTEST.md` 讲：

项目不仅有回归测试，还有压测脚本和压测说明，主要关注的是秒杀链路。

压测时会重点区分：

单 token 压测

更偏向压接口和验证 Lua 的重复下单拦截，不是真实多用户抢库存

多 token 压测

更接近真实多用户秒杀场景

这点很重要，因为它说明你知道“什么压测结果代表什么含义”，不是只会跑工具。

84. 秒杀压测时你最关注哪些指标？

重点会看：

- 总请求数
- 墙钟时间
- 近似 QPS
- 成功率
- 业务失败分布
- 库存是否超卖
- Kafka 消费堆积

- 数据库和 Redis 压力

85. 如果面试官问“你怎么证明自己真的做过测试平台”，你怎么答？

我会从三个层面答：

1. 代码层

我能讲清楚 fixture、Page Object、自动鉴权、Allure、一键脚本是怎么做的

2. 用例层

我能明确说出当前有哪些 API 用例和 UI 用例

3. 结果层

我能说出当前稳定基线是 10 passed, 1 skipped，并解释这个结果为什么和当前项目结构一致

十三、项目真实性与追问专题

86. 这个项目里哪块是你自己改得最多的？

我改得最多的是秒杀链路和周边工程化增强。

因为这块不是只改一个类，而是涉及：

- Lua
- Redis Stream
- Kafka Producer / Consumer
- Redisson
- 失败任务表
- Redis 回滚
- 监控和对账

87. 你改之前是什么样？

可以这样讲：

一开始更偏基础版：

- Redis Lua 前置校验
- 异步化程度和可靠性治理还不够完整

后面我把它逐步增强到了：

- Redis Stream -> Kafka 桥接
- 手动 ACK
- DLT
- 失败任务表
- 回滚与补偿
- lag 监控与库存对账

88. 你为什么想到这样改？

因为我发现“能跑”和“能讲成工程化系统”之间还有很大差距。

如果只有最基础的异步链路，面试官很容易继续问：

- 消费失败怎么办
- 重复消费怎么办
- Redis 预扣成功但 MySQL 失败怎么办
- Kafka 堆积怎么观测

所以我后续做的改造，其实是在把这些追问逐步落地成真实代码。

89. 你改完之后怎么验证的？

我不是只看代码跑不跑，而是通过几种方式验证：

1. API 自动化
2. UI 自动化
3. 压测脚本
4. 日志观察
5. Kafka lag 监控

6. 秒杀库存对账

所以验证不是单点的，而是多角度的。

90. 你踩过什么坑？

比较适合讲的坑有：

1. 配置和环境问题导致链路起不来
2. Redis / Kafka / MySQL 三方一致性问题
3. UI 自动化环境稳定性问题
4. 限流虽然简单接入了，但窗口边界问题后续还要继续优化

这种回答更真实，也更像你真做过项目。

91. 这个项目里哪块最像真实公司项目？

我觉得最像的是秒杀链路增强后的这一版，因为它已经不只是一个“秒杀 Demo”，而是开始考虑：

- 异步削峰
- 幂等
- 重试
- 死信
- 补偿
- 回滚
- lag 监控
- 库存对账

这些都非常接近真实高并发业务系统会考虑的问题。

92. 哪块最不像真实公司项目？

我会诚实说：

1. 限流规则还主要写在代码里，动态化不够
2. 监控还是偏日志和定时任务，还没接完整指标平台

3. 自动化测试覆盖到主路径了，但还不是非常大规模的企业级测试体系
 4. 配置管理虽然做了占位符和 local profile，但还没有做到完整多环境治理
-

十四、细节拷打专题

93. 你的验证码 Redis key 是什么？TTL 是多少？

- key: `login:code:{phone}`
- TTL: 2 分钟

94. 你的登录会话 Redis key 是什么？

- key: `login:token:{token}`

95. 你的秒杀库存 key 前缀是什么？

- `seckill:stock:`

96. 你的博客点赞 key 前缀是什么？

- `blog:liked:`

97. 你的 Feed key 前缀是什么？

- `feed:`

98. 你的 GEO key 前缀是什么？

- `shop:geo:`

99. 你的签到 key 前缀是什么？

- `sign:`

100. 你的 Kafka Topic 默认名是什么?

- `seckill-order`

101. 你的 Kafka Consumer 组名默认是什么?

- `seckill-order-g1`

102. 你的 Redis Stream 名字是什么?

- `stream.orders`

103. 你的本地缓存默认 TTL 大概是多少?

按当前 `RedisConstants` :

- 商户本地缓存 TTL: 5 分钟
- 商户类型本地缓存 TTL: 10 分钟

104. Redis 缓存的商户 TTL 大概是多少?

- 商户 Redis TTL: 30 分钟
- 商户类型 Redis TTL: 30 分钟

105. 你的秒杀 Consumer 失败重试几次?

根据 `KafkaConfig`, 当前配置是:

- 间隔 1 秒
- 重试 3 次

之后超限进入 DLT。

106. 你的失败任务自动重试间隔是多少？

按当前配置：

- 默认 15000ms，也就是 15 秒

107. 你的失败任务默认最大重试次数是多少？

- 5 次

十五、最推荐直接背的答案

108. 最推荐背的项目介绍

我的 hm-dianping 是一个基于 Spring Boot、Redis、MySQL 和 Kafka 的本地生活服务项目，业务上包括登录、商户查询、博客探店、关注 Feed 和优惠券秒杀。我在这个项目里重点补了两块，一块是商户查询的 Caffeine + Redis 二级缓存，另一块是秒杀链路的 Redis Lua + Kafka 异步削峰，并在此基础上继续增强了手动 ACK、死信、失败任务表、Redis 回滚、限流和自动化测试。

109. 最推荐背的秒杀链路答案

秒杀入口先通过 Redis Lua 原子校验库存和一人一单，校验通过后把请求写入 Redis Stream，再由 Relay 转发到 Kafka，最后 Consumer 异步创建订单并扣减 MySQL 库存。这样做的好处是入口响应快、能削峰、能解耦，同时还能继续增强重试、死信、补偿和对账能力。

110. 最推荐背的缓存答案

商户详情和商户类型列表我做成了 L1 Caffeine + L2 Redis + L3 MySQL 的二级缓存方案，同时处理了缓存穿透、击穿、雪崩和多节点本地缓存失效问题。

111. 最推荐背的限流答案

我给验证码、登录和秒杀入口补了一套基于 Spring AOP + Redis 的轻量级限流，当前采用固定窗口计数和验证码冷却时间控制，因为它实现简单、低侵入、适合快速落地。

112. 最推荐背的测试答案

我在项目里搭了一套 API + UI 一体化自动化测试能力，基于 PyTest + Selenium + Allure，并做了自动发码、Redis 读验证码、自动登录拿 token 的鉴权链路。当前稳定基线按现有测试文档是 10 passed, 1 skipped，覆盖了登录鉴权、秒杀、重复下单拦截、登出失效以及 UI 秒杀入口这些高价值场景。

十六、复习建议

如果时间不多，建议按这个顺序背：

1. 先背第 1、2、4 节
2. 再背第 7、8、9 节秒杀、Kafka、幂等和补偿
3. 再背第 4 节缓存
4. 再补第 10 节限流
5. 最后看第 12 节测试和第 14 节细节拷打

如果你后面还想继续压缩，我可以再从这份文档里继续给你拆两份：

- 项目面试题库-速记版.md
 - 项目面试题库-追问版.md
-